CHAPTER 12

Physical Storage Systems

In preceding chapters, we have emphasized the higher-level models of a database. For example, at the *conceptual* or *logical* level, we viewed the database, in the relational model, as a collection of tables. Indeed, the logical model of the database is the correct level for database *users* to focus on. This is because the goal of a database system is to simplify and facilitate access to data; users of the system should not be burdened unnecessarily with the physical details of the implementation of the system.

In this chapter, however, as well as in Chapter 13, Chapter 14, Chapter 15, and Chapter 16, we probe below the higher levels as we describe various methods for implementing the data models and languages presented in preceding chapters. We start with characteristics of the underlying storage media, with a particular focus on magnetic disks and flash-based solid-state disks, and then discuss how to create highly reliable storage structures by using multiple storage devices.

12.1 Overview of Physical Storage Media

Several types of data storage exist in most computer systems. These storage media are classified by the speed with which data can be accessed, by the cost per unit of data to buy the medium, and by the medium's reliability. Among the media typically available are these:

- **Cache**. The cache is the fastest and most costly form of storage. Cache memory is relatively small; its use is managed by the computer system hardware. We shall not be concerned about managing cache storage in the database system. It is, however, worth noting that database implementors do pay attention to cache effects when designing query processing data structures and algorithms, and we shall return to this issue in later chapters.
- **Main memory**. The storage medium used for data that are available to be operated on is main memory. The general-purpose machine instructions operate on main memory. Main memory may contain tens of gigabytes of data on a personal com-

puter, and even hundreds to thousands of gigabytes of data in large server systems. It is generally too small (or too expensive) for storing the entire database for very large databases, but many enterprise databases can fit in main memory. However, the contents of main memory are lost in the event of a power failure or system crash; main memory is therefore said to be **volatile**.

- **Flash memory**. Flash memory differs from main memory in that stored data are retained even if power is turned off (or fails)—that is, it is **non-volatile**. Flash memory has a lower cost per byte than main memory, but a higher cost per byte than magnetic disks.

  Flash memory is widely used for data storage in devices such as cameras and cell phones. Flash memory is also used for storing data in "USB flash drives," also known as "pen drives," which can be plugged into the *Universal Serial Bus* (USB) slots of computing devices.

  Flash memory is also increasingly used as a replacement for magnetic disks in personal computers as well as in servers. A **solid-state drive** (**SSD**) uses flash memory internally to store data but provides an interface similar to a magnetic disk, allowing data to be stored or retrieved in units of a block; such an interface is called a *block-oriented interface*. Block sizes typically range from 512 bytes to 8-kilobytes. As of 2018, a 1-terabyte SSD costs around $250. We provide more details about flash memory in Section 12.4.

- **Magnetic-disk storage**. The primary medium for the long-term online storage of data is the magnetic disk drive, which is also referred to as the **hard disk drive** (**HDD**). Magnetic disk, like flash memory, is non-volatile: that is, magnetic disk storage survives power failures and system crashes. Disks may sometimes fail and destroy data, but such failures are quite rare compared to system crashes or power failures.

  To access data stored on magnetic disk, the system must first move the data from disk to main memory, from where they can be accessed. After the system has performed the designated operations, the data that have been modified must be written to disk.

  Disk capacities have grown steadily over the years. As of 2018, the size of magnetic disks ranges from 500 gigabytes to 14 terabytes, and a 1-terabyte disk costs about $50, while an 8-terabyte disk around $150. Although significantly cheaper than SSDs, magnetic disks provide lower performance in terms of number of data access operations that they can support per second. We provide further details about magnetic disks in Section 12.3.

- **Optical storage**. The *digital video disk* (DVD) is an optical storage medium, with data written and read back using a laser light source. The *Blu-ray DVD* format has a capacity of 27 gigabytes to 128 gigabytes, depending on the number of layers supported. Although the original (and still main) use of DVDs was to store video data, they are capable of storing any type of digital data, including backups of

database contents. DVDs are not suitable for storing active database data since the time required to access a given piece of data can be quite long compared to the time taken by a magnetic disk.

Some DVD versions are read-only, written at the factory where they are produced, other versions support *write-once*, allowing them to be written once, but not overwritten, and some versions can be rewritten multiple times. Disks that can be written only once are called **write-once, read-many** (WORM) disks.

Optical disk **jukebox** systems contain a few drives and numerous disks that can be loaded into one of the drives automatically (by a robot arm) on demand.

- **Tape storage**. Tape storage is used primarily for backup and archival data. *Archival* data refers to data that must be stored safely for a long period of time, often for legal reasons. Magnetic tape is cheaper than disks and can safely store data for many years. However, access to data is much slower because the tape must be accessed sequentially from the beginning of the tape; tapes can be very long, requiring tens to hundreds of seconds to access data. For this reason, tape storage is referred to as **sequential-access** storage. In contrast, magnetic disk and SSD storage are referred to as **direct-access** storage because it is possible to read data from any location on disk.

  Tapes have a high capacity (1 to 12 terabyte capacities are currently available), and can be removed from the tape drive. Tape drives tend to be expensive, but individual tapes are usually significantly cheaper than magnetic disks of the same capacity. As a result, tapes are well suited to cheap archival storage and to transferring large amounts of data between different locations. Archival storage of large video files, as well as storage of large volumes of scientific data, which can range up to many petabytes (1 petabyte = 10^{15} bytes) of data, are two common use cases for tapes.

  Tape libraries (jukeboxes) are used to hold large collections of tapes, allowing automated storage and retrieval of tapes without human intervention.

The various storage media can be organized in a hierarchy (Figure 12.1) according to their speed and their cost. The higher levels are expensive, but fast. As we move down the hierarchy, the cost per bit decreases, whereas the access time increases. This trade-off is reasonable; if a given storage system were both faster and less expensive than another—other properties being the same—then there would be no reason to use the slower, more expensive memory.

The fastest storage media—for example, cache and main memory—are referred to as **primary storage**. The media in the next level in the hierarchy—for example, flash memory and magnetic disks—are referred to as **secondary storage**, or **online storage**. The media in the lowest level in the hierarchy—for example, magnetic tape and optical-disk jukeboxes—are referred to as **tertiary storage**, or **offline storage**.

In addition to the speed and cost of the various storage systems, there is also the issue of storage volatility. In the hierarchy shown in Figure 12.1, the storage systems

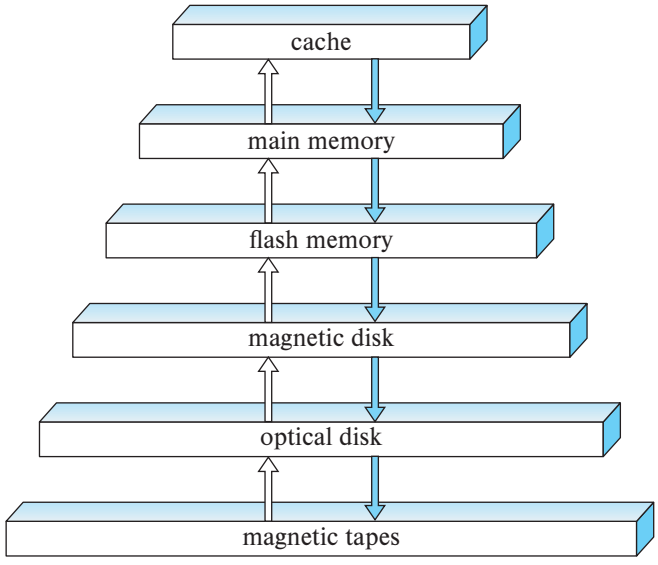


Figure 12.1 Storage device hierarchy.

from main memory up are volatile, whereas the storage systems from flash memory down are non-volatile. Data must be written to non-volatile storage for safekeeping. We shall return to the subject of safe storage of data in the face of system failures later, in Chapter 19.

12.2 Storage Interfaces

Magnetic disks as well as flash-based solid-state disks are connected to a computer system through a high-speed interconnection. Disks typically support either the **Serial ATA** (**SATA**) interface, or the **Serial Attached SCSI** (**SAS**) interface; the SAS interface is typically used only in servers. The SATA-3 version of SATA nominally supports 6 gigabytes per second, allowing data transfer speeds of up to 600 megabytes per second, while SAS version 3 supports data transfer rates of 12 gigabits per second. The **Non-Volatile Memory Express** (**NVMe**) interface is a logical interface standard developed to better support SSDs and is typically used with the PCIe interface (the PCIe interface provides high-speed data transfer internal to computer systems).

While disks are usually connected directly by cables to the disk interface of the computer system, they can be situated remotely and connected by a high-speed network to the computer. In the **storage area network** (**SAN**) architecture, large numbers of disks are connected by a high-speed network to a number of server computers. The disks are usually organized locally using a storage organization technique called *redundant arrays of independent disks* (RAID) (described later, in Section 12.5), to give the servers a logical view of a very large and very reliable disk. Interconnection technologies used

in storage area networks include iSCSI, which allows SCSI commands to be sent over an IP network, Fiber Channel FC, which supports transfer rates of 1.6 to 12 gigabytes per second, depending on the version, and InfiniBand, which provides very low latency high-bandwidth network communication.

Network attached storage (NAS) is an alternative to SAN. NAS is much like SAN, except that instead of the networked storage appearing to be a large disk, it provides a file system interface using networked file system protocols such as NFS or CIFS. Recent years have also seen the growth of **cloud storage**, where data are stored in the cloud and accessed via an API. Cloud storage has a very high latency of tens to hundreds of milliseconds, if the data are not co-located with the database, and is thus not ideal as the underlying storage for databases. However, applications often use cloud storage for storing objects. Cloud-based storage systems are discussed further in Section 21.7.

12.3 Magnetic Disks

Magnetic disks provide the bulk of secondary storage for modern computer systems. Magnetic disk capacities have been growing steadily year after year, but the storage requirements of large applications have also been growing very fast, in some cases even faster than the growth rate of disk capacities. Very large databases at "web-scale" require thousands to tens of thousands of disks to store their data.[1]

In recent years, SSD storage sizes have grown rapidly, and the cost of SSDs has come down significantly; the increasing affordability of SSDs coupled with their much better performance has resulted in SSDs increasingly becoming a competitor to magnetic disk storage for several applications. However, the fact that the per-byte cost of storage on SSDs is around six to eight times the per-byte cost of storage on magnetic disks means that magnetic disks continue to be the preferred choice for storing very large volumes of data in many applications. Example of such data include video and image data, as well as data that is accessed less frequently, such as user-generated data in many web-scale applications. SSDs have however, increasingly become the preferred choice for enterprise data.

12.3.1 Physical Characteristics of Disks

Figure 12.2 shows a schematic diagram of a magnetic disk, while Figure 12.3 shows the internals of an actual magnetic disk. Each disk **platter** has a flat, circular shape. Its two surfaces are covered with a magnetic material, and information is recorded on the surfaces. Platters are made from rigid metal or glass.

When the disk is in use, a drive motor spins it at a constant high speed, typically 5400 to 10,000 revolutions per minute, depending on the model. There is a read-write head positioned just above the surface of the platter. The disk surface is logically di-

[1] We study later, in Chapter 21, how to partition such large amounts of data across multiple nodes in a parallel computing system.

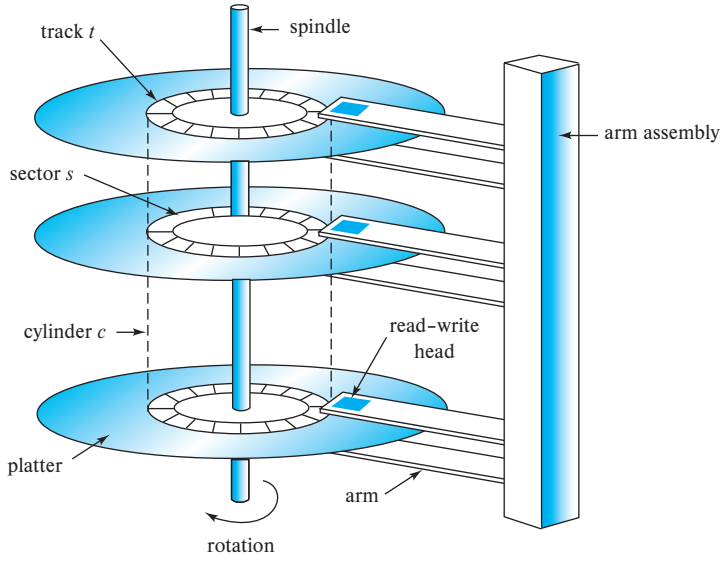


Figure 12.2 Schematic diagram of a magnetic disk.

vided into **tracks**, which are subdivided into **sectors**. A **sector** is the smallest unit of information that can be read from or written to the disk. Sector sizes are typically 512 bytes, and current generation disks have between 2 billion and 24 billion sectors. The inner tracks (closer to the spindle) are of smaller length than the outer tracks, and the outer tracks contain more sectors than the inner tracks.

The **read-write head** stores information on a sector magnetically as reversals of the direction of magnetization of the magnetic material.

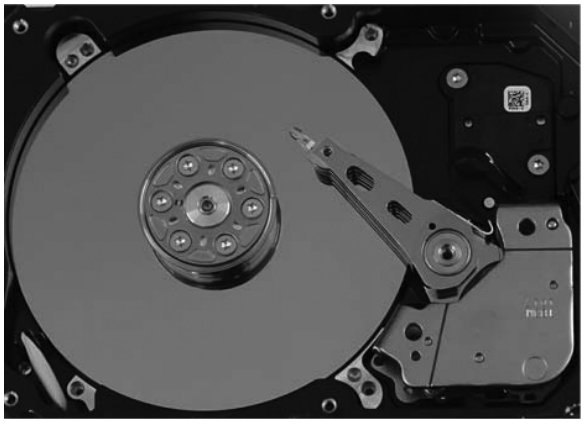


Figure 12.3 Internals of an actual magnetic disk.

Each side of a platter of a disk has a read-write head that moves across the platter to access different tracks. A disk typically contains many platters, and the read-write heads of all the tracks are mounted on a single assembly called a **disk arm** and move together. The disk platters mounted on a spindle and the heads mounted on a disk arm are together known as **head-disk assemblies**. Since the heads on all the platters move together, when the head on one platter is on the *i*th track, the heads on all other platters are also on the *i*th track of their respective platters. Hence, the *i*th tracks of all the platters together are called the *i*th **cylinder**.

The read-write heads are kept as close as possible to the disk surface to increase the recording density. The head typically floats or flies only microns from the disk surface; the spinning of the disk creates a small breeze, and the head assembly is shaped so that the breeze keeps the head floating just above the disk surface. Because the head floats so close to the surface, platters must be machined carefully to be flat.

Head crashes can be a problem. If the head contacts the disk surface, the head can scrape the recording medium off the disk, destroying the data that had been there. In older-generation disks, the head touching the surface caused the removed medium to become airborne and to come between the other heads and their platters, causing more crashes; a head crash could thus result in failure of the entire disk. Current-generation disk drives use a thin film of magnetic metal as recording medium. They are much less susceptible to failure of the entire disk, but are susceptible to failure of individual sectors.

A **disk controller** interfaces between the computer system and the actual hardware of the disk drive; in modern disk systems, the disk controller is implemented within the disk drive unit. A disk controller accepts high-level commands to read or write a sector, and initiates actions, such as moving the disk arm to the right track and actually reading or writing the data. Disk controllers also attach **checksums** to each sector that is written; the checksum is computed from the data written to the sector. When the sector is read back, the controller computes the checksum again from the retrieved data and compares it with the stored checksum; if the data are corrupted, with a high probability the newly computed checksum will not match the stored checksum. If such an error occurs, the controller will retry the read several times; if the error continues to occur, the controller will signal a read failure.

Another interesting task that disk controllers perform is **remapping of bad sectors**. If the controller detects that a sector is damaged when the disk is initially formatted, or when an attempt is made to write the sector, it can logically map the sector to a different physical location (allocated from a pool of extra sectors set aside for this purpose). The remapping is noted on disk or in non-volatile memory, and the write is carried out on the new location.

12.3.2 Performance Measures of Disks

The main measures of the qualities of a disk are capacity, access time, data-transfer rate, and reliability.

Access time is the time from when a read or write request is issued to when data transfer begins. To access (i.e., to read or write) data on a given sector of a disk, the arm first must move so that it is positioned over the correct track, and then must wait for the sector to appear under it as the disk rotates. The time for repositioning the arm is called the **seek time**, and it increases with the distance that the arm must move. Typical seek times range from 2 to 20 milliseconds depending on how far the track is from the initial arm position. Smaller disks tend to have lower seek times since the head has to travel a smaller distance.

The **average seek time** is the average of the seek times, measured over a sequence of (uniformly distributed) random requests. If all tracks have the same number of sectors, and we disregard the time required for the head to start moving and to stop moving, we can show that the average seek time is one-third the worst-case seek time. Taking these factors into account, the average seek time is around one-half of the maximum seek time. Average seek times currently range between 4 and 10 milliseconds, depending on the disk model.[2]

Once the head has reached the desired track, the time spent waiting for the sector to be accessed to appear under the head is called the **rotational latency time**. Rotational speeds of disks today range from 5400 rotations per minute (90 rotations per second) up to 15,000 rotations per minute (250 rotations per second), or, equivalently, 4 milliseconds to 11.1 milliseconds per rotation. On an average, one-half of a rotation of the disk is required for the beginning of the desired sector to appear under the head. Thus, the **average latency time** of the disk is one-half the time for a full rotation of the disk. Disks with higher rotational speeds are used for applications where latency needs to be minimized.

The access time is then the sum of the seek time and the latency; average access times range from 5 to 20 milliseconds depending on the disk model. Once the first sector of the data to be accessed has come under the head, data transfer begins. The **data-transfer rate** is the rate at which data can be retrieved from or stored to the disk. Current disk systems support maximum transfer rates of 50 to 200 megabytes per second; transfer rates are significantly lower than the maximum transfer rates for inner tracks of the disk, since they have fewer sectors. For example, a disk with a maximum transfer rate of 100 megabytes per second may have a sustained transfer rate of around 30 megabytes per second on its inner tracks.

Requests for disk I/O are typically generated by the file system but can be generated directly by the database system. Each request specifies the address on the disk to be referenced; that address is in the form of a *block number*. A **disk block** is a logical unit of storage allocation and retrieval, and block sizes today typically range from 4 to 16

[2]Smaller 2.5-inch diameter disks have a lesser arm movement distance than larger 3.5-inch disks, and thus have lower seek times. As a result 2.5-inch disks have been the preferred choice for applications where latency needs to be minimized, although SSDs are increasingly preferred for such applications. Larger 3.5-inch diameter disks have a lower cost per byte and are used in data storage applications where cost is an important factor.

kilobytes. Data are transferred between disk and main memory in units of blocks. The term **page** is often used to refer to blocks, although in a few contexts (such as flash memory) they refer to different things.

A sequence of requests for blocks from disk may be classified as a sequential access pattern or a random access pattern. In a **sequential access** pattern, successive requests are for successive block numbers, which are on the same track, or on adjacent tracks. To read blocks in sequential access, a disk seek may be required for the first block, but successive requests would either not require a seek, or require a seek to an adjacent track, which is faster than a seek to a track that is farther away. Data transfer rates are highest with a sequential access pattern, since seek time is minimal.

In contrast, in a **random access** pattern, successive requests are for blocks that are randomly located on disk. Each such request would require a seek. The number of **I/O operations per second** (**IOPS**), that is, the number random block accesses that can be satisfied by a disk in a second, depends on the access time, and the block size, and the data transfer rate of the disk. With a 4-kilobyte block size, current generation disks support between 50 and 200 IOPS, depending on the model. Since only a small amount (one block) of data are read per seek, the data transfer rate is significantly lower with a random access pattern than with a sequential access pattern.

The final commonly used measure of a disk is the **mean time to failure** (**MTTF**),[3] which is a measure of the reliability of the disk. The mean time to failure of a disk (or of any other system) is the amount of time that, on average, we can expect the system to run continuously without any failure. According to vendors' claims, the mean time to failure of disks today ranges from 500,000 to 1,200,000 hours—about 57 to 136 years. In practice the claimed mean time to failure is computed on the probability of failure when the disk is new—the figure means that given 1000 relatively new disks, if the MTTF is 1,200,000 hours, on an average one of them will fail in 1200 hours. A mean time to failure of 1,200,000 hours does not imply that the disk can be expected to function for 136 years! Most disks have an expected life span of about 5 years and have significantly higher rates of failure once they become more than a few years old.

12.4 Flash Memory

There are two types of flash memory, NOR flash and NAND flash. NAND flash is the variant that is predominantly used for data storage. Reading from NAND flash requires an entire *page* of data, which is very commonly 4096 bytes, to be fetched from NAND flash into main memory. Pages in a NAND flash are thus similar to sectors in a magnetic disk.

[3]The term **mean time between failures** (**MTBF**) is often used to refer to MTTF in the context of disk drives, although technically MTBF should only be used in the context of systems that can be repaired after failure, and may fail again; MTBF would then be the sum of MTTF and the mean time to repair. Magnetic disks can almost never be repaired after a failure.

Solid-state disks (SSDs) are built using NAND flash and provide the same block-oriented interface as disk storage. Compared to magnetic disks, SSDs can provide much faster random access: the latency to retrieve a page of data ranges from 20 to 100 microseconds for SSDs, whereas a random access on disk would take 5 to 10 milliseconds. The data transfer rate of SSDs is higher than that of magnetic disks and is usually limited by the interconnect technology; transfer rates range from around 500 megabytes per second with SATA interfaces, up to 3 gigabytes per second using NVMe PCIe interfaces, depending on the specific SSD model, in contrast to a maximum of about 200 megabytes per second with magnetic disk. The power consumption of SSDs is also significantly lower than that of magnetic disks.

Writes to flash memory are a little more complicated. A write to a page of flash memory typically takes about 100 microseconds. However, once written, a page of flash memory cannot be directly overwritten. Instead, it has to be erased and rewritten subsequently. The erase operation must be performed on a group of pages, called an **erase block**, erasing all the pages in the block, and takes about 2 to 5 milliseconds. An erase block (often referred to as just "block" in flash literature), is typically 256 kilobytes to 1 megabyte, and contains around 128 to 256 pages. Further, there is a limit to how many times a flash page can be erased, typically around 100,000 to 1,000,000 times. Once this limit is reached, errors in storing bits are likely to occur.

Flash memory systems limit the impact of both the slow erase speed and the update limits by mapping logical page numbers to physical page numbers. When a logical page is updated, it can be remapped to any already erased physical page, and the original location can be erased later. Each physical page has a small area of memory where its logical address is stored; if the logical address is remapped to a different physical page, the original physical page is marked as deleted. Thus, by scanning the physical pages, we can find where each logical page resides. The logical-to-physical page mapping is replicated in an in-memory **translation table** for quick access.

Blocks containing multiple deleted pages are periodically erased, taking care to first copy nondeleted pages in those blocks to a different block (the translation table is updated for these nondeleted pages). Since each physical page can be updated only a fixed number of times, physical pages that have been erased many times are assigned "cold data," that is, data that are rarely updated, while pages that have not been erased many times are used to store "hot data," that is, data that are updated frequently. This principle of evenly distributing erase operations across physical blocks is called **wear leveling** and is usually performed transparently by flash-memory controllers. If a physical page is damaged due to an excessive number of updates, it can be removed from usage, without affecting the flash memory as a whole.

All the above actions are carried out by a layer of software called the **flash translation layer**; above this layer, flash storage looks identical to magnetic disk storage, providing the same page/sector-oriented interface, except that flash storage is much faster. File systems and database storage structures can thus see an identical logical view of the underlying storage structure, regardless of whether it is flash or magnetic storage.

Note 12.1 STORAGE CLASS MEMORY

Although flash is the most widely used type of non-volatile memory, there have been a number of alternative non-volatile memory technologies developed over the years. Several of these technologies allow direct read and write access to individual bytes or words, avoiding the need to read or write in units of pages (and also avoiding the erase overhead of NAND flash). Such types of non-volatile memory are referred to as **storage class memory**, since they can be treated as a large non-volatile block of memory. The 3D-XPoint memory technology, developed by Intel and Micron, is a recently developed storage class memory technology. In terms of cost per byte, latency of access, and capacity, 3D-XPoint memory lies in between main memory and flash memory. Intel Optane SSDs based on 3D-XPoint started shipping in 2017, and Optane persistent memory modules were announced in 2018.

SSD performance is usually expressed in terms of:

1. The *number of random block reads per second*, with 4-kilobyte blocks being the standard. Typical values in 2018 are about 10,000 random reads per second (also referred to as 10,000 IOPS) with 4-kilobyte blocks, although some models support higher rates.

   Unlike magnetic disks, SSDs can support multiple random requests in parallel, with 32 parallel requests being commonly supported; a flash disk with SATA interface supports nearly 100,000 random 4-kilobyte block reads in a second with 32 requests sent in parallel, while SSDs connected using NVMe PCIe can support over 350,000 random 4-kilobyte block reads per second. These numbers are specified as QD-1 for rates without parallelism and QD-n for n-way parallelism, with QD-32 being the most commonly used number.

2. The *data transfer rate* for sequential reads and sequential writes. Typical rates for both sequential reads and sequential writes are 400 to 500 megabytes per second for SSDs with a SATA 3 interface, and 2 to 3 gigabytes per second for SSDs using NVMe over the PCIe 3.0x4 interface.

3. The *number of random block writes per second*, with 4-kilobyte blocks being the standard. Typical values in 2018 are about 40,000 random 4-kilobyte writes per second for QD-1 (without parallelism), and around 100,000 IOPS for QD-32. although some models support higher rates for both QD-1 and QD-32.

Hybrid disk drives are hard-disk systems that combine magnetic storage with a smaller amount of flash memory, which is used as a cache for frequently accessed data. Frequently accessed data that are rarely updated are ideal for caching in flash memory.

Modern SAN and NAS systems support the use of a combination of magnetic disks and SSDs, and they can be configured to use the SSDs as a cache for data that reside on magnetic disks.

12.5 RAID

The data-storage requirements of some applications (in particular web, database, and multimedia applications) have been growing so fast that a large number of disks are needed to store their data, even though disk-drive capacities have been growing very fast.

Having a large number of disks in a system presents opportunities for improving the rate at which data can be read or written, if the disks are operated in parallel. Several independent reads or writes can also be performed in parallel. Furthermore, this setup offers the potential for improving the reliability of data storage, because redundant information can be stored on multiple disks. Thus, failure of one disk does not lead to loss of data.

A variety of disk-organization techniques, collectively called **redundant arrays of independent disks** (**RAID**), have been proposed to achieve improved performance and reliability.

In the past, system designers viewed storage systems composed of several small, cheap disks as a cost-effective alternative to using large, expensive disks; the cost per megabyte of the smaller disks was less than that of larger disks. In fact, the I in RAID, which now stands for *independent*, originally stood for *inexpensive*. Today, however, all disks are physically small, and larger-capacity disks actually have a lower cost per megabyte. RAID systems are used for their higher reliability and higher performance rate, rather than for economic reasons. Another key justification for RAID use is easier management and operations.

12.5.1 Improvement of Reliability via Redundancy

Let us first consider reliability. The chance that at least one disk out of a set of N disks will fail is much higher than the chance that a specific single disk will fail. Suppose that the mean time to failure of a disk is 100,000 hours, or slightly over 11 years. Then, the mean time to failure of some disk in an array of 100 disks will be 100,000/100 = 1000 hours, or around 42 days, which is not long at all! If we store only one copy of the data, then each disk failure will result in loss of a significant amount of data (as discussed in Section 12.3.1). Such a high frequency of data loss is unacceptable.

The solution to the problem of reliability is to introduce **redundancy**; that is, we store extra information that is not needed normally but that can be used in the event of failure of a disk to rebuild the lost information. Thus, even if a disk fails, data are not lost, so the effective mean time to failure is increased, provided that we count only failures that lead to loss of data or to non-availability of data.

The simplest (but most expensive) approach to introducing redundancy is to duplicate every disk. This technique is called **mirroring** (or, sometimes, *shadowing*). A logical disk then consists of two physical disks, and every write is carried out on both disks. If one of the disks fails, the data can be read from the other. Data will be lost only if the second disk fails before the first failed disk is repaired.

The mean time to failure (where failure is the loss of data) of a mirrored disk depends on the mean time to failure of the individual disks, as well as on the **mean time to repair**, which is the time it takes (on an average) to replace a failed disk and to restore the data on it. Suppose that the failures of the two disks are *independent*; that is, there is no connection between the failure of one disk and the failure of the other. Then, if the mean time to failure of a single disk is 100,000 hours, and the mean time to repair is 10 hours, the **mean time to data loss** of a mirrored disk system is $100,000^2/(2 * 10) = 500 * 10^6$ hours, or 57,000 years! (We do not go into the derivations here; references in the bibliographical notes provide the details.)

You should be aware that the assumption of independence of disk failures is not valid. Power failures and natural disasters such as earthquakes, fires, and floods may result in damage to both disks at the same time. As disks age, the probability of failure increases, increasing the chance that a second disk will fail while the first is being repaired. In spite of all these considerations, however, mirrored-disk systems offer much higher reliability than do single-disk systems. Mirrored-disk systems with mean time to data loss of about 500,000 to 1,000,000 hours, or 55 to 110 years, are available today.

Power failures are a particular source of concern, since they occur far more frequently than do natural disasters. Power failures are not a concern if there is no data transfer to disk in progress when they occur. However, even with mirroring of disks, if writes are in progress to the same block in both disks, and power fails before both blocks are fully written, the two blocks can be in an inconsistent state. The solution to this problem is to write one copy first, then the next, so that one of the two copies is always consistent. Some extra actions are required when we restart after a power failure, to recover from incomplete writes. This matter is examined in Practice Exercise 12.6.

12.5.2 Improvement in Performance via Parallelism

Now let us consider the benefit of parallel access to multiple disks. With disk mirroring, the rate at which read requests can be handled is doubled, since read requests can be sent to either disk (as long as both disks in a pair are functional, as is almost always the case). The transfer rate of each read is the same as in a single-disk system, but the number of reads per unit time has doubled.

With multiple disks, we can improve the transfer rate as well (or instead) by **striping data** across multiple disks. In its simplest form, data striping consists of splitting the bits of each byte across multiple disks; such striping is called **bit-level striping**. For example, if we have an array of eight disks, we write bit i of each byte to disk i. In such an organization, every disk participates in every access (read or write), so the number of accesses that can be processed per second is about the same as on a single disk, but each access can read eight times as much data in the same time as on a single disk.

Block-level striping stripes blocks across multiple disks. It treats the array of disks as a single large disk, and it gives blocks logical numbers; we assume the block numbers start from 0. With an array of n disks, block-level striping assigns logical block i of the disk array to disk $(i \bmod n) + 1$; it uses the $\lfloor i/n \rfloor$th physical block of the disk to store logical block i. For example, with eight disks, logical block 0 is stored in physical block 0 of disk 1, while logical block 11 is stored in physical block 1 of disk 4. When reading a large file, block-level striping fetches n blocks at a time in parallel from the n disks, giving a high data-transfer rate for large reads. When a single block is read, the data-transfer rate is the same as on one disk, but the remaining $n-1$ disks are free to perform other actions.

Block-level striping offers several advantages over bit-level striping, including the ability to support a larger number of block reads per second, and lower latency for single block reads. As a result, bit-level striping is not used in any practical system.

In summary, there are two main goals of parallelism in a disk system:

1. Load-balance multiple small accesses (block accesses), so that the throughput of such accesses increases.
2. Parallelize large accesses so that the response time of large accesses is reduced.

12.5.3 RAID Levels

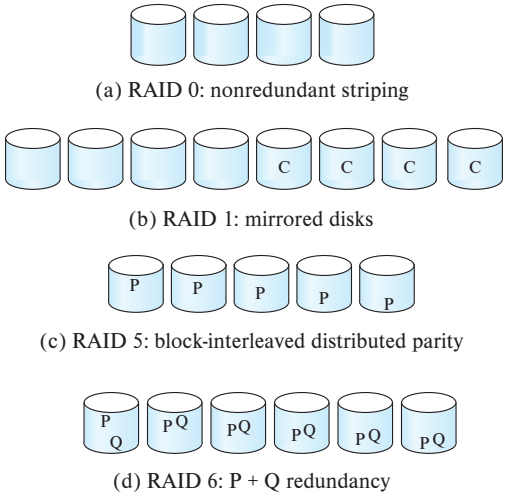
Mirroring provides high reliability, but it is expensive. Striping provides high data-transfer rates, but does not improve reliability. Various alternative schemes aim to provide redundancy at lower cost by combining disk striping with "parity blocks".

Blocks in a RAID system are partitioned into sets, as we shall see. For a given set of blocks, a **parity block** can be computed and stored on disk; the ith bit of the parity block is computed as the "exclusive or" (XOR) of the ith bits of the all blocks in the set. If the contents of any one of the blocks in a set is lost due to a failure, the block contents can be recovered by computing the bitwise-XOR of the remaining blocks in the set, along with the parity block.

Whenever a block is written, the parity block for its set must be recomputed and written to disk. The new value of the parity block can be computed by either (i) reading all the other blocks in the set from disk and computing the new parity block, or (ii) by computing the XOR of the old value of the parity block with the old and new value of the updated block.

These schemes have different cost-performance trade-offs. The schemes are classified into **RAID levels.**[4]. Figure 12.4 illustrates the four levels that are used in practice. In the figure, P indicates error-correcting bits, and C indicates a second copy of the data. For all levels, the figure depicts four disks' worth of data, and the extra disks depicted are used to store redundant information for failure recovery.

[4]There are 7 different RAID levels, numbered 0 to 6; Levels 2, 3, and 4 are not used in practice anymore and thus are not covered in the text



(a) RAID 0: nonredundant striping

(b) RAID 1: mirrored disks

(c) RAID 5: block-interleaved distributed parity

(d) RAID 6: P + Q redundancy

Figure 12.4 RAID levels.

- **RAID level 0** refers to disk arrays with striping at the level of blocks, but without any redundancy (such as mirroring or parity bits). Figure 12.4a shows an array of size 4.

- **RAID level 1** refers to disk mirroring with block striping. Figure 12.4b shows a mirrored organization that holds four disks' worth of data.

  Note that some vendors use the term **RAID level 1+0** or **RAID level 10** to refer to mirroring with striping, and they use the term RAID level 1 to refer to mirroring without striping. Mirroring without striping can also be used with arrays of disks, to give the appearance of a single large, reliable disk: if each disk has M blocks, logical blocks 0 to $M - 1$ are stored on disk 0, M to $2M - 1$ on disk 1(the second disk), and so on, and each disk is mirrored.[5]

- **RAID level 5** refers to block-interleaved distributed parity. The data and parity are partitioned among all $N + 1$ disks. For each set of N logical blocks, one of the disks stores the parity, and the other N disks store the blocks. The parity blocks are stored on different disks for different sets of N blocks. Thus, all disks can participate in satisfying read requests.[6]

  Figure 12.4c shows the setup. The P's are distributed across all the disks. For example, with an array of five disks, the parity block, labeled Pk, for logical blocks

[5]Note that some vendors use the term RAID 0+1 to refer to a version of RAID that uses striping to create a RAID 0 array, and mirrors the array onto another array, with the difference from RAID 1 being that if a disk fails, the RAID 0 array containing the disk becomes unusable. The mirrored array can still be used, so there is no loss of data. This arrangement is inferior to RAID 1 when a disk has failed, since the other disks in the RAID 0 array can continue to be used in RAID 1, but remain idle in RAID 0+1.

[6]In RAID level 4 (which is not used in practice) all parity blocks are stored on one disk. That disk would not be useful for reads, and it would also have a higher load than other disks if there were many random writes.

$4k, 4k + 1, 4k + 2, 4k + 3$ is stored in disk k mod 5; the corresponding blocks of the other four disks store the four data blocks $4k$ to $4k + 3$. The following table indicates how the first 20 blocks, numbered 0 to 19, and their parity blocks are laid out. The pattern shown gets repeated on further blocks.

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4

Note that a parity block cannot store parity for blocks in the same disk, since then a disk failure would result in loss of data as well as of parity, and hence would not be recoverable.

- **RAID level 6**, the P + Q redundancy scheme, is much like RAID level 5, but it stores extra redundant information to guard against multiple disk failures. Instead of using parity, level 6 uses error-correcting codes such as the Reed-Solomon codes (see the bibliographical notes). In the scheme in Figure 12.4g, two bits of redundant data are stored for every four bits of data—unlike one parity bit in level 5—and the system can tolerate two disk failures.

  The letters P and Q in the figure denote blocks containing the two corresponding error-correcting blocks for a given set of data blocks. The layout of blocks is an extension of that for RAID 5. For example, with six disks, the two parity blocks, labeled Pk and Qk, for logical blocks $4k, 4k + 1, 4k + 2, and 4k + 3$ are stored in disk k mod 6 and $(k + 1)$ mod 6, and the corresponding blocks of the other four disks store the four data blocks $4k$ to $4k + 3$.

Finally, we note that several variations have been proposed to the basic RAID schemes described here, and different vendors use different terminologies for the variants. Some vendors support nested schemes that create multiple separate RAID arrays, and then stripe data across the RAID arrays; one of RAID levels 1, 5 or 6 is chosen for the individual arrays. References to further information on this idea are provided in the Further Reading section at the end of the chapter.

12.5.4 Hardware Issues

RAID can be implemented with no change at the hardware level, using only software modification. Such RAID implementations are called **software RAID**. However, there are significant benefits to be had by building special-purpose hardware to support RAID, which we outline below; systems with special hardware support are called **hardware RAID** systems.

Hardware RAID implementations can use non-volatile RAM to record writes before they are performed. In case of power failure, when the system comes back up, it retrieves information about any incomplete writes from non-volatile RAM and then completes the writes. Normal operations can then commence.

In contrast, with software RAID extra work needs to be done to detect blocks that may have been partially written before power failure. For RAID 1, all blocks of the disks are scanned to see if any pair of blocks on the two disks have different contents. For RAID 5, the disks need to be scanned and parity recomputed for each set of blocks and compared to the stored parity. Such scans take a long time, and they are done in the background using a small fraction of the disks' available bandwidth. See Practice Exercise 12.6 for details of how to recover data to the latest value, when an inconsistency is detected; we revisit this issue in the context of database system recovery in Section 19.2.1. The RAID system is said to be *resynchronizing* (or *resynching*) during this phase; normal reads and writes are allowed while resynchronization is in progress, but a failure of a disk during this phase could result in data loss for blocks with incomplete writes. Hardware RAID does not have this limitation.

Even if all writes are completed properly, there is a small chance of a sector in a disk becoming unreadable at some point, even though it was successfully written earlier. Reasons for loss of data on individual sectors could range from manufacturing defects to data corruption on a track when an adjacent track is written repeatedly. Such loss of data that were successfully written earlier is sometimes referred to as a *latent failure*, or as *bit rot*. When such a failure happens, if it is detected early the data can be recovered from the remaining disks in the RAID organization. However, if such a failure remains undetected, a single disk failure could lead to data loss if a sector in one of the other disks has a latent failure.

To minimize the chance of such data loss, good RAID controllers perform **scrubbing**; that is, during periods when disks are idle, every sector of every disk is read, and if any sector is found to be unreadable, the data are recovered from the remaining disks in the RAID organization, and the sector is written back. (If the physical sector is damaged, the disk controller would remap the logical sector address to a different physical sector on disk.)

Server hardware is often designed to permit **hot swapping**; that is, faulty disks can be removed and replaced by new ones without turning power off. The RAID controller can detect that a disk was replaced by a new one and can immediately proceed to reconstruct the data that was on the old disk, and write it to the new disk. Hot swapping reduces the mean time to repair, since replacement of a disk does not have to wait until a time when the system can be shut down. In fact, many critical systems today run on a 24×7 schedule; that is, they run 24 hours a day, 7 days a week, providing no time for shutting down and replacing a failed disk. Further, many RAID implementations assign a spare disk for each array (or for a set of disk arrays). If a disk fails, the spare disk is immediately used as a replacement. As a result, the mean time to repair is reduced greatly, minimizing the chance of any data loss. The failed disk can be replaced at leisure.

The power supply, or the disk controller, or even the system interconnection in a RAID system could become a single point of failure that could stop the functioning of the RAID system. To avoid this possibility, good RAID implementations have multiple redundant power supplies (with battery backups so they continue to function even if power fails). Such RAID systems have multiple disk interfaces and multiple interconnections to connect the RAID system to the computer system (or to a network of computer systems). Thus, failure of any single component will not stop the functioning of the RAID system.

12.5.5 Choice of RAID Level

The factors to be taken into account in choosing a RAID level are:

- Monetary cost of extra disk-storage requirements.
- Performance requirements in terms of number of I/O operations per second.
- Performance when a disk has failed.
- Performance during rebuild (i.e., while the data in a failed disk are being rebuilt on a new disk).

The time to rebuild the data of a failed disk can be significant, and it varies with the RAID level that is used. Rebuilding is easiest for RAID level 1, since data can be copied from another disk; for the other levels, we need to access all the other disks in the array to rebuild data of a failed disk. The **rebuild performance** of a RAID system may be an important factor if continuous availability of data is required, as it is in high-performance database systems. Furthermore, since rebuild time can form a significant part of the repair time, rebuild performance also influences the mean time to data loss.

RAID level 0 is used in a few high-performance applications where data safety is not critical, but not anywhere else.

RAID level 1 is popular for applications such as storage of log files in a database system, since it offers the best write performance. RAID level 5 has a lower storage overhead than level 1, but it has a higher time overhead for writes. For applications where data are read frequently, and written rarely, level 5 is the preferred choice.

Disk-storage capacities have been increasing rapidly for many years. Capacities were effectively doubling every 13 months at one point; although the current rate of growth is much less now, capacities have continued to increase rapidly. The cost per byte of disk storage has been falling at about the same rate as the capacity increase. As a result, for many existing database applications with moderate storage requirements, the monetary cost of the extra disk storage needed for mirroring has become relatively small (the extra monetary cost, however, remains a significant issue for storage-intensive applications such as video data storage). Disk access speeds have not improved significantly in recent years, while the number of I/O operations required per second has increased tremendously, particularly for web application servers.

RAID level 5 has a significant overhead for random writes, since a single random block write requires 2 block reads (to get the old values of the block and parity block) and 2 block writes to write these blocks back. In contrast, the overhead is low for large sequential writes, since the parity block can be computed from the new blocks in most cases, without any reads. RAID level 1 is therefore the RAID level of choice for many applications with moderate storage requirements and high random I/O requirements.

RAID level 6 offers better reliability than level 1 or 5, since it can tolerate two disk failures without losing data. In terms of performance during normal operation, it is similar to RAID level 5, but it has a higher storage cost than RAID level 5. RAID level 6 is used in applications where data safety is very important. It is being viewed as increasingly important since latent sector failures are not uncommon, and it may take a long time to be detected and repaired. A failure of a different disk before a latent failure is detected and repaired would then be similar to a two-disk failure for that sector and result in loss of data of that sector. RAID levels 1 and 5 would suffer from data loss in such a scenario, unlike level 6.

Mirroring can also be extended to store copies on three disks instead of two to survive two-disk failures. Such triple-redundancy schemes are not commonly used in RAID systems, although they are used in distributed file systems, where data are stored in multiple machines, since the probability of machine failure is significantly higher than that of disk failure.

RAID system designers have to make several other decisions as well. For example, how many disks should there be in an array? How many bits should be protected by each parity bit? If there are more disks in an array, data-transfer rates are higher, but the system will be more expensive. If there are more bits protected by a parity bit, the space overhead due to parity bits is lower, but there is an increased chance that a second disk will fail before the first failed disk is repaired, and that will result in data loss.

12.5.6 Other RAID Applications

The concepts of RAID have been generalized to other storage devices, including in the flash memory devices within SSDs, arrays of tapes, and even to the broadcast of data over wireless systems. Individual flash pages have a higher rate of data loss than sectors of magnetic disks. Flash devices such as SSDs implement RAID internally, to ensure that the device does not lose data due to the loss of a flash page. When applied to arrays of tapes, the RAID structures are able to recover data even if one of the tapes in an array of tapes is damaged. When applied to broadcast of data, a block of data are split into short units and is broadcast along with a parity unit; if one of the units is not received for any reason, it can be reconstructed from the other units.

12.6 Disk-Block Access

Requests for disk I/O are generated by the database system, with the query processing subsystem responsible for most of the disk I/O. Each request specifies a disk identifier and a logical block number on the disk; in case database data are stored in operating

system files, the request instead specifies the file identifier and a block number within the file. Data are transferred between disk and main memory in units of blocks.

As we saw earlier, a sequence of requests for blocks from disk may be classified as a sequential access pattern or a random access pattern. In a *sequential access* pattern, successive requests are for successive block numbers, which are on the same track, or on adjacent tracks. In contrast, in a *random access* pattern, successive requests are for blocks that are randomly located on disk. Each such request would require a seek, resulting in a longer access time, and a lower number of random I/O operations per second.

A number of techniques have been developed for improving the speed of access to blocks, by minimizing the number of accesses, and in particular minimizing the number of random accesses. We describe these techniques below. Reducing the number of random accesses is very important for data stored on magnetic disks; SSDs support much faster random access than do magnetic disks, so the impact of random access is less with SSDs, but data access from SSDs can still benefit from some of the techniques described below.

- **Buffering**. Blocks that are read from disk are stored temporarily in an in-memory buffer, to satisfy future requests. Buffering is done by both the operating system and the database system. Database buffering is discussed in more detail in Section 13.5.
- **Read-ahead**. When a disk block is accessed, consecutive blocks from the same track are read into an in-memory buffer even if there is no pending request for the blocks. In the case of sequential access, such **read-ahead** ensures that many blocks are already in memory when they are requested, and it minimizes the time wasted in disk seeks and rotational latency per block read. Operating systems also routinely perform read-ahead for consecutive blocks of an operating system file. Read-ahead is, however, not very useful for random block accesses.
- **Scheduling**. If several blocks from a cylinder need to be transferred from disk to main memory, we may be able to save access time by requesting the blocks in the order in which they will pass under the heads. If the desired blocks are on different cylinders, it is advantageous to request the blocks in an order that minimizes disk-arm movement. **Disk-arm–scheduling** algorithms attempt to order accesses to tracks in a fashion that increases the number of accesses that can be processed. A commonly used algorithm is the **elevator algorithm**, which works in the same way many elevators do. Suppose that, initially, the arm is moving from the innermost track toward the outside of the disk. Under the elevator algorithm's control, for each track for which there is an access request, the arm stops at that track, services requests for the track, and then continues moving outward until there are no waiting requests for tracks farther out. At this point, the arm changes direction and moves toward the inside, again stopping at each track for which there is a re-

quest, until it reaches a track where there is no request for tracks farther toward the center. Then, it reverses direction and starts a new cycle.

Disk controllers usually perform the task of reordering read requests to improve performance, since they are intimately aware of the organization of blocks on disk, of the rotational position of the disk platters, and of the position of the disk arm. To enable such reordering, the disk controller interface must allow multiple requests to be added to a queue; results may be returned in a different order from the request order.

- **File organization**. To reduce block-access time, we can organize blocks on disk in a way that corresponds closely to the way we expect data to be accessed. For example, if we expect a file to be accessed sequentially, then we should ideally keep all the blocks of the file sequentially on adjacent cylinders. Modern disks hide the exact block location from the operating system but use a logical numbering of blocks that gives consecutive numbers to blocks that are adjacent to each other. By allocating consecutive blocks of a file to disk blocks that are consecutively numbered, operating systems ensure that files are stored sequentially.

  Storing a large file in a single long sequence of consecutive blocks poses challenges to disk block allocation; instead, operating systems allocate some number of consecutive blocks (an **extent**) at a time to a file. Different extents allocated to a file may not be adjacent to each other on disk. Sequential access to the file needs one seek per extent, instead of one seek per block if blocks are randomly allocated; with large enough extents, the cost of seeks relative to data transfer costs can be minimized.

  Over time, a sequential file that has multiple small appends may become **fragmented**; that is, its blocks become scattered all over the disk. To reduce fragmentation, the system can make a backup copy of the data on disk and restore the entire disk. The restore operation writes back the blocks of each file contiguously (or nearly so). Some systems (such as different versions of the Windows operating system) have utilities that scan the disk and then move blocks to decrease the fragmentation. The performance increases realized from these techniques can be quite significant.

- **Non-volatile write buffers**. Since the contents of main memory are lost in a power failure, information about database updates has to be recorded on disk to survive possible system crashes. For this reason, the performance of update-intensive database applications, such as transaction-processing systems, is heavily dependent on the latency of disk writes.

  We can use *non-volatile random-access memory* (NVRAM) to speed up disk writes. The contents of NVRAM are not lost in power failure. NVRAM was implemented using battery-backed-up RAM in earlier days, but flash memory is currently the primary medium for non-volatile write buffering. The idea is that, when the database system (or the operating system) requests that a block be written to disk, the disk controller writes the block to a **non-volatile write buffer** and imme-

diately notifies the operating system that the write completed successfully. The controller can subsequently write the data to their destination on disk in a way that minimizes disk arm movement, using the elevator algorithm, for example. If such write reordering is done without using non-volatile write buffers, the database state may become inconsistent in the event of a system crash; recovery algorithms that we study later in Chapter 19 depend on writes being written in the specified order. When the database system requests a block write, it notices a delay only if the NVRAM buffer is full. On recovery from a system crash, any pending buffered writes in the NVRAM are written back to the disk. NVRAM buffers are found in certain high-end disks, but are more frequently found in RAID controllers.

In addition to the above low-level optimizations, optimizations to minimize random accesses can be done at a higher level, by clever design of query processing algorithms. We study efficient query processing techniques in Chapter 15.

12.7 Summary

- Several types of data storage exist in most computer systems. They are classified by the speed with which they can access data, by their cost per unit of data to buy the memory, and by their reliability. Among the media available are cache, main memory, flash memory, magnetic disks, optical disks, and magnetic tapes.
- Magnetic disks are mechanical devices, and data access requires a read–write head to move to the required cylinder, and the rotation of the platters must then bring the required sector under the read–write head. Magnetic disks thus have a high latency for data access.
- SSDs have a much lower latency for data access, and higher data transfer bandwidth than magnetic disks. However, they also have a higher cost per byte than magnetic disks.
- Disks are vulnerable to failure, which could result in loss of data stored on the disk. We can reduce the likelihood of irretrievable data loss by retaining multiple copies of data.
- Mirroring reduces the probability of data loss greatly. More sophisticated methods based on redundant arrays of independent disks (RAID) offer further benefits. By striping data across disks, these methods offer high throughput rates on large accesses; by introducing redundancy across disks, they improve reliability greatly.
- Several different RAID organizations are possible, each with different cost, performance, and reliability characteristics. RAID level 1 (mirroring) and RAID level 5 are the most commonly used.
- Several techniques have been developed to optimize disk block access, such as read ahead, buffering, disk arm scheduling, prefetching, and non-volatile write buffers.

Review Terms

- Physical storage media
  - Cache
  - Main memory
  - Flash memory
  - Magnetic disk
  - Optical storage
  - Tape storage
- Volatile storage
- Non-volatile storage
- Sequential-access
- Direct-access
- Storage interfaces
  - Serial ATA (SATA)
  - Serial Attached SCSI (SAS)
  - Non-Volatile Memory Express (NVMe)
  - Storage area network (SAN)
  - Network attached storage (NAS)
- Magnetic disk
  - Platter
  - Hard disks
  - Tracks
  - Sectors
  - Read–write head
  - Disk arm
  - Cylinder
  - Disk controller
  - Checksums
  - Remapping of bad sectors
- Disk block
- Performance measures of disks
  - Access time
  - Seek time
  - Latency time
  - I/O operations per second (IOPS)
  - Rotational latency
  - Data-transfer rate
  - Mean time to failure (MTTF)
- Flash Storage
  - Erase Block
  - Wear leveling
  - Flash translation table
  - Flash Translation Layer
- Storage class memory
  - 3D-XPoint
- Redundant arrays of independent disks (RAID)
  - Mirroring
  - Data striping
  - Bit-level striping
  - Block-level striping
- RAID levels
  - Level 0 (block striping, no redundancy)
  - Level 1 (block striping, mirroring)
  - Level 5 (block striping, distributed parity)

- Level 6 (block striping, P + Q redundancy)
- Rebuild performance
- Software RAID
- Hardware RAID
- Hot swapping
- Optimization of disk-block access
- Disk-arm scheduling
- Elevator algorithm
- File organization
- Defragmenting
- Non-volatile write buffers
- Log disk

Practice Exercises

12.1 SSDs can be used as a storage layer between memory and magnetic disks, with some parts of the database (e.g., some relations) stored on SSDs and the rest on magnetic disks. Alternatively, SSDs can be used as a buffer or cache for magnetic disks; frequently used blocks would reside on the SSD layer, while infrequently used blocks would reside on magnetic disk.

a. Which of the two alternatives would you choose if you need to support real-time queries that must be answered within a guaranteed short period of time? Explain why.

b. Which of the two alternatives would you choose if you had a very large *customer* relation, where only some disk blocks of the relation are accessed frequently, with other blocks rarely accessed.

12.2 Some databases use magnetic disks in a way that only sectors in outer tracks are used, while sectors in inner tracks are left unused. What might be the benefits of doing so?

12.3 Flash storage:

a. How is the flash translation table, which is used to map logical page numbers to physical page numbers, created in memory?

b. Suppose you have a 64-gigabyte flash storage system, with a 4096-byte page size. How big would the flash translation table be, assuming each page has a 32-bit address, and the table is stored as an array?

c. Suggest how to reduce the size of the translation table if very often long ranges of consecutive logical page numbers are mapped to consecutive physical page numbers.

12.4 Consider the following data and parity-block arrangement on four disks:

Disk 1	Disk 2	Disk 3	Disk 4
B_1	B_2	B_3	B_4
P_1	B_5	B_6	B_7
B_8	P_2	B_9	B_{10}
⋮	⋮	⋮	⋮

The B_is represent data blocks; the P_is represent parity blocks. Parity block P_i is the parity block for data blocks B_{4i-3} to B_{4i}. What, if any, problem might this arrangement present?

12.5 A database administrator can choose how many disks are organized into a single RAID 5 array. What are the trade-offs between having fewer disks versus more disks, in terms of cost, reliability, performance during failure, and performance during rebuild?

12.6 A power failure that occurs while a disk block is being written could result in the block being only partially written. Assume that partially written blocks can be detected. An atomic block write is one where either the disk block is fully written or nothing is written (i.e., there are no partial writes). Suggest schemes for getting the effect of atomic block writes with the following RAID schemes. Your schemes should involve work on recovery from failure.

a. RAID level 1 (mirroring)

b. RAID level 5 (block interleaved, distributed parity)

12.7 Storing all blocks of a large file on consecutive disk blocks would minimize seeks during sequential file reads. Why is it impractical to do so? What do operating systems do instead, to minimize the number of seeks during sequential reads?

Exercises

12.8 List the physical storage media available on the computers you use routinely. Give the speed with which data can be accessed on each medium.

12.9 How does the remapping of bad sectors by disk controllers affect data-retrieval rates?

12.10 Operating systems try to ensure that consecutive blocks of a file are stored on consecutive disk blocks. Why is doing so very important with magnetic disks? If SSDs were used instead, is doing so still important, or is it irrelevant? Explain why.

12.11 RAID systems typically allow you to replace failed disks without stopping access to the system. Thus, the data in the failed disk must be rebuilt and written to the replacement disk while the system is in operation. Which of the RAID levels yields the least amount of interference between the rebuild and ongoing disk accesses? Explain your answer.

12.12 What is scrubbing, in the context of RAID systems, and why is scrubbing important?

12.13 Suppose you have data that should not be lost on disk failure, and the application is write-intensive. How would you store the data?

Further Reading

[Hennessy et al. (2017)] is a popular textbook on computer architecture, which includes coverage of cache and memory organization.

The specifications of current-generation magnetic disk drives can be obtained from the web sites of their manufacturers, such as Hitachi, Seagate, Maxtor, and Western Digital. The specifications of current-generation SSDs can be obtained from the web sites of their manufacturers, such as Crucial, Intel, Micron, Samsung, SanDisk, Toshiba and Western Digital.

[Patterson et al. (1988)] provided early coverage of RAID levels and helped standardize the terminology. [Chen et al. (1994)] presents a survey of RAID principles and implementation.

A comprehensive coverage of RAID levels supported by most modern RAID systems, including the nested RAID levels, 10, 50 and 60, which combine RAID levels 1, 5 and 6 with striping as in RAID level 0, can be found in the "Introduction to RAID" chapter of [Cisco (2018)]. Reed-Solomon codes are covered in [Pless (1998)].

Bibliography

[Chen et al. (1994)] P. M. Chen, E. K. Lee, G. A. Gibson, R. H. Katz, and D. A. Patterson, "RAID: High-Performance, Reliable Secondary Storage", *ACM Computing Surveys*, Volume 26, Number 2 (1994), pages 145–185.

[Cisco (2018)] *Cisco UCS Servers RAID Guide*. Cisco (2018).

[Hennessy et al. (2017)] J. L. Hennessy, D. A. Patterson, and D. Goldberg, *Computer Architecture: A Quantitative Approach*, 6th edition, Morgan Kaufmann (2017).

[Patterson et al. (1988)] D. A. Patterson, G. Gibson, and R. H. Katz, "A Case for Redundant Arrays of Inexpensive Disks (RAID)", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1988), pages 109–116.

[Pless (1998)] V. Pless, *Introduction to the Theory of Error-Correcting Codes*, 3rd edition, John Wiley and Sons (1998).

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

Figure 12.3 is due to ©Silberschatz, Korth, and Sudarshan.

CHAPTER 13

Data Storage Structures

In Chapter 12 we studied the characteristics of physical storage media, focusing on magnetic disks and SSDs, and saw how to build fast and reliable storage systems using multiple disks in a RAID structure. In this chapter, we focus on the organization of data stored on the underlying storage media, and how data are accessed.

13.1 Database Storage Architecture

Persistent data are stored on non-volatile storage, which, as we saw in Chapter 12, is typically magnetic disk or SSD. Magnetic disks as well as SSDs are block structured devices, that is, data are read or written in units of a block. In contrast, databases deal with records, which are usually much smaller than a block (although in some cases records may have attributes that are very large).

Most databases use operating system files as an intermediate layer for storing records, which abstract away some details of the underlying blocks. However, to ensure efficient access, as well as to support recovery from failures (as we will see later in Chapter 19), databases must continue to be aware of blocks. Thus, in Section 13.2, we study how individual records are stored in files, taking block structure into account.

Given a set of records, the next decision lies in how to organize them in the file structure; for example, they may stored in sorted order, in the order they are created, or in an arbitrary order. Section 13.3 studies several alternative file organizations.

Section 13.4 then describes how databases organize data about the relational schemas as well as storage organization, in the data dictionary. Information in the data dictionary is crucial for many tasks, for example, to locate and retrieve records of a relation when given the name of the relation.

For a CPU to access data, it must be in main memory, whereas persistent data must be resident on non-volatile storage such as magnetic disks or SSDs. For databases that are larger than main memory, which is the usual case, data must be fetched from non-volatile storage and saved back if it is updated. Section 13.5 describes how databases use a region of memory called the database buffer to store blocks that are fetched from non-volatile storage.

An approach to storing data based on storing all values of a particular column together, rather than storing all attributes of a particular row together, has been found to work very well for analytical query processing. This idea, called *column-oriented storage*, is discussed in Section 13.6.

Some applications need very fast access to data and have small enough data sizes that the entire database can fit into the main memory of a database server machine. In such cases, we can keep a copy of the entire database in memory.[1] Databases that store the entire database in memory and optimize in-memory data structures as well as query processing and other algorithms used by the database to exploit the memory residency of data are called **main-memory databases**. Storage organization in main-memory databases is discussed in Section 13.7. We note that non-volatile memory that allows direct access to individual bytes or cache lines, called *storage class memory*, is under development. Main-memory database architectures can be further optimized for such storage.

13.2 File Organization

A database is mapped into a number of different files that are maintained by the underlying operating system. These files reside permanently on disks. A **file** is organized logically as a sequence of records. These records are mapped onto disk blocks. Files are provided as a basic construct in operating systems, so we shall assume the existence of an underlying *file system*. We need to consider ways of representing logical data models in terms of files.

Each file is also logically partitioned into fixed-length storage units called **blocks**, which are the units of both storage allocation and data transfer. Most databases use block sizes of 4 to 8 kilobytes by default, but many databases allow the block size to be specified when a database instance is created. Larger block sizes can be useful in some database applications.

A block may contain several records; the exact set of records that a block contains is determined by the form of physical data organization being used. We shall assume that *no record is larger than a block*. This assumption is realistic for most data-processing applications, such as our university example. There are certainly several kinds of large data items, such as images, that can be significantly larger than a block. We briefly discuss how to handle such large data items in Section 13.2.2, by storing large data items separately, and storing a pointer to the data item in the record.

In addition, we shall require that *each record is entirely contained in a single block*; that is, no record is contained partly in one block, and partly in another. This restriction simplifies and speeds up access to data items.

[1] To be safe, not only should the current database fit in memory, but there should be a reasonable certainty that the database will continue to fit in memory in the medium term future, despite potential growth of the organization.

In a relational database, tuples of distinct relations are generally of different sizes. One approach to mapping the database to files is to use several files and to store records of only one fixed length in any given file. An alternative is to structure our files so that we can accommodate multiple lengths for records; however, files of fixed-length records are easier to implement than are files of variable-length records. Many of the techniques used for the former can be applied to the variable-length case. Thus, we begin by considering a file of fixed-length records and consider storage of variable-length records later.

13.2.1 Fixed-Length Records

As an example, let us consider a file of *instructor* records for our university database. Each record of this file is defined (in pseudocode) as:

```
type instructor = record
                      ID varchar (5);
                      name varchar(20);
                      dept_name varchar (20);
                      salary numeric (8,2);
                  end
```

Assume that each character occupies 1 byte and that numeric (8,2) occupies 8 bytes. Suppose that instead of allocating a variable amount of bytes for the attributes *ID*, *name*, and *dept_name*, we allocate the maximum number of bytes that each attribute can hold. Then, the *instructor* record is 53 bytes long. A simple approach is to use the first 53 bytes for the first record, the next 53 bytes for the second record, and so on (Figure 13.1).

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.1 File containing *instructor* records.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.2 File of Figure 13.1, with record 3 deleted and all records moved.

However, there are two problems with this simple approach:

1. Unless the block size happens to be a multiple of 53 (which is unlikely), some records will cross block boundaries. That is, part of the record will be stored in one block and part in another. It would thus require two block accesses to read or write such a record.
2. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored.

To avoid the first problem, we allocate only as many records to a block as would fit entirely in the block (this number can be computed easily by dividing the block size by the record size, and discarding the fractional part). Any remaining bytes of each block are left unused.

When a record is deleted, we could move the record that comes after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record has been moved ahead (Figure 13.2). Such an approach requires moving a large number of records. It might be easier simply to move the final record of the file into the space occupied by the deleted record (Figure 13.3).

It is undesirable to move records to occupy the space freed by a deleted record, since doing so requires additional block accesses. Since insertions tend to be more frequent than deletions, it is acceptable to leave open the space occupied by the deleted record and to wait for a subsequent insertion before reusing the space. A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.

At the beginning of the file, we allocate a certain number of bytes as a **file header**. The header will contain a variety of information about the file. For now, all we need

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Figure 13.3 File of Figure 13.1, with record 3 deleted and final record moved.

to store there is the address of the first record whose contents are deleted. We use this first record to store the address of the second available record, and so on. Intuitively, we can think of these stored addresses as *pointers*, since they point to the location of a record. The deleted records thus form a linked list, which is often referred to as a **free list**. Figure 13.4 shows the file of Figure 13.1, with the free list, after records 1, 4, and 6 have been deleted.

On insertion of a new record, we use the record pointed to by the header. We change the header pointer to point to the next available record. If no space is available, we add the new record to the end of the file.

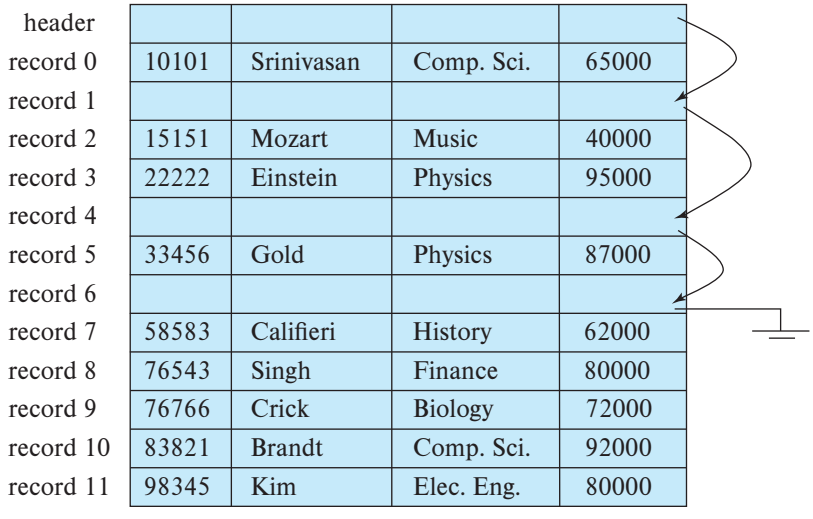


Figure 13.4 File of Figure 13.1, with free list after deletion of records 1, 4, and 6.

Insertion and deletion for files of fixed-length records are simple to implement because the space made available by a deleted record is exactly the space needed to insert a record. If we allow records of variable-length in a file, this match no longer holds. An inserted record may not fit in the space left free by a deleted record, or it may fill only part of that space.

13.2.2 Variable-Length Records

Variable-length records arise in database systems due to several reasons. The most common reason is the presence of variable length fields, such as strings. Other reasons include record types that contain repeating fields such as arrays or multisets, and the presence of multiple record types within a file.

Different techniques for implementing variable-length records exist. Two different problems must be solved by any such technique:

1. How to represent a single record in such a way that individual attributes can be extracted easily, even if they are of variable length
2. How to store variable-length records within a block, such that records in a block can be extracted easily

The representation of a record with variable-length attributes typically has two parts: an initial part with fixed-length information, whose structure is the same for all records of the same relation, followed by the contents of variable-length attributes. Fixed-length attributes, such as numeric values, dates, or fixed-length character strings are allocated as many bytes as required to store their value. Variable-length attributes, such as varchar types, are represented in the initial part of the record by a pair (*offset*, *length*), where *offset* denotes where the data for that attribute begins within the record, and *length* is the length in bytes of the variable-sized attribute. The values for the variable-length attributes are stored consecutively, after the initial fixed-length part of the record. Thus, the initial part of the record stores a fixed size of information about each attribute, whether it is fixed-length or variable-length.

An example of such a record representation is shown in Figure 13.5. The figure shows an *instructor* record whose first three attributes *ID*, *name*, and *dept_name* are variable-length strings, and whose fourth attribute *salary* is a fixed-sized number. We assume that the offset and length values are stored in two bytes each, for a total of 4

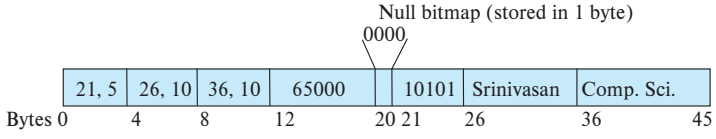


Figure 13.5 Representation of a variable-length record of the *instructor* relation.

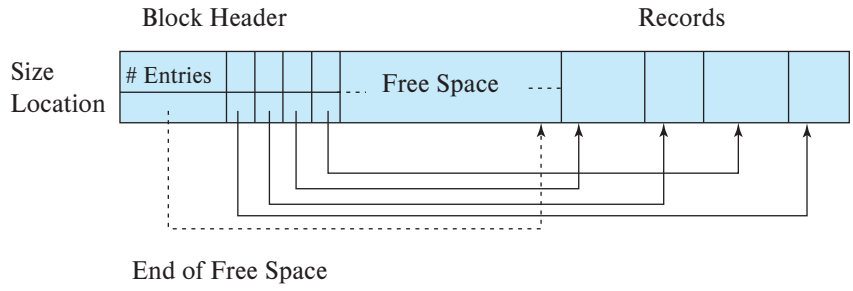


Figure 13.6 Slotted-page structure.

bytes per attribute. The *salary* attribute is assumed to be stored in 8 bytes, and each string takes as many bytes as it has characters.

The figure also illustrates the use of a **null bitmap**, which indicates which attributes of the record have a null value. In this particular record, if the salary were null, the fourth bit of the bitmap would be set to 1, and the *salary* value stored in bytes 12 through 19 would be ignored. Since the record has four attributes, the null bitmap for this record fits in 1 byte, although more bytes may be required with more attributes. In some representations, the null bitmap is stored at the beginning of the record, and for attributes that are null, no data (value, or offset/length) are stored at all. Such a representation would save some storage space, at the cost of extra work to extract attributes of the record. This representation is particularly useful for certain applications where records have a large number of fields, most of which are null.

We next address the problem of storing variable-length records in a block. The **slotted-page structure** is commonly used for organizing records within a block and is shown in Figure 13.6.[2] There is a header at the beginning of each block, containing the following information:

- The number of record entries in the header
- The end of free space in the block
- An array whose entries contain the location and size of each record

The actual records are allocated *contiguously* in the block, starting from the end of the block. The free space in the block is contiguous between the final entry in the header array and the first record. If a record is inserted, space is allocated for it at the end of free space, and an entry containing its size and location is added to the header.

If a record is deleted, the space that it occupies is freed, and its entry is set to *deleted* (its size is set to −1, for example). Further, the records in the block before the deleted record are moved, so that the free space created by the deletion gets occupied, and all

[2]Here, "page" is synonymous with "block."

free space is again between the final entry in the header array and the first record. The end-of-free-space pointer in the header is appropriately updated as well. Records can be grown or shrunk by similar techniques, as long as there is space in the block. The cost of moving the records is not too high, since the size of a block is limited: typical values are around 4 to 8 kilobytes.

The slotted-page structure requires that there be no pointers that point directly to records. Instead, pointers must point to the entry in the header that contains the actual location of the record. This level of indirection allows records to be moved to prevent fragmentation of space inside a block, while supporting indirect pointers to the record.

13.2.3 Storing Large Objects

Databases often store data that can be much larger than a disk block. For instance, an image or an audio recording may be multiple megabytes in size, while a video object may be multiple gigabytes in size. Recall that SQL supports the types **blob** and **clob**, which store binary and character large objects.

Many databases internally restrict the size of a record to be no larger than the size of a block.[3] These databases allow records to logically contain large objects, but they store the large objects separate from the other (short) attributes of records in which they occur. A (logical) pointer to the object is then stored in the record containing the large object.

Large objects may be stored either as files in a file system area managed by the database, or as file structures stored in and managed by the database. In the latter case, such in-database large objects can optionally be represented using B^+-tree file organizations, which we study in Section 14.4.1, to allow efficient access to any location within the object. B^+-tree file organizations permit us to read an entire object, or specified byte ranges in the object, as well as to insert and delete parts of the object.

However, there are some performance issues with storing very large objects in databases. The efficiency of accessing large objects via database interfaces is one concern. A second concern is the size of database backups. Many enterprises periodically create "database dumps," that is, backup copies of their databases; storing large objects in the database can result in a large increase in the size of the database dumps.

Many applications therefore choose to store very large objects, such as video data, outside of the database, in a file system. In such cases, the application may store the file name (usually a path in the file system) as an attribute of a record in the database. Storing data in files outside the database can result in file names in the database pointing to files that do not exist, perhaps because they have been deleted, which results in a form of foreign-key constraint violation. Further, database authorization controls are not applicable to data stored in the file system.

[3]This restriction helps simplify buffer management; as we see in Section 13.5, disk blocks are brought into an area of memory called the buffer before they are accessed. Records larger than a block would get split between blocks, which may be different areas of the buffer, and thus cannot be guaranteed to be in a contiguous area of memory.

Some databases support file system integration with the database, to ensure that constraints are satisfied (for example, deletion of files will be blocked if the database has a pointer to the file), and to ensure that access authorizations are enforced. Files can be accessed both from a file system interface and from the database SQL interface. For example, Oracle supports such integration through its SecureFiles and Database File System features.

13.3 Organization of Records in Files

So far, we have studied how records are represented in a file structure. A relation is a set of records. Given a set of records, the next question is how to organize them in a file. Several of the possible ways of organizing records in files are:

- **Heap file organization**. Any record can be placed anywhere in the file where there is space for the record. There is no ordering of records. Typically, there is either a single file or a set of files for each relation. Heap file organization is discussed in Section 13.3.1.
- **Sequential file organization**. Records are stored in sequential order, according to the value of a "search key" of each record. Section 13.3.2 describes this organization.
- **Multitable clustering file organization**: Generally, a separate file or set of files is used to store the records of each relation. However, in a *multitable clustering file organization*, records of several different relations are stored in the same file, and in fact in the same block within a file, to reduce the cost of certain join operations. Section 13.3.3 describes the multitable clustering file organization.
- **B+-tree file organization**. The traditional sequential file organization described in Section 13.3.2 does support ordered access even if there are insert, delete, and update operations, which may change the ordering of records. However, in the face of a large number of such operations, efficiency of ordered access suffers. We study another way of organizing records, called the *B^+-tree file organization*, in Section 14.4.1. The B^+-tree file organization is related to the B^+-tree index structure described in that chapter and can provide efficient ordered access to records even if there are a large number of insert, delete, or update operations. Further, it supports very efficient access to specific records, based on the search key.
- **Hashing file organization**. A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed. Section 14.5 describes this organization; it is closely related to the indexing structures described in that chapter.

13.3.1 Heap File Organization

In a heap file organization, a record may be stored anywhere in the file corresponding to a relation. Once placed in a particular location, the record is not usually moved.[4]

When a record is inserted in a file, one option for choosing the location is to always add it at the end of the file. However, if records get deleted, it makes sense to use the space thus freed up to store new records. It is important for a database system to be able to efficiently find blocks that have free space, without having to sequentially search through all the blocks of the file.

Most databases use a space-efficient data structure called a **free-space map** to track which blocks have free space to store records. The free-space map is commonly represented by an array containing 1 entry for each block in the relation. Each entry represents a fraction f such that at least a fraction f of the space in the block is free. In PostgreSQL, for example, an entry is 1 byte, and the value stored in the entry must be divided by 256 to get the free-space fraction. The array is stored in a file, whose blocks are fetched into memory, [5] as required. Whenever a record is inserted, deleted, or changed in size, if the occupancy fraction changes enough to affect the entry value, the entry is updated in the free-space map. An example of a free-space map for a file with 16 blocks is shown below. We assume that 3 bits are used to store the occupancy fraction; the value at position i should be divided by 8 to get the free-space fraction for block i.

4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6

For example, a value of 7 indicates that at least $7/8th$ of the space in the block is free.

To find a block to store a new record of a given size, the database can scan the free-space map to find a block that has enough free space to store that record. If there is no such block, a new block is allocated for the relation.

While such a scan is much faster than actually fetching blocks to find free space, it can still be very slow for large files. To further speed up the task of locating a block with sufficient free space, we can create a second-level free-space map, which has, say 1 entry for every 100 entries of the main free-space map. That 1 entry stores the maximum value amongst the 100 entries in the main free-space map that it corresponds to. The free-space map below is a second level free-space map for our earlier example, with 1 entry for every 4 entries in the main free-space map.

4	7	2	6

[4] Records may be occasionally moved, for example, if the database sorts the records of the relation; but note that even if the relation is reordered by sorting, subsequent insertions and updates may result in the records no longer being ordered.

[5] Via the database buffer, which we discuss in Section 13.5.

With 1 entry for every 100 entries in the main free-space map, a scan of the second-level free space map would take only 1/100th of the time to scan the main free-space map; once a suitable entry indicating enough free space is found there, its corresponding 100 entries in the main free-space map can be examined to find a block with sufficient free space. Such a block must exist, since the second-level free-space map entry stores the maximum of the entries in the main free-space map. To deal with very large relations, we can create more levels beyond the second level, using the same idea.

Writing the free-space map to disk every time an entry in the map is updated would be very expensive. Instead, the free-space map is written periodically; as a result, the free-space map on disk may be outdated, and when a database starts up, it may get outdated data about available free space. The free-space map may, as a result, claim a block has free space when it does not; such an error will be detected when the block is fetched, and can be dealt with by a further search in the free-space map to find another block. On the other hand, the free-space map may claim that a block does not have free space when it does; generally this will not result in any problem other than unused free space. To fix any such errors, the relation is scanned periodically and the free-space map recomputed and written to disk.

13.3.2 Sequential File Organization

A **sequential file** is designed for efficient processing of records in sorted order based on some search key. A **search key** is any attribute or set of attributes; it need not be the primary key, or even a superkey. To permit fast retrieval of records in search-key order, we chain together records by pointers. The pointer in each record points to the next record in search-key order. Furthermore, to minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

Figure 13.7 shows a sequential file of *instructor* records taken from our university example. In that example, the records are stored in search-key order, using *ID* as the search key.

The sequential file organization allows records to be read in sorted order; that can be useful for display purposes, as well as for certain query-processing algorithms that we shall study in Chapter 15.

It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion. We can manage deletion by using pointer chains, as we saw previously. For insertion, we apply the following two rules:

1. Locate the record in the file that comes before the record to be inserted in search-key order.
2. If there is a free record (i.e., space left after a deletion) within the same block as this record, insert the new record there. Otherwise, insert the new record in

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

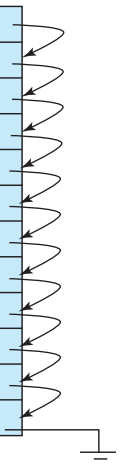


Figure 13.7 Sequential file for *instructor* records.

an *overflow block*. In either case, adjust the pointers so as to chain together the records in search-key order.

Figure 13.8 shows the file of Figure 13.7 after the insertion of the record (32222, Verdi, Music, 48000). The structure in Figure 13.8 allows fast insertion of new records, but it forces sequential file-processing applications to process records in an order that does not match the physical order of the records.

If relatively few records need to be stored in overflow blocks, this approach works well. Eventually, however, the correspondence between search-key order and physical order may be totally lost over a period of time, in which case sequential processing will become much less efficient. At this point, the file should be **reorganized** so that it is once again physically in sequential order. Such reorganizations are costly and must be done during times when the system load is low. The frequency with which reorganizations are needed depends on the frequency of insertion of new records. In the extreme case in which insertions rarely occur, it is possible always to keep the file in physically sorted order. In such a case, the pointer field in Figure 13.7 is not needed.

The B^+-tree file organization, which we describe in Section 14.4.1, provides efficient ordered access even if there are many inserts, deletes, and updates, without requiring expensive reorganizations.

13.3.3 Multitable Clustering File Organization

Most relational database systems store each relation in a separate file, or a separate set of files. Thus, each file, and as a result, each block, stores records of only one relation, in such a design.

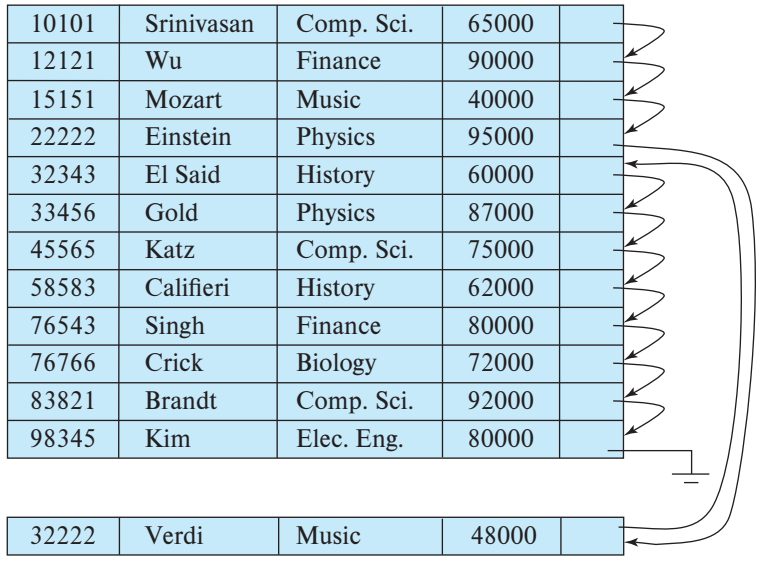


Figure 13.8 Sequential file after an insertion.

However, in some cases it can be useful to store records of more than one relation in a single block. To see the advantage of storing records of multiple relations in one block, consider the following SQL query for the university database:

select *dept_name*, *building*, *budget*, *ID*, *name*, *salary*
from *department* **natural join** *instructor*;

This query computes a join of the *department* and *instructor* relations. Thus, for each tuple of *department*, the system must locate the *instructor* tuples with the same value for *dept_name*. Ideally, these records will be located with the help of *indices*, which we shall discuss in Chapter 14. Regardless of how these records are located, however, they need to be transferred from disk into main memory. In the worst case, each record will reside on a different block, forcing us to do one block read for each record required by the query.

As a concrete example, consider the *department* and *instructor* relations of Figure 13.9 and Figure 13.10, respectively (for brevity, we include only a subset of the tuples

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Physics	Watson	70000

Figure 13.9 The *department* relation.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

Figure 13.10 The *instructor* relation.

of the relations we have used thus far). In Figure 13.11, we show a file structure designed for the efficient execution of queries involving the natural join of *department* and *instructor*. All the *instructor* tuples for a particular *dept_name* are stored near the *department* tuple for that *dept_name*. We say that the two relations are clustered on the key *dept_name*. We assume that each record contains the identifier of the relation to which it belongs, although this is not shown in Figure 13.11.

Although not depicted in the figure, it is possible to store the value of the *dept_name* attribute, which defines the clustering, only once for a group of tuples (from both relations), reducing storage overhead.

This structure allows for efficient processing of the join. When a tuple of the *department* relation is read, the entire block containing that tuple is copied from disk into main memory. Since the corresponding *instructor* tuples are stored on the disk near the *department* tuple, the block containing the *department* tuple contains tuples of the *instructor* relation needed to process the query. If a department has so many instructors that the *instructor* records do not fit in one block, the remaining records appear on nearby blocks.

A **multitable clustering file organization** is a file organization, such as that illustrated in Figure 13.11, that stores related records of two or more relations in each block.[6] The **cluster key** is the attribute that defines which records are stored together; in our preceding example, the cluster key is *dept_name*.

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

Figure 13.11 Multitable clustering file structure.

[6]Note that the word *cluster* is often used to refer to a group of machines that together constitute a parallel database; that use of the word *cluster* is unrelated to the concept of multitable clustering.

Although a multitable clustering file organization can speed up certain join queries, it can result in slowing processing of other types of queries. For example, in our preceding example,

```
select *
from department;
```

requires more block accesses than it did in the scheme under which we stored each relation in a separate file, since each block now contains significantly fewer *department* records. To locate efficiently all tuples of the *department* relation within a particular block, we can chain together all the records of that relation using pointers; however, the number of blocks read does not get affected by using such chains.

When multitable clustering is to be used depends on the types of queries that the database designer believes to be most frequent. Careful use of multitable clustering can produce significant performance gains in query processing.

Multitable clustering is supported by the Oracle database system. Clusters can be created by using a **create cluster** command, with a specified cluster key. An extension of the **create table** command can be used to specify that a relation is to be stored in a specific cluster, with a particular attribute used as the cluster key. Multiple relations can thus be allocated to a cluster.

13.3.4 Partitioning

Many databases allow the records in a relation to be partitioned into smaller relations that are stored separately. Such **table partitioning** is typically done on the basis of an attribute value; for example, records in a *transaction* relation in an accounting database may be partitioned by year into smaller relations corresponding to each year, such as *transaction_2018*, *transaction_2019*, and so on. Queries can be written based on the *transaction* relation but are translated into queries on the year-wise relations. Most accesses are to records of the current year and include a selection based on the year. Query optimizers can rewrite such a query to only access the smaller relation corresponding to the requested year, and they can avoid reading records corresponding to other years. For example, a query

```
select *
from transaction
where year=2019
```

would only access the relation *transaction_2019*, ignoring the other relations, while a query without the selection condition would read all the relations.

The cost of some operations, such as finding free space for a record, increase with relation size; by reducing the size of each relation, partitioning helps reduce such overheads. Partitioning can also be used to store different parts of a relation on different

storage devices; for example, in the year 2019, *transaction_2018* and earlier year transactions can which are infrequently accessed could be stored on magnetic disk, while *transaction_2019* could be stored on SSD, for faster access.

13.4 Data-Dictionary Storage

So far, we have considered only the representation of the relations themselves. A relational database system needs to maintain data *about* the relations, such as the schema of the relations. In general, such "data about data" are referred to as **metadata**.

Relational schemas and other metadata about relations are stored in a structure called the **data dictionary** or **system catalog**. Among the types of information that the system must store are these:

- Names of the relations
- Names of the attributes of each relation
- Domains and lengths of attributes
- Names of views defined on the database, and definitions of those views
- Integrity constraints (e.g., key constraints)

In addition, many systems keep the following data on users of the system:

- Names of users, the default schemas of the users, and passwords or other information to authenticate users
- Information about authorizations for each user

Further, the database may store statistical and descriptive data about the relations and attributes, such as the number of tuples in each relation, or the number of distinct values for each attribute.

The data dictionary may also note the storage organization (heap, sequential, hash, etc.) of relations, and the location where each relation is stored:

- If relations are stored in operating system files, the dictionary would note the names of the file (or files) containing each relation.
- If the database stores all relations in a single file, the dictionary may note the blocks containing records of each relation in a data structure such as a linked list.

In Chapter 14, in which we study indices, we shall see a need to store information about each index on each of the relations:

- Name of the index

- Name of the relation being indexed
- Attributes on which the index is defined
- Type of index formed

All this metadata information constitutes, in effect, a miniature database. Some database systems store such metadata by using special-purpose data structures and code. It is generally preferable to store the data about the database as relations in the database itself. By using database relations to store system metadata, we simplify the overall structure of the system and harness the full power of the database for fast access to system data.

The exact choice of how to represent system metadata by relations must be made by the system designers. We show the schema diagram of a toy data dictionary in Figure 13.12, storing part of the information mentioned above. The schema is only illustrative; real implementations store far more information than what the figure shows. Read the manuals for whichever database you use to see what system metadata it maintains.

In the metadata representation shown, the attribute *index_attributes* of the relation *Index_metadata* is assumed to contain a list of one or more attributes, which can be represented by a character string such as "*dept_name*, *building*". The *Index_metadata* relation is thus not in first normal form; it can be normalized, but the preceding representation is likely to be more efficient to access. The data dictionary is often stored in a nonnormalized form to achieve fast access.

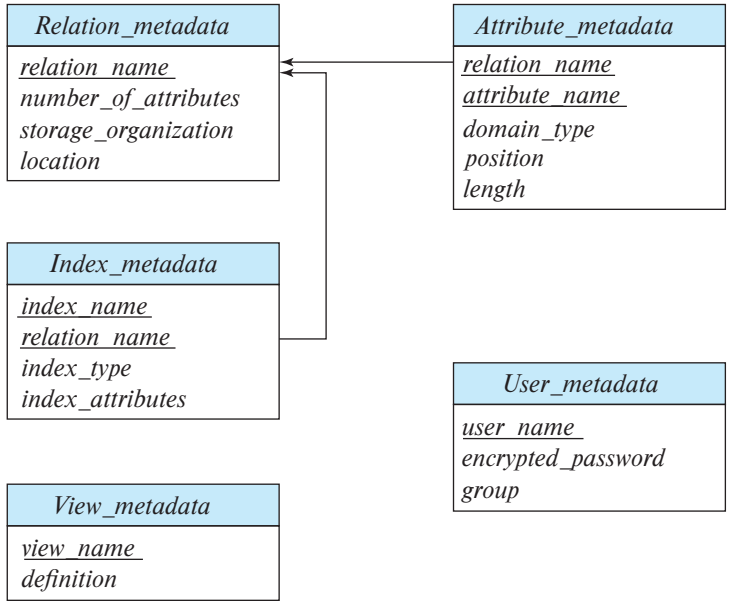


Figure 13.12 Relational schema representing part of the system metadata.

Whenever the database system needs to retrieve records from a relation, it must first consult the *Relation_metadata* relation to find the location and storage organization of the relation, and then fetch records using this information.

However, the storage organization and location of the *Relation_metadata* relation itself must be recorded elsewhere (e.g., in the database code itself, or in a fixed location in the database), since we need this information to find the contents of *Relation_metadata*.

Since system metadata are frequently accessed, most databases read it from the database into in-memory data structures that can be accessed very efficiently. This is done as part of the database startup, before the database starts processing any queries.

13.5 Database Buffer

The size of main memory on servers has increased greatly over the years, and many medium-sized databases can fit in memory. However, a server has many demands on its memory, and the amount of memory it can give to a database may be much smaller than the database size even for medium-sized databases. And many large databases are much larger than the available memory on servers.

Thus, even today, database data reside primarily on disk in most databases, and they must be brought into memory to be read or updated; updated data blocks must be written back to disk subsequently.

Since data access from disk is much slower than in-memory data access, a major goal of the database system is to minimize the number of block transfers between the disk and memory. One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory. The goal is to maximize the chance that, when a block is accessed, it is already in main memory, and, thus, no disk access is required.

Since it is not possible to keep all blocks in main memory, we need to manage the allocation of the space available in main memory for the storage of blocks. The **buffer** is that part of main memory available for storage of copies of disk blocks. There is always a copy kept on disk of every block, but the copy on disk may be a version of the block older than the version in the buffer. The subsystem responsible for the allocation of buffer space is called the **buffer manager**.

13.5.1 Buffer Manager

Programs in a database system make requests (i.e., calls) on the buffer manager when they need a block from disk. If the block is already in the buffer, the buffer manager passes the address of the block in main memory to the requester. If the block is not in the buffer, the buffer manager first allocates space in the buffer for the block, throwing out some other block, if necessary, to make space for the new block. The thrown-out block is written back to disk only if it has been modified since the most recent time that it was written to the disk. Then, the buffer manager reads in the requested block from the disk to the buffer, and passes the address of the block in main memory to the

requester. The internal actions of the buffer manager are transparent to the programs that issue disk-block requests.

If you are familiar with operating-system concepts, you will note that the buffer manager appears to be nothing more than a virtual-memory manager, like those found in most operating systems. One difference is that the size of the database might be larger than the hardware address space of a machine, so memory addresses are not sufficient to address all disk blocks. Further, to serve the database system well, the buffer manager must use techniques more sophisticated than typical virtual-memory management schemes:

13.5.1.1 Buffer replacement strategy

When there is no room left in the buffer, a block must be **evicted**, that is, removed, from the buffer before a new one can be read in. Most operating systems use a **least recently used** (**LRU**) scheme, in which the block that was referenced least recently is written back to disk and is removed from the buffer. This simple approach can be improved on for database applications(see Section 13.5.2).

13.5.1.2 Pinned blocks

Once a block has been brought into the buffer, a database process can read the contents of the block from the buffer memory. However, while the block is being read, if a concurrent process evicts the block and replaces it with a different block, the reader that was reading the contents of the old block will see incorrect data; if the block was being written when it was evicted, the writer would end up damaging the contents of the replacement block.

It is therefore important that before a process reads data from a buffer block, it ensures that the block will not get evicted. To do so, the process executes a **pin** operation on the block; the buffer manager never evicts a pinned block. When it has finished reading data, the process should execute an **unpin** operation, allowing the block to be evicted when required. The database code should be written carefully to avoid pinning too many blocks: if all the blocks in the buffer get pinned, no blocks can be evicted, and no other block can be brought into the buffer. If this happens, the database will be unable to carry out any further processing!

Multiple processes can read data from a block that is in the buffer. Each of them is required to execute a pin operation before accessing data, and an unpin after completing access. The block cannot be evicted until all processes that have executed a pin have then executed an unpin operation. A simple way to ensure this property is to keep a **pin count** for each buffer block. Each pin operation increments the count, and an unpin operation decrements the count. A page can be evicted only if the pin count equals 0.

13.5.1.3 Shared and Exclusive Locks on Buffer

A process that adds or deletes a tuple from a page may need to move the page contents around; during this period, no other process should read the contents of the page, since

they may be inconsistent. Database buffer managers allow processes to get shared and exclusive locks on the buffer.

We will study locking in more detail in Chapter 18, but here we discuss a limited form of locking in the context of the buffer manager. The locking system provided by the buffer manager allows a database process to lock a buffer block either in shared mode or in exclusive mode before accessing the block, and to release the lock later, after the access is completed. Here are the rules for locking:

- Any number of processes may have shared locks on a block at the same time.
- Only one process is allowed to get an exclusive lock at a time, and further when a process has an exclusive lock, no other process may have a shared lock on the block. Thus, an exclusive lock can be granted only when no other process has a lock on the buffer block.
- If a process requests an exclusive lock when a block is already locked in shared or exclusive mode, the request is kept pending until all earlier locks are released.
- If a process requests a shared lock when a block is not locked, or already shared locked, the lock may be granted; however, if another process has an exclusive lock, the shared lock is granted only after the exclusive lock has been released.

Locks are acquired and released as follows:

- Before carrying out any operation on a block, a process must pin the block as we saw earlier. Locks are obtained subsequently and must be released before unpinning the block.
- Before reading data from a buffer block, a process must get a shared lock on the block. When it is done reading the data, the process must release the lock.
- Before updating the contents of a buffer block, a process must get an exclusive lock on the block; the lock must be released after the update is complete.

These rules ensure that a block cannot be updated while another process is reading it, and conversely, a block cannot be read while another process is updating it. These rules are required for safety of buffer access; however, to protect a database system from problems due to concurrent access, these steps are not sufficient: further steps need to be taken. These are discussed further in Chapter 17 and Chapter 18.

13.5.1.4 Output of blocks

It is possible to output a block only when the buffer space is needed for another block. However, it makes sense to not wait until the buffer space is needed, but to rather write out updated blocks ahead of such a need. Then, when space is required in the buffer,

a block that has already been written out can be evicted, provided it is not currently pinned.

However, for the database system to be able to recover from crashes (Chapter 19), it is necessary to restrict those times when a block may be written back to disk. For instance, most recovery systems require that a block should not be written to disk while an update on the block is in progress. To enforce this requirement, a process that wishes to write the block to disk must obtain a shared lock on the block.

Most databases have a process that continually detects updated blocks and writes them back to disk.

13.5.1.5 Forced output of blocks

There are situations in which it is necessary to write a block to disk, to ensure that certain data on disk are in a consistent state. Such a write is called a **forced output** of a block. We shall see the reason for forced output in Chapter 19.

Memory contents and thus buffer contents are lost in a crash, whereas data on disk (usually) survive a crash. Forced output is used in conjunction with a logging mechanism to ensure that when a transaction that has performed updates commits, enough data has been written to disk to ensure the updates of the transaction are not lost. How exactly this is done is covered in detail in Chapter 19.

13.5.2 Buffer-Replacement Strategies

The goal of a replacement strategy for blocks in the buffer is to minimize accesses to the disk. For general-purpose programs, it is not possible to predict accurately which blocks will be referenced. Therefore, operating systems use the past pattern of block references as a predictor of future references. The assumption generally made is that blocks that have been referenced recently are likely to be referenced again. Therefore, if a block must be replaced, the least recently referenced block is replaced. This approach is called the **least recently used** (**LRU**) block-replacement scheme.

LRU is an acceptable replacement scheme in operating systems. However, a database system is able to predict the pattern of future references more accurately than an operating system. A user request to the database system involves several steps. The database system is often able to determine in advance which blocks will be needed by looking at each of the steps required to perform the user-requested operation. Thus, unlike operating systems, which must rely on the past to predict the future, database systems may have information regarding at least the short-term future.

To illustrate how information about future block access allows us to improve the LRU strategy, consider the processing of the SQL query:

```
select *
from instructor natural join department;
```

Assume that the strategy chosen to process this request is given by the pseudocode program shown in Figure 13.13. (We shall study other, more efficient, strategies in Chapter 15.)

Assume that the two relations of this example are stored in separate files. In this example, we can see that, once a tuple of *instructor* has been processed, that tuple is not needed again. Therefore, once processing of an entire block of *instructor* tuples is completed, that block is no longer needed in main memory, even though it has been used recently. The buffer manager should be instructed to free the space occupied by an *instructor* block as soon as the final tuple has been processed. This buffer-management strategy is called the **toss-immediate** strategy.

Now consider blocks containing *department* tuples. We need to examine every block of *department* tuples once for each tuple of the *instructor* relation. When processing of a *department* block is completed, we know that that block will not be accessed again until all other *department* blocks have been processed. Thus, the most recently used *department* block will be the final block to be re-referenced, and the least recently used *department* block is the block that will be referenced next. This assumption set is the exact opposite of the one that forms the basis for the LRU strategy. Indeed, the optimal strategy for block replacement for the above procedure is the **most recently used** (**MRU**) strategy. If a *department* block must be removed from the buffer, the MRU strategy chooses the most recently used block (blocks are not eligible for replacement while they are being used).

```
for each tuple i of instructor do
    for each tuple d of department do
        if i[dept_name] = d[dept_name]
        then begin
                let x be a tuple defined as follows:
                x[ID] := i[ID]
                x[dept_name] := i[dept_name]
                x[name] := i[name]
                x[salary] := i[salary]
                x[building] := d[building]
                x[budget] := d[budget]
                include tuple x as part of result of instructor ⋈ department
             end
    end
end
```

Figure 13.13 Procedure for computing join.

For the MRU strategy to work correctly for our example, the system must pin the *department* block currently being processed. After the final *department* tuple has been processed, the block is unpinned, and it becomes the most recently used block.

In addition to using knowledge that the system may have about the request being processed, the buffer manager can use statistical information about the probability that a request will reference a particular relation. For example, the data dictionary, which we saw in Section 13.4, is one of the most frequently accessed parts of the database, since the processing of every query needs to access the data dictionary. Thus, the buffer manager should try not to remove data-dictionary blocks from main memory, unless other factors dictate that it do so. In Chapter 14, we discuss indices for files. Since an index for a file may be accessed more frequently than the file itself, the buffer manager should, in general, not remove index blocks from main memory if alternatives are available.

The ideal database block-replacement strategy needs knowledge of the database operations—both those being performed and those that will be performed in the future. No single strategy is known that handles all the possible scenarios well. Indeed, a surprisingly large number of database systems use LRU, despite that strategy's faults. The practice questions and exercises explore alternative strategies.

The strategy that the buffer manager uses for block replacement is influenced by factors other than the time at which the block will be referenced again. If the system is processing requests by several users concurrently, the concurrency-control subsystem (Chapter 18) may need to delay certain requests, to ensure preservation of database consistency. If the buffer manager is given information from the concurrency-control subsystem indicating which requests are being delayed, it can use this information to alter its block-replacement strategy. Specifically, blocks needed by active (nondelayed) requests can be retained in the buffer at the expense of blocks needed by the delayed requests.

The crash-recovery subsystem (Chapter 19) imposes stringent constraints on block replacement. If a block has been modified, the buffer manager is not allowed to write back the new version of the block in the buffer to disk, since that would destroy the old version. Instead, the block manager must seek permission from the crash-recovery subsystem before writing out a block. The crash-recovery subsystem may demand that certain other blocks be force-output before it grants permission to the buffer manager to output the block requested. In Chapter 19, we define precisely the interaction between the buffer manager and the crash-recovery subsystem.

13.5.3 Reordering of Writes and Recovery

Database buffers allow writes to be performed in-memory and output to disk at a later time, possibly in an order different from the order in which the writes were performed. File systems, too, routinely reorder write operations. However, such reordering can lead to inconsistent data on disk in the event of a system crash.

To understand the problem in the context of a file system, suppose that a file system uses a linked list to track which blocks are part of a file. Suppose also that it inserts a new node at the end of the list by first writing the data for the new node, then updating the pointer from the previous node. Suppose further that the writes were reordered, so the pointer was updated first, and the system crashes before the new node is written. The contents of the node would then be whatever happened to be on that disk earlier, resulting in a corrupted data structure.

To deal with the possibility of such data structure corruption, earlier-generation file systems had to perform a *file system consistency check* on system restart, to ensure that the data structures were consistent. And if they were not, extra steps had to be taken to restore them to consistency. These checks resulted in long delays in system restart after a crash, and the delays became worse as disk systems grew to higher capacities.

The file system can avoid inconsistencies in many cases if it writes updates to metadata in a carefully chosen order. But doing so would mean that optimizations such as disk arm scheduling cannot be done, affecting the efficiency of the update. If a non-volatile write buffer were available, it could be used to perform the writes in order to non-volatile RAM and later reorder the writes when writing them to disk.

However, most disks do not come with a non-volatile write buffer; instead, modern file systems assign a disk for storing a log of the writes in the order that they are performed. Such a disk is called a **log disk**. For each write, the log contains the block number to be written to, and the data to be written, in the order in which the writes were performed. All access to the log disk is sequential, essentially eliminating seek time, and several consecutive blocks can be written at once, making writes to the log disk several times faster than random writes. As before, the data have to be written to their actual location on disk as well, but the write to the actual location can be done later; the writes can be reordered to minimize disk-arm movement.

If the system crashes before some writes to the actual disk location have been completed, when the system comes back up it reads the log disk to find those writes that had not been completed and carries them out then. After the writes have been performed, the records are deleted from the log disk.

File systems that support log disks as above are called **journaling file systems**. Journaling file systems can be implemented even without a separate log disk, keeping data and the log on the same disk. Doing so reduces the monetary cost at the expense of lower performance.

Most modern file systems implement journaling and use the log disk when writing file system metadata such as file allocation information. Journaling file systems allow quick restart without the need for such file system consistency checks.

However, writes performed by applications are usually not written to the log disk. Database systems instead implement their own forms of logging, which we study in Chapter 19, to ensure that the contents of a database can be safely recovered in the event of a failure, even if writes were reordered.

13.6 Column-Oriented Storage

Databases traditionally store all attributes of a tuple together in a record, and tuples are stored in a file as we have just seen. Such a storage layout is referred to as a *row-oriented storage*.

In contrast, in **column-oriented storage**, also called a **columnar storage**, each attribute of a relation is stored separately, with values of the attribute from successive tuples stored at successive positions in the file. Figure 13.14 shows how the *instructor* relation would be stored in column-oriented storage, with each attribute stored separately.

In the simplest form of column-oriented storage, each attribute is stored in a separate file. Further, each file is *compressed*, to reduce its size. (We discuss more complex schemes that store columns consecutively in a single file later in this section.)

If a query needs to access the entire contents of the *i*th row of a table, the values at the *i*th position in each of the columns are retrieved and used to reconstruct the row. Column-oriented storage thus has the drawback that fetching multiple attributes of a single tuple requires multiple I/O operations. Thus, it is not suitable for queries that fetch multiple attributes from a few rows of a relation.

However, column-oriented storage is well suited for data analysis queries, which process many rows of a relation, but often only access some of the attributes. The reasons are as follows:

- **Reduced I/O**. When a query needs to access only a few attributes of a relation with a large number of attributes, the remaining attributes need not be fetched from disk into memory. In contrast, in row-oriented storage, irrelevant attributes are fetched into memory from disk. The reduction in I/O can lead to significant reduction in query execution cost.

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 13.14 Columnar representation of the *instructor* relation.

- **Improved CPU cache performance**. When the query processor fetches the contents of a particular attribute, with modern CPU architectures multiple consecutive bytes, called a cache line, are fetched from memory to CPU cache. If these bytes are accessed later, access is much faster if they are in cache than if they have to be fetched from main memory. However, if these adjacent bytes contain values for attributes that are not be needed by the query, fetching them into cache wastes memory bandwidth and uses up cache space that could have been used for other data. Column-oriented storage does not suffer from this problem, since adjacent bytes are from the same column, and data analysis queries usually access all these values consecutively.
- **Improved compression**. Storing values of the same type together significantly increases the effectiveness of compression, when compared to compressing data stored in row format; in the latter case, adjacent attributes are of different types, reducing the efficiency of compression. Compression significantly reduces the time taken to retrieve data from disk, which is often the highest-cost component for many queries. If the compressed files are stored in memory, the in-memory storage space is also reduced correspondingly, which is particularly important since main memory is significantly more expensive than disk storage.
- **Vector processing**. Many modern CPU architectures support **vector processing**, which allows a CPU operation to be applied in parallel on a number of elements of an array. Storing data columnwise allows vector processing of operations such as comparing an attribute with a constant, which is important for applying selection conditions on a relation. Vector processing can also be used to compute an aggregate of multiple values in parallel, instead of aggregating the values one at a time.

As a result of these benefits, column-oriented storage is increasingly used in data-warehousing applications, where queries are primarily data analysis queries. It should be noted that indexing and query processing techniques need to be carefully designed to get the performance benefits of column-oriented storage. We outline indexing and query processing techniques based on bitmap representations, which are well suited to column-oriented storage, in Section 14.9; further details are provided in Section 24.3.

Databases that use column-oriented storage are referred to as **column stores**, while databases that use row-oriented storage are referred to as **row stores**.

It should be noted that column-oriented storage does have several drawbacks, which make them unsuitable for transaction processing.

- **Cost of tuple reconstruction**. As we saw earlier, reconstructing a tuple from the individual columns can be expensive, negating the benefits of columnar representation if many columns need to be reconstructed. While tuple reconstruction is common in transaction-processing applications, data analysis applications usually

output only a few columns out of many that are stored in "fact tables" in data warehouses.

- **Cost of tuple deletion and update**. Deleting or updating a single tuple in a compressed representation would require rewriting the entire sequence of tuples that are compressed as one unit. Since updates and deletes are common in transaction-processing applications, column-oriented storage would result in a high cost for these operations if a large number of tuples were compressed as one unit.

  In contrast, data-warehousing systems typically do not support updates to tuples, and instead support only insert of new tuples and bulk deletes of a large number of old tuples at a time. Inserts are done at the end of the relation representation, that is, new tuples are appended to the relation. Since small deletes and updates do not occur in a data warehouse, large sequences of attribute values can be stored and compressed together as one unit, allowing for better compression than with small sequences.

- **Cost of decompression**. Fetching data from a compressed representation requires *decompression*, which in the simplest compressed representations requires reading all the data from the beginning of a file. Transaction processing queries usually only need to fetch a few records; sequential access is expensive in such a scenario, since many irrelevant records may have to be decompressed to access a few relevant records.

Since data analysis queries tend to access many consecutive records, the time spent on decompression is typically not wasted. However, even data analysis queries do not need to access records that fail selection conditions, and attributes of such records should be skipped to reduce disk I/O.

To allow skipping of attribute values from such records, compressed representations for column stores allow decompression to start at any of a number of points in the file, skipping earlier parts of the file. This could be done by starting compression afresh after every 10,000 values (for example). By keeping track of where in the file the data start for each group of 10,000 values, it is possible to access the ith value by going to the start of the group $\lfloor i/10000 \rfloor$ and starting decompression from there.

ORC and Parquet are columnar file representations used in many big-data processing applications. In ORC, a row-oriented representation is converted to column-oriented representation as follows: A sequence of tuples occupying several hundred megabytes is broken up into a columnar representation called a **stripe**. An ORC file contains several such stripes, with each stripe occupying around 250 megabytes.

Figure 13.15 illustrates some details of the ORC file format. Each stripe has index data followed by row data. The row data area stores a compressed representation of the sequence of value for the first column, followed by the compressed representation of the second column, and so on. The index data region of a stripe stores for each attribute the starting point within the stripe for each group of (say) 10,000 values of

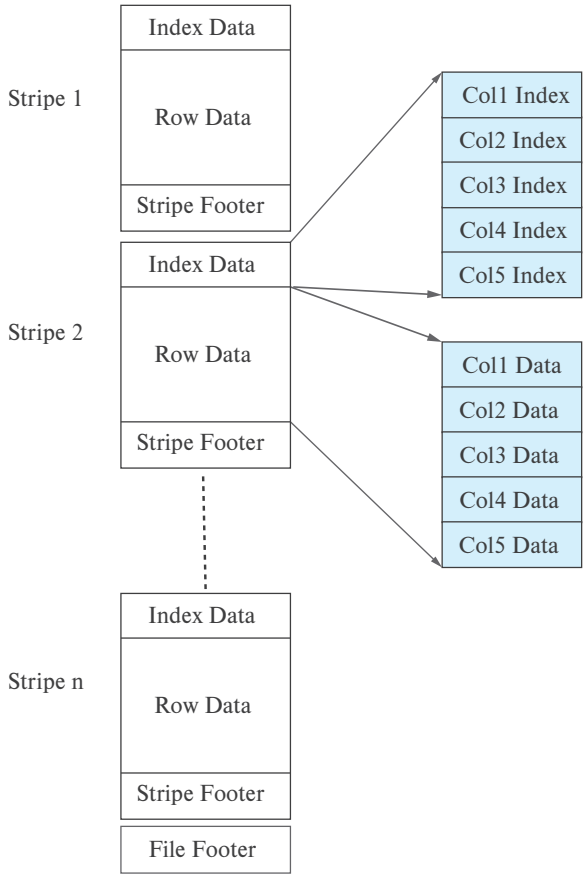


Figure 13.15 Columnar data representation in the ORC file format.

that attribute.[7] The index is useful for quick access to a desired tuple or sequence of tuples; the index also allows queries containing selections to skip groups of tuples if the query determines that no tuple in those groups satisfies the selections. ORC files store several other pieces of information in the stripe footer and file footer, which we skip here.

Some column-store systems allow groups of columns that are often accessed together to be stored together, instead of breaking up each column into a different file. Such systems thus allow a spectrum of choices that range from pure column-oriented storage, where every column is stored separately, to pure row-oriented storage, where all columns are stored together. The choice of which attributes to store together depends on the query workload.

[7] ORC files have some other information that we ignore here.

Some of the benefits of column-oriented storage can be obtained even in a row-oriented storage system by logically decomposing a relation into multiple relations. For example, the *instructor* relation could be decomposed into three relations, containing (*ID*, *name*), (*ID*, *dept_name*) and (*ID*, *salary*), respectively. Then, queries that access only the name do not have to fetch the *dept_name* and *salary* attributes. However, in this case the same *ID* attribute occurs in three tuples, resulting in wasted space.

Some database systems use a column-oriented representation for data within a disk block, without using compression.[8] Thus, a block contains data for a set of tuples, and all attributes for that set of tuples are stored in the same block. Such a scheme is useful in transaction-processing systems, since retrieving all attribute values does not require multiple disk accesses. At the same time, using column-oriented storage within the block provides the benefits of more efficient memory access and cache usage, as well as the potential for using vector processing on the data. However, this scheme does not allow irrelevant disk blocks to be skipped when only a few attributes are retrieved, nor does it give the benefits of compression. Thus, it represents a point in the space between pure row-oriented storage and pure column-oriented storage.

Some databases, such as SAP HANA support two underlying storage systems, one a row-oriented one designed for transaction processing, and the second a column-oriented one, designed for data analysis. Tuples are normally created in the row-oriented store but are later migrated to the column-oriented store when they are no longer likely to be accessed in a row-oriented manner. Such systems are called **hybrid row/column stores**.

In other cases, applications store transactional data in a row-oriented store, but copy data periodically (e.g., once a day or a few times a day) to a data warehouse, which may use a column-oriented storage system.

Sybase IQ was one of the early products to use column-oriented storage, but there are now several research projects and companies that have developed database systems based on column stores, including C-Store, Vertica, MonetDB, Vectorwise, among others. See Further Reading at the end of the chapter for more details.

13.7 Storage Organization in Main-Memory Databases

Today, main-memory sizes are large enough, and main memory is cheap enough, that many organizational databases fit entirely in memory. Such large main memories can be used by allocating a large amount of memory to the database buffer, which will allow the entire database to be loaded into buffer, avoiding disk I/O operations for reading data; updated blocks still have to be written back to disk for persistence. Thus, such a setup would provide much better performance than one where only part of the database can fit in the buffer.

[8] Compression can be applied to data in a disk block, but accessing them requires decompression, and the decompressed data may no longer fit in a block. Significant changes need to be made to the database code, including buffer management, to handle such issues.

However, if the entire database fits in memory, performance can be improved significantly by tailoring the storage organization and database data structures to exploit the fact that data are fully in memory. A **main-memory database** is a database where all data reside in memory; main-memory database systems are typically designed to optimize performance by making use of this fact. In particular, they do away entirely with the buffer manager.

As an example of optimizations that can be done with memory-resident data, consider the cost of accessing a record, given a record pointer. With disk-based databases, records are stored in blocks, and pointers to records consist of a block identifier and an offset or slot number within the block. Following such a record pointer requires checking if the block is in the buffer (usually done by using an in-memory hash index), and if it is, finding where in the buffer it is located. If it is not in buffer, it has to be fetched. All these actions take a significant number of CPU cycles.

In contrast, in a main-memory database, it is possible to keep direct pointers to records in memory, and accessing a record is just an in-memory pointer traversal, which is a very efficient operation. This is possible as long as records are not moved around. Indeed, one reason for such movement, namely loading into buffer and eviction from buffer, is no longer an issue.

If records are stored in a slotted-page structure within a block, records may move within a block as other records are deleted or resized. Direct pointers to records are not possible in that case, although records can be accessed with one level of indirection through the entries in the slotted page header. Locking of the block may be required to ensure that a record does not get moved while another process is reading its data. To avoid these overheads, many main-memory databases do not use a slotted-page structure for allocating records. Instead they directly allocate records in main memory, and ensure that records never get moved due to updates to other records. However, a problem with direct allocation of records is that memory may get fragmented if variable sized records are repeatedly inserted and deleted. The database must ensure that main memory does not get fragmented over time, either by using suitably designed memory management schemes or by periodically performing compaction of memory; the latter scheme will result in record movement, but it can be done without acquiring locks on blocks.

If a column-oriented storage scheme is used in main memory, all the values of a column can be stored in consecutive memory locations. However, if there are appends to the relation, ensuring contiguous allocation would require existing data be reallocated. To avoid this overhead, the logical array for a column may be divided into multiple physical arrays. An indirection table stores pointers to all the physical arrays. This scheme is depicted in Figure 13.16. To find the ith element of a logical array, the indirection table is used to locate the physical array containing the ith element, and then an appropriate offset is computed and looked up within that physical array.

There are other ways in which processing can be optimized with main-memory databases, as we shall see in later chapters.

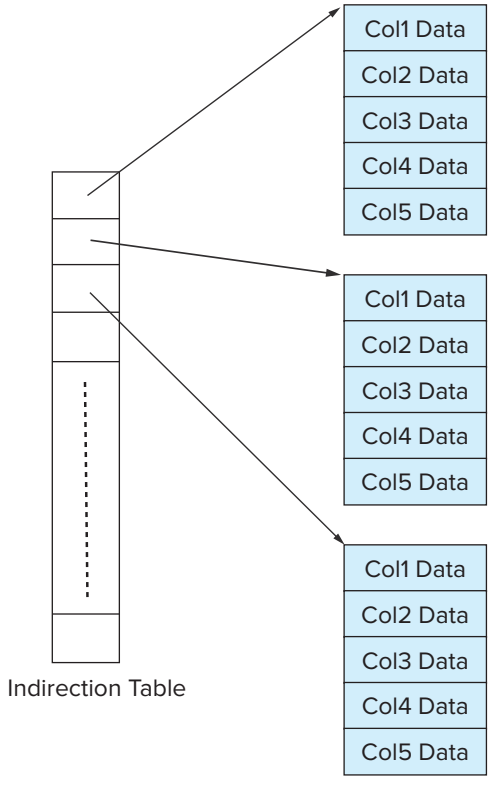


Figure 13.16 In-memory columnar data representation.

13.8 Summary

- We can organize a file logically as a sequence of records mapped onto disk blocks. One approach to mapping the database to files is to use several files, and to store records of only one fixed length in any given file. An alternative is to structure files so that they can accommodate multiple lengths for records. The slotted-page method is widely used to handle varying-length records within a disk block.
- Since data are transferred between disk storage and main memory in units of a block, it is worthwhile to assign file records to blocks in such a way that a single block contains related records. If we can access several of the records we want with only one block access, we save disk accesses. Since disk accesses are usually the bottleneck in the performance of a database system, careful assignment of records to blocks can pay significant performance dividends.
- The data dictionary, also referred to as the system catalog, keeps track of metadata, that is, data about data, such as relation names, attribute names and types, storage information, integrity constraints, and user information.

- One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory. Since it is not possible to keep all blocks in main memory, we need to manage the allocation of the space available in main memory for the storage of blocks. The *buffer* is that part of main memory available for storage of copies of disk blocks. The subsystem responsible for the allocation of buffer space is called the *buffer manager*.
- Column-oriented storage systems provide good performance for many data warehousing applications.

Review Terms

- File Organization
  - File
  - Blocks
- Fixed-length records
- File header
- Free list
- Variable-length records
- Null bitmap
- Slotted-page structure
- Large objects
- Organization of records
  - Heap file organization
  - Sequential file organization
  - Multitable clustering file organization
  - B^+-tree file organizations
  - Hashing file organization
- Free-space map
- Sequential file
- Search key
- Cluster key
- Table partitioning
- Data-dictionary storage
  - Metadata
  - Data dictionary
  - System catalog
- Database buffer
  - Buffer manager
  - Pinned blocks
  - Evicted blocks
  - Forced output of blocks
  - Shared and exclusive locks
- Buffer-replacement strategies
  - Least recently used (LRU)
  - Toss-immediate
  - Most recently used (MRU)
- Output of blocks
- Forced output of blocks
- Log disk
- Journaling file systems
- Column-oriented storage
  - Columnar storage
  - Vector processing
  - Column stores
  - Row stores
  - Stripe
  - Hybrid row/column storage
- Main-memory database

Practice Exercises

13.1 Consider the deletion of record 5 from the file of Figure 13.3. Compare the relative merits of the following techniques for implementing the deletion:

a. Move record 6 to the space occupied by record 5, and move record 7 to the space occupied by record 6.

b. Move record 7 to the space occupied by record 5.

c. Mark record 5 as deleted, and move no records.

13.2 Show the structure of the file of Figure 13.4 after each of the following steps:

a. Insert (24556, Turnamian, Finance, 98000).

b. Delete record 2.

c. Insert (34556, Thompson, Music, 67000).

13.3 Consider the relations *section* and *takes*. Give an example instance of these two relations, with three sections, each of which has five students. Give a file structure of these relations that uses multitable clustering.

13.4 Consider the bitmap representation of the free-space map, where for each block in the file, two bits are maintained in the bitmap. If the block is between 0 and 30 percent full the bits are 00, between 30 and 60 percent the bits are 01, between 60 and 90 percent the bits are 10, and above 90 percent the bits are 11. Such bitmaps can be kept in memory even for quite large files.

a. Outline two benefits and one drawback to using two bits for a block, instead of one byte as described earlier in this chapter.

b. Describe how to keep the bitmap up to date on record insertions and deletions.

c. Outline the benefit of the bitmap technique over free lists in searching for free space and in updating free space information.

13.5 It is important to be able to quickly find out if a block is present in the buffer, and if so where in the buffer it resides. Given that database buffer sizes are very large, what (in-memory) data structure would you use for this task?

13.6 Suppose your university has a very large number of *takes* records, accumulated over many years. Explain how table partitioning can be done on the *takes* relation, and what benefits it could offer. Explain also one potential drawback of the technique.

13.7 Give an example of a relational-algebra expression and a query-processing strategy in each of the following situations:

a. MRU is preferable to LRU.

b. LRU is preferable to MRU.

13.8 PostgreSQL normally uses a small buffer, leaving it to the operating system buffer manager to manage the rest of main memory available for file system buffering. Explain (a) what is the benefit of this approach, and (b) one key limitation of this approach.

Exercises

13.9 In the variable-length record representation, a null bitmap is used to indicate if an attribute has the null value.

a. For variable-length fields, if the value is null, what would be stored in the offset and length fields?

b. In some applications, tuples have a very large number of attributes, most of which are null. Can you modify the record representation such that the only overhead for a null attribute is the single bit in the null bitmap?

13.10 Explain why the allocation of records to blocks affects database-system performance significantly.

13.11 List two advantages and two disadvantages of each of the following strategies for storing a relational database:

a. Store each relation in one file.

b. Store multiple relations (perhaps even the entire database) in one file.

13.12 In the sequential file organization, why is an overflow *block* used even if there is, at the moment, only one overflow record?

13.13 Give a normalized version of the *Index_metadata* relation, and explain why using the normalized version would result in worse performance.

13.14 Standard buffer managers assume each block is of the same size and costs the same to read. Consider a buffer manager that, instead of LRU, uses the rate of reference to objects, that is, how often an object has been accessed in the last n seconds. Suppose we want to store in the buffer objects of varying sizes, and varying read costs (such as web pages, whose read cost depends on the site from which they are fetched). Suggest how a buffer manager may choose which block to evict from the buffer.

Further Reading

[Hennessy et al. (2017)] is a popular textbook on computer architecture, which includes coverage of hardware aspects of translation look-aside buffers, caches, and memory-management units.

The storage structure of specific database systems, such as IBM DB2, Oracle, Microsoft SQL Server, and PostgreSQL are documented in their respective system manuals, which are available online.

Algorithms for buffer management in database systems, along with a performance evaluation, were presented by [Chou and Dewitt (1985)]. Buffer management in operating systems is discussed in most operating-system texts, including in [Silberschatz et al. (2018)].

[Abadi et al. (2008)] presents a comparison of column-oriented and row-oriented storage, including issues related to query processing and optimization.

Sybase IQ, developed in the mid 1990s, was the first commercially successful column-oriented database, designed for analytics. MonetDB and C-Store were column-oriented databases developed as academic research projects. The Vertica column-oriented database is a commercial database that grew out of C-Store, while VectorWise is a commercial database that grew out of MonetDB. As its name suggests, VectorWise supports vector processing of data, and as a result supports very high processing rates for many analytical queries. [Stonebraker et al. (2005)] describe C-Store, while [Idreos et al. (2012)] give an overview of the MonetDB project and [Zukowski et al. (2012)] describes Vectorwise.

The ORC and Parquet columnar file formats were developed to support compressed storage of data for big-data applications that run on the Apache Hadoop platform.

Bibliography

[Abadi et al. (2008)] D. J. Abadi, S. Madden, and N. Hachem, "Column-Stores vs. Row-Stores: How Different Are They Really?", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (2008), pages 967–980.

[Chou and Dewitt (1985)] H. T. Chou and D. J. Dewitt, "An Evaluation of Buffer Management Strategies for Relational Database Systems", In *Proc. of the International Conf. on Very Large Databases* (1985), pages 127–141.

[Hennessy et al. (2017)] J. L. Hennessy, D. A. Patterson, and D. Goldberg, *Computer Architecture: A Quantitative Approach*, 6th edition, Morgan Kaufmann (2017).

[Idreos et al. (2012)] S. Idreos, F. Groffen, N. Nes, S. Manegold, K. S. Mullender, and M. L. Kersten, "MonetDB: Two Decades of Research in Column-oriented Database Architectures", *IEEE Data Engineering Bulletin*, Volume 35, Number 1 (2012), pages 40–45.

[Silberschatz et al. (2018)] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th edition, John Wiley and Sons (2018).

[Stonebraker et al. (2005)] M. Stonebraker, D. J. Abadi, A. Batkin, X. Chen, M. Cherniack, M. Ferreira, E. Lau, A. Lin, S. Madden, E. J. O'Neil, P. E. O'Neil, A. Rasin, N. Tran, and S. B. Zdonik, "C-Store: A Column-oriented DBMS", In *Proc. of the International Conf. on Very Large Databases* (2005), pages 553–564.

[Zukowski et al. (2012)] M. Zukowski, M. van de Wiel, and P. A. Boncz, "Vectorwise: A Vectorized Analytical DBMS", In *Proc. of the International Conf. on Data Engineering* (2012), pages 1349–1350.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

CHAPTER 14

Indexing

Many queries reference only a small proportion of the records in a file. For example, a query like "Find all instructors in the Physics department" or "Find the total number of credits earned by the student with *ID* 22201" references only a fraction of the *instructor* or *student* records. It is inefficient for the system to read every tuple in the *instructor* relation to check if the *dept_name* value is "Physics". Likewise, it is inefficient to read the entire *student* relation just to find the one tuple for the *ID* "22201". Ideally, the system should be able to locate these records directly. To allow these forms of access, we design additional structures that we associate with files.

14.1 Basic Concepts

An index for a file in a database system works in much the same way as the index in this textbook. If we want to learn about a particular topic (specified by a word or a phrase) in this textbook, we can search for the topic in the index at the back of the book, find the pages where it occurs, and then read the pages to find the information for which we are looking. The words in the index are in sorted order, making it easy to find the word we want. Moreover, the index is much smaller than the book, further reducing the effort needed.

Database-system indices play the same role as book indices in libraries. For example, to retrieve a *student* record given an *ID*, the database system would look up an index to find on which disk block[1] the corresponding record resides, and then fetch the disk block, to get the appropriate *student* record.

Indices are critical for efficient processing of queries in databases. Without indices, every query would end up reading the entire contents of every relation that it uses; doing so would be unreasonably expensive for queries that only fetch a few records, for example, a single *student* record, or the records in the *takes* relation corresponding to a single student.

[1] As in earlier chapters, we use the term *disk* to refer to persistent storage devices, such as magnetic disks and solid-state drives.

Implementing an index on the *student* relation by keeping a sorted list of students' *ID* would not work well on very large databases, since (i) the index would itself be very big, (ii) even though keeping the index sorted reduces the search time, finding a student can still be rather time-consuming, and (iii) updating a sorted list as students are added or removed from the database can be very expensive. Instead, more sophisticated indexing techniques are used in database systems. We shall discuss several of these techniques in this chapter.

There are two basic kinds of indices:

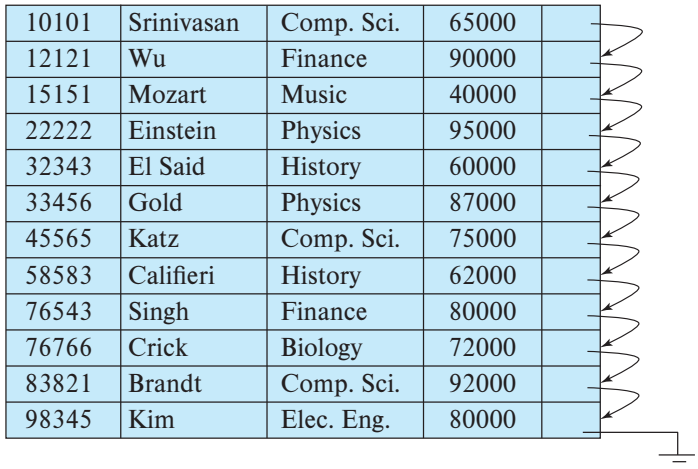
- **Ordered indices**. Based on a sorted ordering of the values.
- **Hash indices**. Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a *hash function*.

We shall consider several techniques for ordered indexing. No one technique is the best. Rather, each technique is best suited to particular database applications. Each technique must be evaluated on the basis of these factors:

- **Access types**: The types of access that are supported efficiently. Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.
- **Access time**: The time it takes to find a particular data item, or set of items, using the technique in question.
- **Insertion time**: The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.
- **Deletion time**: The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
- **Space overhead**: The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

We often want to have more than one index for a file. For example, we may wish to search for a book by author, by subject, or by title.

An attribute or set of attributes used to look up records in a file is called a **search key**. Note that this definition of *key* differs from that used in *primary key*, *candidate key*, and *superkey*. This duplicate meaning for *key* is (unfortunately) well established in practice. Using our notion of a search key, we see that if there are several indices on a file, there are several search keys.



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Figure 14.1 Sequential file for *instructor* records.

14.2 Ordered Indices

To gain fast random access to records in a file, we can use an index structure. Each index structure is associated with a particular search key. Just like the index of a book or a library catalog, an **ordered index** stores the values of the search keys in sorted order and associates with each search key the records that contain it.

The records in the indexed file may themselves be stored in some sorted order, just as books in a library are stored according to some attribute such as the Dewey decimal number. A file may have several indices, on different search keys. If the file containing the records is sequentially ordered, a **clustering index** is an index whose search key also defines the sequential order of the file. Clustering indices are also called **primary indices**; the term *primary index* may appear to denote an index on a primary key, but such indices can in fact be built on any search key. The search key of a clustering index is often the primary key, although that is not necessarily so. Indices whose search key specifies an order different from the sequential order of the file are called **nonclustering indices**, or **secondary indices**. The terms "clustered" and "nonclustered" are often used in place of "clustering" and "nonclustering."

In Section 14.2.1 through Section 14.2.3, we assume that all files are ordered sequentially on some search key. Such files, with a clustering index on the search key, are called **index-sequential files**. They represent one of the oldest index schemes used in database systems. They are designed for applications that require both sequential processing of the entire file and random access to individual records. In Section 14.2.4 we cover secondary indices.

Figure 14.1 shows a sequential file of *instructor* records taken from our university example. In the example of Figure 14.1, the records are stored in sorted order of instructor *ID*, which is used as the search key.

14.2.1 Dense and Sparse Indices

An **index entry**, or **index record**, consists of a search-key value and pointers to one or more records with that value as their search-key value. The pointer to a record consists of the identifier of a disk block and an offset within the disk block to identify the record within the block.

There are two types of ordered indices that we can use:

- **Dense index**: In a dense index, an index entry appears for every search-key value in the file. In a dense clustering index, the index record contains the search-key value and a pointer to the first data record with that search-key value. The rest of the records with the same search-key value would be stored sequentially after the first record, since, because the index is a clustering one, records are sorted on the same search key.

  In a dense nonclustering index, the index must store a list of pointers to all records with the same search-key value.

- **Sparse index**: In a sparse index, an index entry appears for only some of the search-key values. Sparse indices can be used only if the relation is stored in sorted order of the search key; that is, if the index is a clustering index. As is true in dense indices, each index entry contains a search-key value and a pointer to the first data record with that search-key value. To locate a record, we find the index entry with the largest search-key value that is less than or equal to the search-key value for which we are looking. We start at the record pointed to by that index entry and follow the pointers in the file until we find the desired record.

Figure 14.2 and Figure 14.3 show dense and sparse indices, respectively, for the *instructor* file. Suppose that we are looking up the record of instructor with *ID* "22222". Using the dense index of Figure 14.2, we follow the pointer directly to the desired record. Since *ID* is a primary key, there exists only one such record and the search is complete. If we are using the sparse index (Figure 14.3), we do not find an index entry for "22222". Since the last entry (in numerical order) before "22222" is "10101", we follow that pointer. We then read the *instructor* file in sequential order until we find the desired record.

Consider a (printed) dictionary. The header of each page lists the first word alphabetically on that page. The words at the top of each page of the book index together form a sparse index on the contents of the dictionary pages.

As another example, suppose that the search-key value is not a primary key. Figure 14.4 shows a dense clustering index for the *instructor* file with the search key being *dept_name*. Observe that in this case the *instructor* file is sorted on the search key *dept_name*, instead of *ID*, otherwise the index on *dept_name* would be a nonclustering index. Suppose that we are looking up records for the History department. Using the dense index of Figure 14.4, we follow the pointer directly to the first History record. We process this record and follow the pointer in that record to locate the next record in

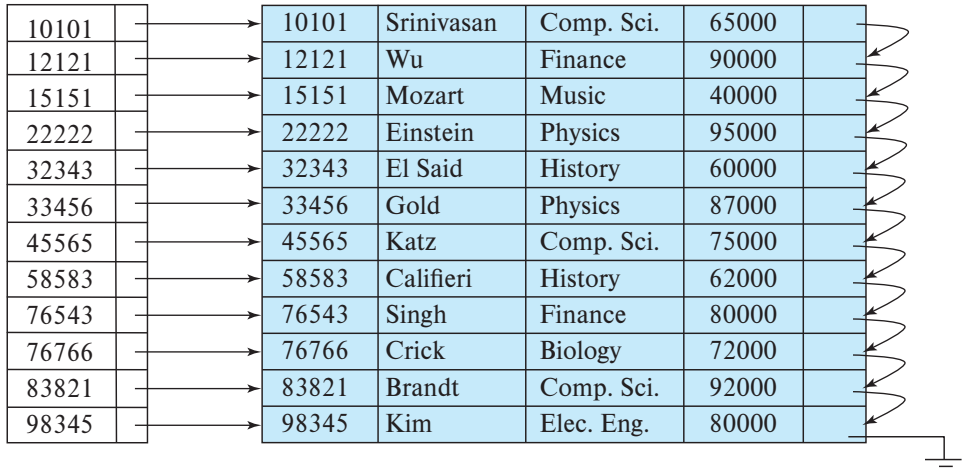


Figure 14.2 Dense index.

search-key (*dept_name*) order. We continue processing records until we encounter a record for a department other than History.

As we have seen, it is generally faster to locate a record if we have a dense index rather than a sparse index. However, sparse indices have advantages over dense indices in that they require less space and they impose less maintenance overhead for insertions and deletions.

There is a trade-off that the system designer must make between access time and space overhead. Although the decision regarding this trade-off depends on the specific application, a good compromise is to have a sparse index with one index entry per

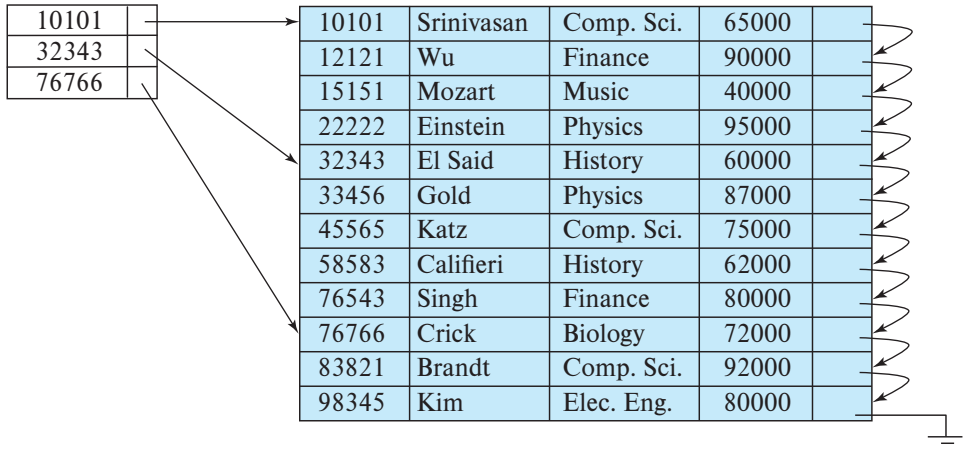


Figure 14.3 Sparse index.

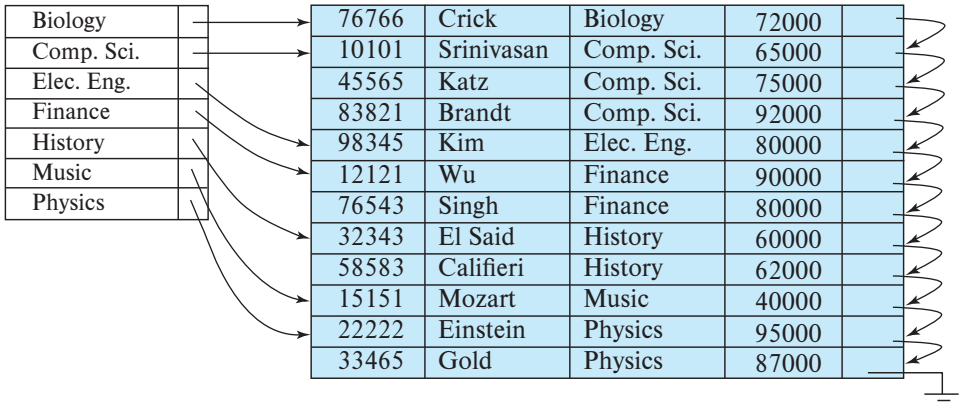


Figure 14.4 Dense index with search key *dept_name*.

block. The reason this design is a good trade-off is that the dominant cost in processing a database request is the time that it takes to bring a block from disk into main memory. Once we have brought in the block, the time to scan the entire block is negligible. Using this sparse index, we locate the block containing the record that we are seeking. Thus, unless the record is on an overflow block (see Section 13.3.2), we minimize block accesses while keeping the size of the index (and thus our space overhead) as small as possible.

For the preceding technique to be fully general, we must consider the case where records for one search-key value occupy several blocks. It is easy to modify our scheme to handle this situation.

14.2.2 Multilevel Indices

Suppose we build a dense index on a relation with 1,000,000 tuples. Index entries are smaller than data records, so let us assume that 100 index entries fit on a 4-kilobyte block. Thus, our index occupies 10,000 blocks. If the relation instead had 100,000,000 tuples, the index would instead occupy 1,000,000 blocks, or 4 gigabytes of space. Such large indices are stored as sequential files on disk.

If an index is small enough to be kept entirely in main memory, the search time to find an entry is low. However, if the index is so large that not all of it can be kept in memory, index blocks must be fetched from disk when required. (Even if an index is smaller than the main memory of a computer, main memory is also required for a number of other tasks, so it may not be possible to keep the entire index in memory.) The search for an entry in the index then requires several disk-block reads.

Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index would occupy b blocks, binary search requires as many as $\lceil \log_2(b) \rceil$ blocks to be read. ($\lceil x \rceil$ denotes the least integer that is greater than or equal to x; that is, we round upward.) Note that the blocks that are read are not adjacent

to each other, so each read requires a random (i.e., non-sequential) I/O operation. For a 10,000-block index, binary search requires 14 random block reads. On a magnetic disk system where a random block read takes on average 10 milliseconds, the index search will take 140 milliseconds. This may not seem much, but we would be able to carry out only seven index searches a second on a single disk, whereas a more efficient search mechanism would let us carry out far more searches per second, as we shall see shortly. Note that, if overflow blocks have been used, binary search is only possible on the non-overflow blocks, and the actual cost may be even higher than the logarithmic bound above. A sequential search requires b sequential block reads, which may take even longer (although in some cases the lower cost of sequential block reads may result in sequential search being faster than a binary search). Thus, the process of searching a large index may be costly.

To deal with this problem, we treat the index just as we would treat any other sequential file, and we construct a sparse outer index on the original index, which we now call the inner index, as shown in Figure 14.5. Note that the index entries are always in sorted order, allowing the outer index to be sparse. To locate a record, we first use binary search on the outer index to find the record for the largest search-key value less

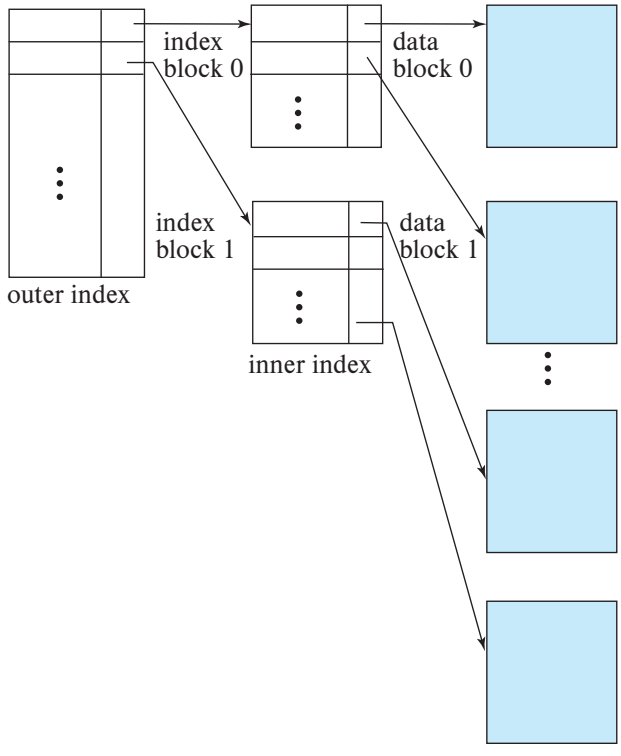


Figure 14.5 Two-level sparse index.

than or equal to the one that we desire. The pointer points to a block of the inner index. We scan this block until we find the record that has the largest search-key value less than or equal to the one that we desire. The pointer in this record points to the block of the file that contains the record for which we are looking.

In our example, an inner index with 10,000 blocks would require 10,000 entries in the outer index, which would occupy just 100 blocks. If we assume that the outer index is already in main memory, we would read only one index block for a search using a multilevel index, rather than the 14 blocks we read with binary search. As a result, we can perform 14 times as many index searches per second.

If our file is extremely large, even the outer index may grow too large to fit in main memory. With a 100,000,000-tuple relation, the inner index would occupy 1,000,000 blocks, and the outer index would occupy 10,000 blocks, or 40 megabytes. Since there are many demands on main memory, it may not be possible to reserve that much main memory just for this particular outer index. In such a case, we can create yet another level of index. Indeed, we can repeat this process as many times as necessary. Indices with two or more levels are called **multilevel indices**. Searching for records with a multilevel index requires significantly fewer I/O operations than does searching for records by binary search.[2]

Multilevel indices are closely related to tree structures, such as the binary trees used for in-memory indexing. We shall examine the relationship later, in Section 14.3.

14.2.3 Index Update

Regardless of what form of index is used, every index must be updated whenever a record is either inserted into or deleted from the file. Further, in case a record in the file is updated, any index whose search-key attribute is affected by the update must also be updated; for example, if the department of an instructor is changed, an index on the *dept_name* attribute of *instructor* must be updated correspondingly. Such a record update can be modeled as a deletion of the old record, followed by an insertion of the new value of the record, which results in an index deletion followed by an index insertion. As a result we only need to consider insertion and deletion on an index, and we do not need to consider updates explicitly.

We first describe algorithms for updating single-level indices.

14.2.3.1 Insertion

First, the system performs a lookup using the search-key value that appears in the record to be inserted. The actions the system takes next depend on whether the index is dense or sparse:

[2]In the early days of disk-based indices, each level of the index corresponded to a unit of physical storage. Thus, we may have indices at the track, cylinder, and disk levels. Such a hierarchy does not make sense today since disk subsystems hide the physical details of disk storage, and the number of disks and platters per disk is very small compared to the number of cylinders or bytes per track.

- Dense indices:
  1. If the search-key value does not appear in the index, the system inserts an index entry with the search-key value in the index at the appropriate position.
  2. Otherwise the following actions are taken:
     a. If the index entry stores pointers to all records with the same search-key value, the system adds a pointer to the new record in the index entry.
     b. Otherwise, the index entry stores a pointer to only the first record with the search-key value. The system then places the record being inserted after the other records with the same search-key values.
- Sparse indices: We assume that the index stores an entry for each block. If the system creates a new block, it inserts the first search-key value (in search-key order) appearing in the new block into the index. On the other hand, if the new record has the least search-key value in its block, the system updates the index entry pointing to the block; if not, the system makes no change to the index.

14.2.3.2 Deletion

To delete a record, the system first looks up the record to be deleted. The actions the system takes next depend on whether the index is dense or sparse:

- Dense indices:
  1. If the deleted record was the only record with its particular search-key value, then the system deletes the corresponding index entry from the index.
  2. Otherwise the following actions are taken:
     a. If the index entry stores pointers to all records with the same search-key value, the system deletes the pointer to the deleted record from the index entry.
     b. Otherwise, the index entry stores a pointer to only the first record with the search-key value. In this case, if the deleted record was the first record with the search-key value, the system updates the index entry to point to the next record.
- Sparse indices:
  1. If the index does not contain an index entry with the search-key value of the deleted record, nothing needs to be done to the index.
  2. Otherwise the system takes the following actions:
     a. If the deleted record was the only record with its search key, the system replaces the corresponding index record with an index record for the next search-key value (in search-key order). If the next search-key value already has an index entry, the entry is deleted instead of being replaced.

b. Otherwise, if the index entry for the search-key value points to the record being deleted, the system updates the index entry to point to the next record with the same search-key value.

Insertion and deletion algorithms for multilevel indices are a simple extension of the scheme just described. On deletion or insertion, the system updates the lowest-level index as described. As far as the second level is concerned, the lowest-level index is merely a file containing records—thus, if there is any change in the lowest-level index, the system updates the second-level index as described. The same technique applies to further levels of the index, if there are any.

14.2.4 Secondary Indices

Secondary indices must be dense, with an index entry for every search-key value, and a pointer to every record in the file. A clustering index may be sparse, storing only some of the search-key values, since it is always possible to find records with intermediate search-key values by a sequential access to a part of the file, as described earlier. If a secondary index stores only some of the search-key values, records with intermediate search-key values may be anywhere in the file and, in general, we cannot find them without searching the entire file.

A secondary index on a candidate key looks just like a dense clustering index, except that the records pointed to by successive values in the index are not stored sequentially. In general, however, secondary indices may have a different structure from clustering indices. If the search key of a clustering index is not a candidate key, it suffices if the index points to the first record with a particular value for the search key, since the other records can be fetched by a sequential scan of the file.

In contrast, if the search key of a secondary index is not a candidate key, it is not enough to point to just the first record with each search-key value. The remaining records with the same search-key value could be anywhere in the file, since the records are ordered by the search key of the clustering index, rather than by the search key of the secondary index. Therefore, a secondary index must contain pointers to all the records.

If a relation can have more than one record containing the same search key value (that is, two or more records can have the same values for the indexed attributes), the search key is said to be a **nonunique search key**.

One way to implement secondary indices on nonunique search keys is as follows: Unlike the case of primary indices, the pointers in such a secondary index do not point directly to the records. Instead, each pointer in the index points to a bucket that in turn contains pointers to the file. Figure 14.6 shows the structure of a secondary index that uses such an extra level of indirection on the *instructor* file, on the search key *dept_name*.

However, this approach has a few drawbacks. First, index access takes longer, due to an extra level of indirection, which may require a random I/O operation. Second,

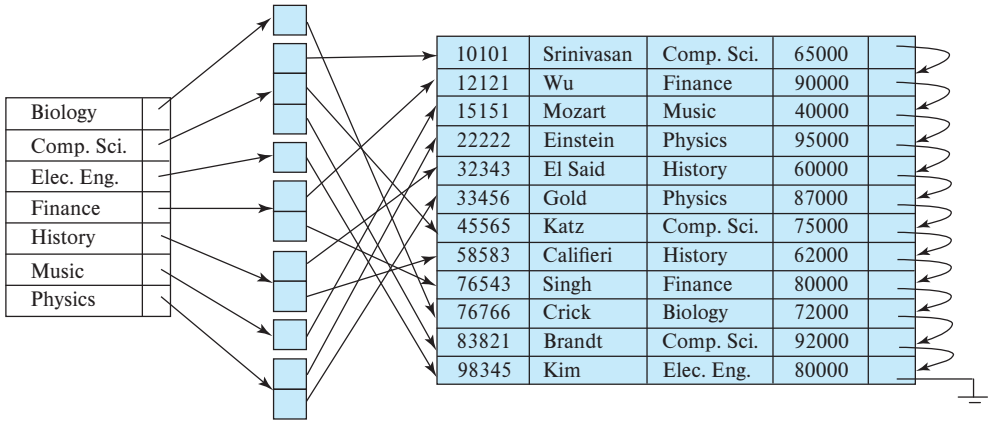


Figure 14.6 Secondary index on *instructor* file, on noncandidate key *dept_name*.

if a key has very few or no duplicates, if a whole block is allocated to its associated bucket, a lot of space would be wasted. Later in this chapter, we study more efficient alternatives for implementing secondary indices, which avoid these drawbacks.

A sequential scan in clustering index order is efficient because records in the file are stored physically in the same order as the index order. However, we cannot (except in rare special cases) store a file physically ordered by both the search key of the clustering index and the search key of a secondary index. Because secondary-key order and physical-key order differ, if we attempt to scan the file sequentially in secondary-key order, the reading of each record is likely to require the reading of a new block from disk, which is very slow.

The procedure described earlier for deletion and insertion can also be applied to secondary indices; the actions taken are those described for dense indices storing a pointer to every record in the file. If a file has multiple indices, whenever the file is modified, *every* index must be updated.

Secondary indices improve the performance of queries that use keys other than the search key of the clustering index. However, they impose a significant overhead on modification of the database. The designer of a database decides which secondary indices are desirable on the basis of an estimate of the relative frequency of queries and modifications.

14.2.5 Indices on Multiple Keys

Although the examples we have seen so far have had a single attribute in a search key, in general a search key can have more than one attribute. A search key containing more than one attribute is referred to as a **composite search key**. The structure of the index is the same as that of any other index, the only difference being that the search key is not a single attribute, but rather is a list of attributes. The search key can be represented as a tuple of values, of the form $(a_1, \ldots, a_n)$, where the indexed attributes are $A_1, \ldots, A_n$.

The ordering of search-key values is the *lexicographic ordering*. For example, for the case of two attribute search keys, $(a_1, a_2) < (b_1, b_2)$ if either $a_1 < b_1$ or $a_1 = b_1$ and $a_2 < b_2$. Lexicographic ordering is basically the same as alphabetic ordering of words.

As an example, consider an index on the *takes* relation, on the composite search key (*course_id*, *semester*, *year*). Such an index would be useful to find all students who have registered for a particular course in a particular semester/year. An ordered index on a composite key can also be used to answer several other kinds of queries efficiently, as we shall see in Section 14.6.2.

14.3 B$^+$-Tree Index Files

The main disadvantage of the index-sequential file organization is that performance degrades as the file grows, both for index lookups and for sequential scans through the data. Although this degradation can be remedied by reorganization of the file, frequent reorganizations are undesirable.

The **B$^+$-tree index** structure is the most widely used of several index structures that maintain their efficiency despite insertion and deletion of data. A B$^+$-tree index takes the form of a **balanced tree** in which every path from the root of the tree to a leaf of the tree is of the same length. Each nonleaf node in the tree (other than the root) has between $\lceil n/2 \rceil$ and n children, where n is fixed for a particular tree; the root has between 2 and n children.

We shall see that the B$^+$-tree structure imposes performance overhead on insertion and deletion and adds space overhead. The overhead is acceptable even for frequently modified files, since the cost of file reorganization is avoided. Furthermore, since nodes may be as much as half empty (if they have the minimum number of children), there is some wasted space. This space overhead, too, is acceptable given the performance benefits of the B$^+$-tree structure.

14.3.1 Structure of a B$^+$-Tree

A B$^+$-tree index is a multilevel index, but it has a structure that differs from that of the multilevel index-sequential file. We assume for now that there are no duplicate search key values, that is, each search key is unique and occurs in at most one record; we consider the issue of nonunique search keys later.

Figure 14.7 shows a typical node of a B$^+$-tree. It contains up to $n - 1$ search-key values $K_1, K_2, \ldots, K_{n-1}$, and n pointers $P_1, P_2, \ldots, P_n$. The search-key values within a node are kept in sorted order; thus, if $i < j$, then $K_i < K_j$.

Figure 14.7 Typical node of a B$^+$-tree.

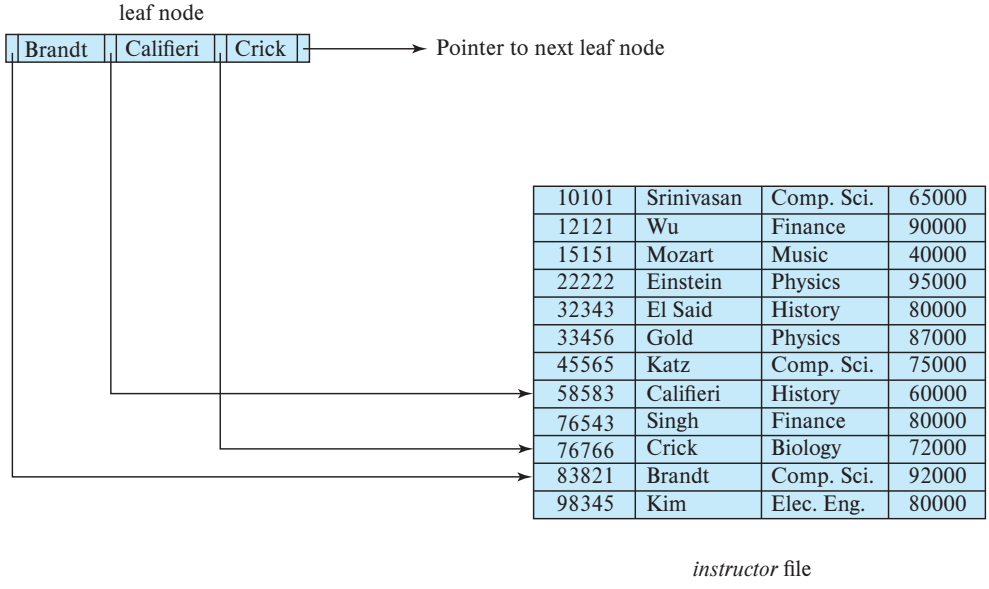


Figure 14.8 A leaf node for *instructor* B+-tree index ($n = 4$).

We consider first the structure of the **leaf nodes**. For $i = 1, 2, \ldots, n-1$, pointer P_i points to a file record with search-key value K_i. Pointer P_n has a special purpose that we shall discuss shortly.

Figure 14.8 shows one leaf node of a B+-tree for the *instructor* file, in which we have chosen n to be 4, and the search key is *name*.

Now that we have seen the structure of a leaf node, let us consider how search-key values are assigned to particular nodes. Each leaf can hold up to $n - 1$ values. We allow leaf nodes to contain as few as $\lceil (n-1)/2 \rceil$ values. With $n = 4$ in our example B+-tree, each leaf must contain at least two values, and at most three values.

If L_i and L_j are leaf nodes and $i < j$ (that is, L_i is to the left of L_j in the tree), then every search-key value v_i in L_i is less than every search-key value v_j in L_j.

If the B+-tree index is used as a dense index (as is usually the case), every search-key value must appear in some leaf node.

Now we can explain the use of the pointer P_n. Since there is a linear order on the leaves based on the search-key values that they contain, we use P_n to chain together the leaf nodes in search-key order. This ordering allows for efficient sequential processing of the file.

The **nonleaf nodes** of the B+-tree form a multilevel (sparse) index on the leaf nodes. The structure of nonleaf nodes is the same as that for leaf nodes, except that all pointers are pointers to tree nodes. A nonleaf node may hold up to n pointers and *must* hold at least $\lceil n/2 \rceil$ pointers. The number of pointers in a node is called the *fanout* of the node. Nonleaf nodes are also referred to as **internal nodes**.

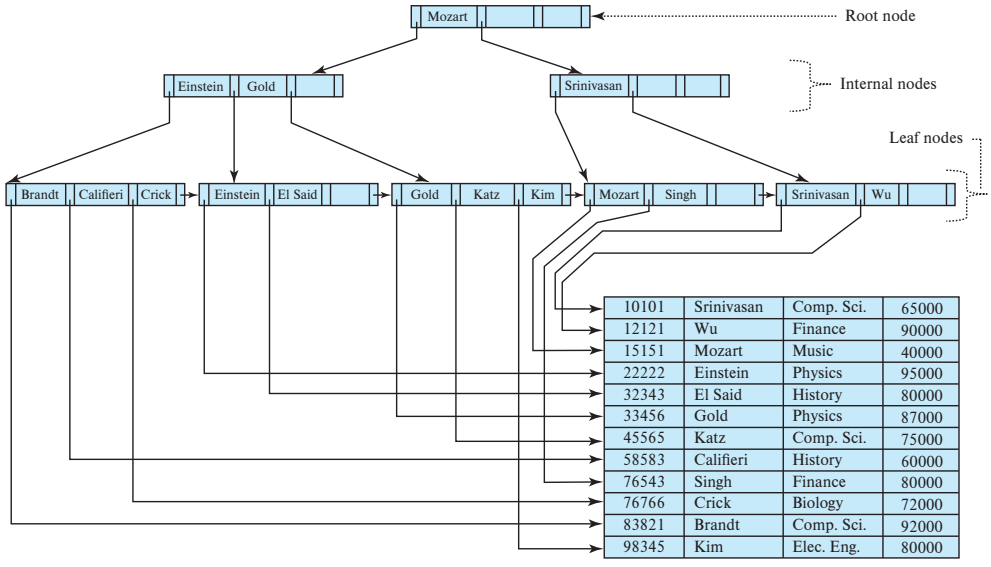


Figure 14.9 B$^+$-tree for *instructor* file ($n = 4$).

Let us consider a node containing m pointers ($m \leq n$). For $i = 2, 3, \ldots, m - 1$, pointer P_i points to the subtree that contains search-key values less than K_i and greater than or equal to K_{i-1}. Pointer P_m points to the part of the subtree that contains those key values greater than or equal to K_{m-1}, and pointer P_1 points to the part of the subtree that contains those search-key values less than K_1.

Unlike other nonleaf nodes, the root node can hold fewer than $\lceil n/2 \rceil$ pointers; however, it must hold at least two pointers, unless the tree consists of only one node. It is always possible to construct a B$^+$-tree, for any n, that satisfies the preceding requirements.

Figure 14.9 shows a complete B$^+$-tree for the *instructor* file (with $n = 4$). We have omitted null pointers for simplicity; any pointer field in the figure that does not have an arrow is understood to have a null value.

Figure 14.10 shows another B$^+$-tree for the *instructor* file, this time with $n = 6$. Observe that the height of this tree is less than that of the previous tree, which had $n = 4$.

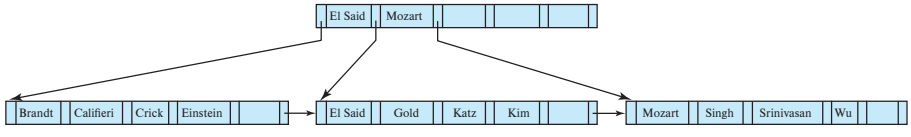


Figure 14.10 B$^+$-tree for *instructor* file with $n = 6$.

These examples of B+-trees are all balanced. That is, the length of every path from the root to a leaf node is the same. This property is a requirement for a B+-tree. Indeed, the "B" in B+-tree stands for "balanced." It is the balance property of B+-trees that ensures good performance for lookup, insertion, and deletion.

In general, search keys could have duplicates. One way to handle the case of nonunique search keys is to modify the tree structure to store each search key at a leaf node as many times as it appears in records, with each copy pointing to one record. The condition that $K_i < K_j$ if $i < j$ will need to be modified to $K_i \leq K_j$. However, this approach can result in duplicate search key values at internal nodes, making the insertion and deletion procedures more complicated and expensive. Another alternative is to store a set (or bucket) of record pointers with each search key value, as we saw earlier. This approach is more complicated and can result in inefficient access, especially if the number of record pointers for a particular key is very large.

Most database implementations instead make search keys unique as follows: Suppose the desired search key attribute a_i of relation r is nonunique. Let A_p be the primary key of r. Then the unique composite search key (a_i, A_p) is used instead of a_i when building the index. (Any set of attributes that together with a_i guarantee uniqueness can also be used instead of A_p.) For example, if we wished to create an index on the *instructor* relation on the attribute *name*, we instead create an index on the composite search key (*name*, ID), since ID is the primary key for *instructor*. Index lookups on just *name* can be efficiently handled using this index, as we shall see shortly. Section 14.3.5 covers issues in handling of nonunique search keys in more detail.

In our examples, we show indices on some nonunique search keys, such as *instructor.name*, assuming for simplicity that there are no duplicates; in reality most databases would automatically add extra attributes internally, to ensure the absence of duplicates.

14.3.2 Queries on B+-Trees

Let us consider how we process queries on a B+-tree. Suppose that we wish to find a record with a given value v for the search key. Figure 14.11 presents pseudocode for a function *find*(v) to carry out this task, assuming there are no duplicates, that is, there is at most one record with a particular search key. We address the issue of nonunique search keys later in this section.

Intuitively, the function starts at the root of the tree and traverses the tree down until it reaches a leaf node that would contain the specified value if it exists in the tree. Specifically, starting with the root as the current node, the function repeats the following steps until a leaf node is reached. First, the current node is examined, looking for the smallest i such that search-key value K_i is greater than or equal to v. Suppose such a value is found; then, if K_i is equal to v, the current node is set to the node pointed to by P_{i+1}, otherwise $K_i > v$, and the current node is set to the node pointed to by P_i. If no such value K_i is found, then $v > K_{m-1}$, where P_m is the last nonnull pointer in the node. In this case the current node is set to that pointed to by P_m. The above procedure is repeated, traversing down the tree until a leaf node is reached.

```
function find(v)
/* Assumes no duplicate keys, and returns pointer to the record with
 * search key value v if such a record exists, and null otherwise */
    Set C = root node
    while (C is not a leaf node) begin
        Let i = smallest number such that v ≤ C.K_i
        if there is no such number i then begin
            Let P_m = last non-null pointer in the node
            Set C = C.P_m
        end
        else if (v = C.K_i) then Set C = C.P_{i+1}
        else Set C = C.P_i /* v < C.K_i */
    end
    /* C is a leaf node */
    if for some i, K_i = v
        then return P_i
        else return null ; /* No record with key value v exists*/
```

Figure 14.11 Querying a B$^+$-tree.

At the leaf node, if there is a search-key value $K_i = v$, pointer P_i directs us to a record with search-key value K_i. The function then returns the pointer to the record, P_i. If no search key with value v is found in the leaf node, no record with key value v exists in the relation, and function *find* returns null, to indicate failure.

B$^+$-trees can also be used to find all records with search key values in a specified range [*lb*, *ub*]. For example, with a B$^+$-tree on attribute *salary* of *instructor*, we can find all *instructor* records with salary in a specified range such as [50000, 100000] (in other words, all salaries between 50000 and 100000). Such queries are called **range queries**.

To execute such queries, we can create a procedure *findRange* (*lb*, *ub*), shown in Figure 14.12. The procedure does the following: it first traverses to a leaf in a manner similar to *find*(*lb*); the leaf may or may not actually contain value *lb*. It then steps through records in that and subsequent leaf nodes collecting pointers to all records with key values $C.K_i$ s.t. $lb \leq C.K_i \leq ub$ into a set resultSet. The function stops when $C.K_i > ub$, or there are no more keys in the tree.

A real implementation would provide a version of *findRange* supporting an iterator interface similar to that provided by the JDBC `ResultSet`, which we saw in Section 5.1.1. Such an iterator interface would provide a method *next*(), which can be called repeatedly to fetch successive records. The *next*() method would step through the entries at the leaf level, in a manner similar to *findRange*, but each call takes only one step and records where it left off, so that successive calls to *next*() step through successive en-

```
function findRange(lb, ub)
/* Returns all records with search key value V such that lb ≤ V ≤ ub. */
    Set resultSet = {};
    Set C = root node
    while (C is not a leaf node) begin
        Let i = smallest number such that lb ≤ C.K_i
        if there is no such number i then begin
            Let P_m = last non-null pointer in the node
            Set C = C.P_m
        end
        else if (lb = C.K_i) then Set C = C.P_{i+1}
        else Set C = C.P_i /* lb < C.K_i */
    end
    /* C is a leaf node */
    Let i be the least value such that K_i ≥ lb
    if there is no such i
        then Set i = 1 + number of keys in C; /* To force move to next leaf */
    Set done = false;
    while (not done) begin
        Let n = number of keys in C.
        if ( i ≤ n and C.K_i ≤ ub) then begin
            Add C.P_i to resultSet
            Set i = i + 1
        end
        else if  (i ≤ n and C.K_i > ub)
            then Set done = true;
        else if (i > n and C.P_{n+1} is not null)
            then Set C = C.P_{n+1}, and i = 1 /* Move to next leaf */
        else Set done = true; /* No more leaves to the right */
    end
    return resultSet;
```

Figure 14.12 Range query on a B+-tree.

tries. We omit details for simplicity, and leave the pseudocode for the iterator interface as an exercise for the interested reader.

We now consider the cost of querying on a B+-tree index. In processing a query, we traverse a path in the tree from the root to some leaf node. If there are N records in the file, the path is no longer than $\lceil \log_{\lceil n/2 \rceil}(N) \rceil$.

Typically, the node size is chosen to be the same as the size of a disk block, which is typically 4 kilobytes. With a search-key size of 12 bytes, and a disk-pointer size of

8 bytes, n is around 200. Even with a more conservative estimate of 32 bytes for the search-key size, n is around 100. With $n = 100$, if we have 1 million search-key values in the file, a lookup requires only $\lceil \log_{50}(1{,}000{,}000) \rceil = 4$ nodes to be accessed. Thus, at most four blocks need to be read from disk to traverse the path from the root to a leaf. The root node of the tree is usually heavily accessed and is likely to be in the buffer, so typically only three or fewer blocks need to be read from disk.

An important difference between B$^+$-tree structures and in-memory tree structures, such as binary trees, is the size of a node, and as a result, the height of the tree. In a binary tree, each node is small and has at most two pointers. In a B$^+$-tree, each node is large—typically a disk block—and a node can have a large number of pointers. Thus, B$^+$-trees tend to be fat and short, unlike thin and tall binary trees. In a balanced binary tree, the path for a lookup can be of length $\lceil \log_2(N) \rceil$, where N is the number of records in the file being indexed. With $N = 1{,}000{,}000$ as in the previous example, a balanced binary tree requires around 20 node accesses. If each node were on a different disk block, 20 block reads would be required to process a lookup, in contrast to the four block reads for the B$^+$-tree. The difference is significant with a magnetic disk, since each block read could require a disk arm seek which, together with the block read, takes about 10 milliseconds on a magnetic disk. The difference is not quite as drastic with flash storage, where a read of a 4 kilobyte page takes around 10 to 100 microseconds, but it is still significant.

After traversing down to the leaf level, queries on a single value of a unique search key require one more random I/O operation to fetch any matching record.

Range queries have an additional cost, after traversing down to the leaf level: all the pointers in the given range must be retrieved. These pointers are in consecutive leaf nodes; thus, if M such pointers are retrieved, at most $\lceil M/(n/2) \rceil + 1$ leaf nodes need to be accessed to retrieve the pointers (since each leaf node has at least $n/2$ pointers, but even two pointers may be split across two pages). To this cost, we need to add the cost of accessing the actual records. For secondary indices, each such record may be on a different block, which could result in M random I/O operations in the worst case. For clustered indices, these records would be in consecutive blocks, with each block containing multiple records, resulting in a significantly lower cost.

Now, let us consider the case of nonunique keys. As explained earlier, if we wish to create an index on an attribute a_i that is not a candidate key, and may thus have duplicates, we instead create an index on a composite key that is duplicate-free. The composite key is created by adding extra attributes, such as the primary key, to a_i, to ensure uniqueness. Suppose we created an index on the composite key (a_i, A_p) instead of creating an index on a_i.

An important question, then, is how do we retrieve all tuples with a given value v for a_i using the above index? This question is easily answered by using the function findRange(lb, ub), with $lb = (v, -\infty)$ and $ub = (v, \infty)$, where $-\infty$ and ∞ denote the smallest and largest possible values of A_p. All records with $a_i = v$ would be returned by the above function call. Range queries on a_i can be handled similarly. These range

queries retrieve pointers to the records quite efficiently, although retrieval of the records may be expensive, as discussed earlier.

14.3.3 Updates on B+-Trees

When a record is inserted into, or deleted from a relation, indices on the relation must be updated correspondingly. Recall that updates to a record can be modeled as a deletion of the old record followed by insertion of the updated record. Hence we only consider the case of insertion and deletion.

Insertion and deletion are more complicated than lookup, since it may be necessary to **split** a node that becomes too large as the result of an insertion, or to **coalesce** nodes (i.e., combine nodes) if a node becomes too small (fewer than $\lceil n/2 \rceil$ pointers). Furthermore, when a node is split or a pair of nodes is combined, we must ensure that balance is preserved. To introduce the idea behind insertion and deletion in a B+-tree, we shall assume temporarily that nodes never become too large or too small. Under this assumption, insertion and deletion are performed as defined next.

- **Insertion**. Using the same technique as for lookup from the *find*() function (Figure 14.11), we first find the leaf node in which the search-key value would appear. We then insert an entry (i.e., a search-key value and record pointer pair) in the leaf node, positioning it such that the search keys are still in order.
- **Deletion**. Using the same technique as for lookup, we find the leaf node containing the entry to be deleted by performing a lookup on the search-key value of the deleted record; if there are multiple entries with the same search-key value, we search across all entries with the same search-key value until we find the entry that points to the record being deleted. We then remove the entry from the leaf node. All entries in the leaf node that are to the right of the deleted entry are shifted left by one position, so that there are no gaps in the entries after the entry is deleted.

We now consider the general case of insertion and deletion, dealing with node splitting and node coalescing.

14.3.3.1 Insertion

We now consider an example of insertion in which a node must be split. Assume that a record is inserted on the *instructor* relation, with the *name* value being Adams. We then need to insert an entry for "Adams" into the B+-tree of Figure 14.9. Using the algorithm for lookup, we find that "Adams" should appear in the leaf node containing "Brandt", "Califieri", and "Crick." There is no room in this leaf to insert the search-key value "Adams." Therefore, the node is *split* into two nodes. Figure 14.13 shows the two leaf nodes that result from the split of the leaf node on inserting "Adams". The search-key values "Adams" and "Brandt" are in one leaf, and "Califieri" and "Crick" are in the other. In general, we take the n search-key values (the $n - 1$ values in the leaf

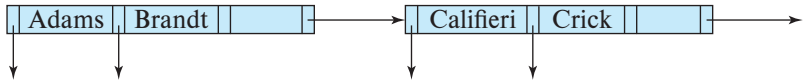


Figure 14.13 Split of leaf node on insertion of "Adams".

node plus the value being inserted), and put the first $\lceil n/2 \rceil$ in the existing node and the remaining values in a newly created node.

Having split a leaf node, we must insert the new leaf node into the B$^+$-tree structure. In our example, the new node has "Califieri" as its smallest search-key value. We need to insert an entry with this search-key value, and a pointer to the new node, into the parent of the leaf node that was split. The B$^+$-tree of Figure 14.14 shows the result of the insertion. It was possible to perform this insertion with no further node split, because there was room in the parent node for the new entry. If there were no room, the parent would have had to be split, requiring an entry to be added to its parent. In the worst case, all nodes along the path to the root must be split. If the root itself is split, the entire tree becomes deeper.

Splitting of a nonleaf node is a little different from splitting of a leaf node. Figure 14.15 shows the result of inserting a record with search key "Lamport" into the tree shown in Figure 14.14. The leaf node in which "Lamport" is to be inserted already has entries "Gold", "Katz", and "Kim", and as a result the leaf node has to be split. The new right-hand-side node resulting from the split contains the search-key values "Kim" and "Lamport". An entry (Kim, $n1$) must then be added to the parent node, where $n1$ is a pointer to the new node, However, there is no space in the parent node to add a new entry, and the parent node has to be split. To do so, the parent node is conceptually expanded temporarily, the entry added, and the overfull node is then immediately split.

When an overfull nonleaf node is split, the child pointers are divided among the original and the newly created nodes; in our example, the original node is left with the first three pointers, and the newly created node to the right gets the remaining two pointers. The search key values are, however, handled a little differently. The search key values that lie between the pointers moved to the right node (in our example, the value "Kim") are moved along with the pointers, while those that lie between the pointers that stay on the left (in our example, "Califieri" and "Einstein") remain undisturbed.

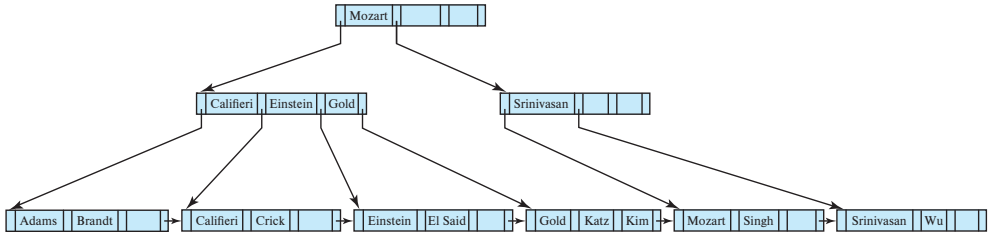


Figure 14.14 Insertion of "Adams" into the B$^+$-tree of Figure 14.9.

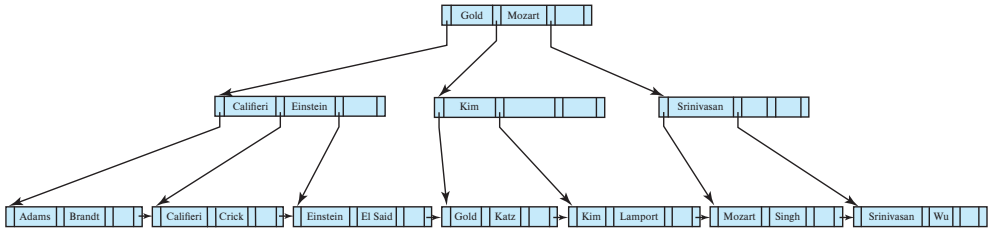


Figure 14.15 Insertion of "Lamport" into the B+-tree of Figure 14.14.

However, the search key value that lies between the pointers that stay on the left, and the pointers that move to the right node is treated differently. In our example, the search key value "Gold" lies between the three pointers that went to the left node, and the two pointers that went to the right node. The value "Gold" is not added to either of the split nodes. Instead, an entry (Gold, $n2$) is added to the parent node, where $n2$ is a pointer to the newly created node that resulted from the split. In this case, the parent node is the root, and it has enough space for the new entry.

The general technique for insertion into a B+-tree is to determine the leaf node l into which insertion must occur. If a split results, insert the new node into the parent

---

procedure *insert*(*value* K, *pointer* P)
  if (tree is empty) create an empty leaf node L, which is also the root
  else Find the leaf node L that should contain key value K
  if (L has less than $n - 1$ key values)
    then insert_in_leaf (L, K, P)
    else begin /* L has $n - 1$ key values already, split it */
      Create node L'
      Copy $L.P_1 \ldots L.K_{n-1}$ to a block of memory T that can
        hold n (pointer, key-value) pairs
      insert_in_leaf (T, K, P)
      Set $L'.P_n = L.P_n$; Set $L.P_n = L'$
      Erase $L.P_1$ through $L.K_{n-1}$ from L
      Copy $T.P_1$ through $T.K_{\lceil n/2 \rceil}$ from T into L starting at $L.P_1$
      Copy $T.P_{\lceil n/2 \rceil + 1}$ through $T.K_n$ from T into L' starting at $L'.P_1$
      Let K' be the smallest key-value in L'
      insert_in_parent(L, K', L')
    end

Figure 14.16 Insertion of entry in a B+-tree.

of node l. If this insertion causes a split, proceed recursively up the tree until either an insertion does not cause a split or a new root is created.

Figure 14.16 outlines the insertion algorithm in pseudocode. The procedure *insert* inserts a key-value pointer pair into the index, using two subsidiary procedures *insert_in_leaf* and *insert_in_parent*, shown in Figure 14.17. In the pseudocode, L, N, P and T denote pointers to nodes, with L being used to denote a leaf node. $L.K_i$ and $L.P_i$ denote the ith value and the ith pointer in node L, respectively; $T.K_i$ and $T.P_i$ are used similarly. The pseudocode also makes use of the function $parent(N)$ to find the parent of a node N. We can compute a list of nodes in the path from the root to the leaf while initially finding the leaf node, and we can use it later to find the parent of any node in the path efficiently.

```
procedure insert_in_leaf (node L, value K, pointer P)
    if (K < L.K_1)
        then insert P, K into L just before L.P_1
        else begin
            Let K_i be the highest value in L that is less than or equal to K
            Insert P, K into L just after L.K_i
        end
procedure insert_in_parent(node N, value K', node N')
    if (N is the root of the tree)
        then begin
            Create a new node R containing N, K', N'    /* N and N' are pointers */
            Make R the root of the tree
            return
        end
    Let P = parent (N)
    if (P has less than n pointers)
        then insert (K', N') in P just after N
        else begin /* Split P */
            Copy P to a block of memory T that can hold P and (K', N')
            Insert (K', N') into T just after N
            Erase all entries from P; Create node P'
            Copy T.P_1 ... T.P_⌈(n+1)/2⌉ into P
            Let K'' = T.K_⌈(n+1)/2⌉
            Copy T.P_⌈(n+1)/2⌉+1 ... T.P_n+1 into P'
            insert_in_parent(P, K'', P')
        end
```

Figure 14.17 Subsidiary procedures for insertion of entry in a B$^+$-tree.

The procedure *insert_in_parent* takes as parameters N, K', N', where node N was split into N and N', with K' being the least value in N'. The procedure modifies the parent of N to record the split. The procedures *insert_into_index* and *insert_in_parent* use a temporary area of memory T to store the contents of a node being split. The procedures can be modified to copy data from the node being split directly to the newly created node, reducing the time required for copying data. However, the use of the temporary space T simplifies the procedures.

14.3.3.2 Deletion

We now consider deletions that cause tree nodes to contain too few pointers. First, let us delete "Srinivasan" from the B+-tree of Figure 14.14. The resulting B+-tree appears in Figure 14.18. We now consider how the deletion is performed. We first locate the entry for "Srinivasan" by using our lookup algorithm. When we delete the entry for "Srinivasan" from its leaf node, the node is left with only one entry, "Wu". Since, in our example, $n = 4$ and $1 < \lceil (n-1)/2 \rceil$, we must either merge the node with a sibling node or redistribute the entries between the nodes, to ensure that each node is at least half-full. In our example, the underfull node with the entry for "Wu" can be merged with its left sibling node. We merge the nodes by moving the entries from both the nodes into the left sibling and deleting the now-empty right sibling. Once the node is deleted, we must also delete the entry in the parent node that pointed to the just deleted node.

In our example, the entry to be deleted is (Srinivasan, *n3*), where *n3* is a pointer to the leaf containing "Srinivasan". (In this case the entry to be deleted in the nonleaf node happens to be the same value as that deleted from the leaf; that would not be the case for most deletions.) After deleting the above entry, the parent node, which had a search key value "Srinivasan" and two pointers, now has one pointer (the leftmost pointer in the node) and no search-key values. Since $1 < \lceil n/2 \rceil$ for $n = 4$, the parent node is underfull. (For larger n, a node that becomes underfull would still have some values as well as pointers.)

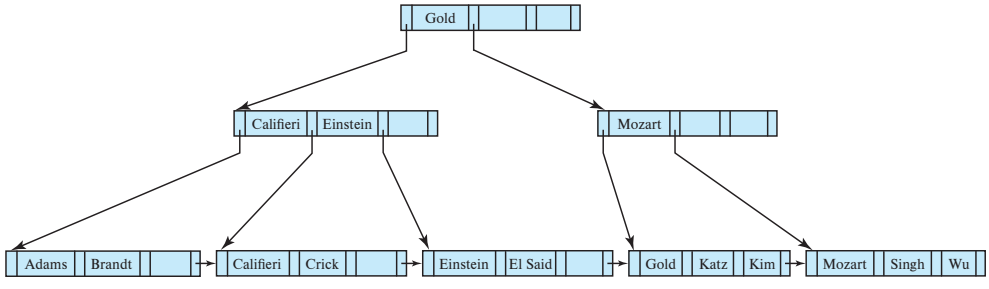


Figure 14.18 Deletion of "Srinivasan" from the B+-tree of Figure 14.14.

In this case, we look at a sibling node; in our example, the only sibling is the nonleaf node containing the search keys "Califieri", "Einstein", and "Gold". If possible, we try to coalesce the node with its sibling. In this case, coalescing is not possible, since the node and its sibling together have five pointers, against a maximum of four. The solution in this case is to **redistribute** the pointers between the node and its sibling, such that each has at least $\lceil n/2 \rceil = 2$ child pointers. To do so, we move the rightmost pointer from the left sibling (the one pointing to the leaf node containing "Gold") to the underfull right sibling. However, the underfull right sibling would now have two pointers, namely, its leftmost pointer, and the newly moved pointer, with no value separating them. In fact, the value separating them is not present in either of the nodes, but is present in the parent node, between the pointers from the parent to the node and its sibling. In our example, the value "Mozart" separates the two pointers and is present in the right sibling after the redistribution. Redistribution of the pointers also means that the value "Mozart" in the parent no longer correctly separates search-key values in the two siblings. In fact, the value that now correctly separates search-key values in the two sibling nodes is the value "Gold", which was in the left sibling before redistribution.

As a result, as can be seen in the B^+-tree in Figure 14.18, after redistribution of pointers between siblings, the value "Gold" has moved up into the parent, while the value that was there earlier, "Mozart", has moved down into the right sibling.

We next delete the search-key values "Singh" and "Wu" from the B^+-tree of Figure 14.18. The result is shown in Figure 14.19. The deletion of the first of these values does not make the leaf node underfull, but the deletion of the second value does. It is not possible to merge the underfull node with its sibling, so a redistribution of values is carried out, moving the search-key value "Kim" into the node containing "Mozart", resulting in the tree shown in Figure 14.19. The value separating the two siblings has been updated in the parent, from "Mozart" to "Kim".

Now we delete "Gold" from the above tree; the result is shown in Figure 14.20. This results in an underfull leaf, which can now be merged with its sibling. The resultant deletion of an entry from the parent node (the nonleaf node containing "Kim") makes the parent underfull (it is left with just one pointer). This time around, the parent node can be merged with its sibling. This merge results in the search-key value "Gold"

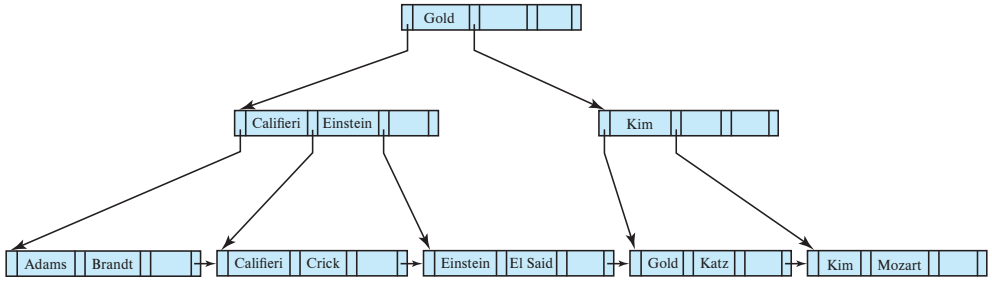


Figure 14.19 Deletion of "Singh" and "Wu" from the B^+-tree of Figure 14.18.

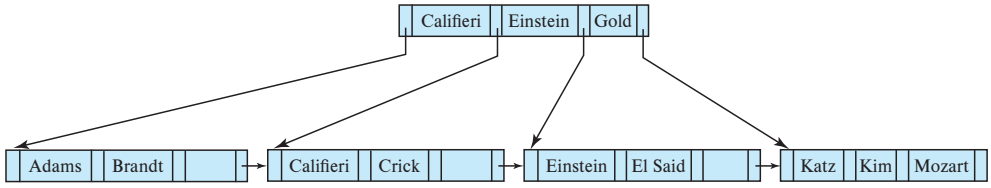


Figure 14.20 Deletion of "Gold" from the B+-tree of Figure 14.19.

moving down from the parent into the merged node. As a result of this merge, an entry is deleted from its parent, which happens to be the root of the tree. And as a result of that deletion, the root is left with only one child pointer and no search-key value, violating the condition that the root must have at least two children. As a result, the root node is deleted and its sole child becomes the root, and the depth of the B+-tree has been decreased by 1.

It is worth noting that, as a result of deletion, a key value that is present in a nonleaf node of the B+-tree may not be present at any leaf of the tree. For example, in Figure 14.20, the value "Gold" has been deleted from the leaf level but is still present in a nonleaf node.

In general, to delete a value in a B+-tree, we perform a lookup on the value and delete it. If the node is too small, we delete it from its parent. This deletion results in recursive application of the deletion algorithm until the root is reached, a parent remains adequately full after deletion, or redistribution is applied.

Figure 14.21 outlines the pseudocode for deletion from a B+-tree. The procedure *swap_variables*(N, N') merely swaps the values of the (pointer) variables N and N'; this swap has no effect on the tree itself. The pseudocode uses the condition "too few pointers/values." For nonleaf nodes, this criterion means less than $\lceil n/2 \rceil$ pointers; for leaf nodes, it means less than $\lceil (n-1)/2 \rceil$ values. The pseudocode redistributes entries by borrowing a single entry from an adjacent node. We can also redistribute entries by repartitioning entries equally between the two nodes. The pseudocode refers to deleting an entry (K, P) from a node. In the case of leaf nodes, the pointer to an entry actually precedes the key value, so the pointer P precedes the key value K. For nonleaf nodes, P follows the key value K.

14.3.4 Complexity of B+-Tree Updates

Although insertion and deletion operations on B+-trees are complicated, they require relatively few I/O operations, which is an important benefit since I/O operations are expensive. It can be shown that the number of I/O operations needed in the worst case for an insertion is proportional to $\log_{\lceil n/2 \rceil}(N)$, where n is the maximum number of pointers in a node, and N is the number of records in the file being indexed.

The worst-case complexity of the deletion procedure is also proportional to $\log_{\lceil n/2 \rceil}(N)$, provided there are no duplicate values for the search key; we discuss the case of nonunique search keys later in this chapter.

```
procedure delete(value K, pointer P)
    find the leaf node L that contains (K, P)
    delete_entry(L, K, P)

procedure delete_entry(node N, value K, pointer P)
    delete (K, P) from N
    if (N is the root and N has only one remaining child)
    then make the child of N the new root of the tree and delete N
    else if (N has too few values/pointers) then begin
        Let N' be the previous or next child of parent(N)
        Let K' be the value between pointers N and N' in parent(N)
        if (entries in N and N' can fit in a single node)
            then begin /* Coalesce nodes */
                if (N is a predecessor of N') then swap_variables(N, N')
                if (N is not a leaf)
                    then append K' and all pointers and values in N to N'
                    else append all (K_i, P_i) pairs in N to N'; set N'.P_n = N.P_n
                delete_entry(parent(N), K', N); delete node N
            end
        else begin /* Redistribution: borrow an entry from N' */
            if (N' is a predecessor of N) then begin
                if (N is a nonleaf node) then begin
                    let m be such that N'.P_m is the last pointer in N'
                    remove (N'.K_{m-1}, N'.P_m) from N'
                    insert (N'.P_m, K') as the first pointer and value in N,
                        by shifting other pointers and values right
                    replace K' in parent(N) by N'.K_{m-1}
                end
                else begin
                    let m be such that (N'.P_m, N'.K_m) is the last pointer/value
                        pair in N'
                    remove (N'.P_m, N'.K_m) from N'
                    insert (N'.P_m, N'.K_m) as the first pointer and value in N,
                        by shifting other pointers and values right
                    replace K' in parent(N) by N'.K_m
                end
            end
            else … symmetric to the then case …
        end
    end
```

Figure 14.21 Deletion of entry from a B$^+$-tree.

In other words, the cost of insertion and deletion operations in terms of I/O operations is proportional to the height of the B+-tree, and is therefore low. It is the speed of operation on B+-trees that makes them a frequently used index structure in database implementations.

In practice, operations on B+-trees result in fewer I/O operations than the worst-case bounds. With fanout of 100, and assuming accesses to leaf nodes are uniformly distributed, the parent of a leaf node is 100 times more likely to get accessed than the leaf node. Conversely, with the same fanout, the total number of nonleaf nodes in a B+-tree would be just a little more than 1/100th of the number of leaf nodes. As a result, with memory sizes of several gigabytes being common today, for B+-trees that are used frequently, even if the relation is very large it is quite likely that most of the nonleaf nodes are already in the database buffer when they are accessed. Thus, typically only one or two I/O operations are required to perform a lookup. For updates, the probability of a node split occurring is correspondingly very small. Depending on the ordering of inserts, with a fanout of 100, only from 1 in 100 to 1 in 50 insertions will result in a node split, requiring more than one block to be written. As a result, on an average an insert will require just a little more than one I/O operation to write updated blocks.

Although B+-trees only guarantee that nodes will be at least half full, if entries are inserted in random order, nodes can be expected to be more than two-thirds full on average. If entries are inserted in sorted order, on the other hand, nodes will be only half full. (We leave it as an exercise to the reader to figure out why nodes would be only half full in the latter case.)

14.3.5 Nonunique Search Keys

We have assumed so far that search keys are unique. Recall also that we described earlier, in Section 14.3.1, how to make search keys unique by creating a composite search key containing the original search key and extra attributes, that together are unique across all records.

The extra attribute can be a record-id, which is a pointer to the record, or a primary key, or any other attribute whose value is unique among all records with the same search-key value. The extra attribute is called a **uniquifier** attribute.

A search with the original search-key attribute can be carried out using a range search as we saw in Section 14.3.2; alternatively, we can create a variant of the *findRange* function that takes only the original search key value as parameter and ignores the value of the uniquifier attribute when comparing search-key values.

It is also possible to modify the B+-tree structure to support duplicate search keys. The insert, delete, and lookup methods all have to be modified correspondingly.

- One alternative is to store each key value only once in the tree, and to keep a bucket (or list) of record pointers with a search-key value, to handle nonunique search keys. This approach is space efficient since it stores the key value only once; however, it creates several complications when B+-trees are implemented. If the

buckets are kept in the leaf node, extra code is needed to deal with variable-size buckets, and to deal with buckets that grow larger than the size of the leaf node. If the buckets are stored in separate blocks, an extra I/O operation may be required to fetch records.

- Another option is to store the search key value once per record; this approach allows a leaf node to be split in the usual way if it is found to be full during an insert. However, this approach makes handling of split and search on internal nodes significantly more complicated, since two leaves may contain the same search key value. It also has a higher space overhead, since key values are stored as many times as there are records containing that value.

A major problem with both these approaches, as compared to the unique search-key approach, lies in the efficiency of record deletion. (The complexity of lookup and insertion are the same with both these approaches, as well as with the unique search-key approach.) Suppose a particular search-key value occurs a large number of times, and one of the records with that search key is to be deleted. The deletion may have to search through a number of entries with the same search-key value, potentially across multiple leaf nodes, to find the entry corresponding to the particular record being deleted. Thus, the worst-case complexity of deletion may be linear in the number of records.

In contrast, record deletion can be done efficiently using the unique search key approach. When a record is to be deleted, the composite search-key value is computed from the record and then used to look up the index. Since the value is unique, the corresponding leaf-level entry can be found with a single traversal from root to leaf, with no further accesses at the leaf level. The worst-case cost of deletion is logarithmic in the number of records, as we saw earlier.

Due to the inefficiency of deletion, as well as other complications due to duplicate search keys, B^+-tree implementations in most database systems only handle unique search keys, and they automatically add record-ids or other attributes to make nonunique search keys unique.

14.4 B^+-Tree Extensions

In this section, we discuss several extensions and variations of the B^+-tree index structure.

14.4.1 B^+-Tree File Organization

As mentioned in Section 14.3, the main drawback of index-sequential file organization is the degradation of performance as the file grows: With growth, an increasing percentage of index entries and actual records become out of order and are stored in overflow blocks. We solve the degradation of index lookups by using B^+-tree indices on the file.

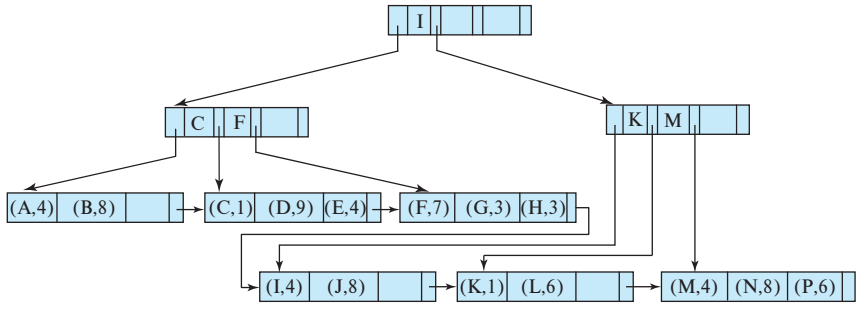


Figure 14.22 B+-tree file organization.

We solve the degradation problem for storing the actual records by using the leaf level of the B+-tree to organize the blocks containing the actual records. We use the B+-tree structure not only as an index, but also as an organizer for records in a file. In a **B+-tree file organization**, the leaf nodes of the tree store records, instead of storing pointers to records. Figure 14.22 shows an example of a B+-tree file organization. Since records are usually larger than pointers, the maximum number of records that can be stored in a leaf node is less than the number of pointers in a nonleaf node. However, the leaf nodes are still required to be at least half full.

Insertion and deletion of records from a B+-tree file organization are handled in the same way as insertion and deletion of entries in a B+-tree index. When a record with a given key value v is inserted, the system locates the block that should contain the record by searching the B+-tree for the largest key in the tree that is $\leq v$. If the block located has enough free space for the record, the system stores the record in the block. Otherwise, as in B+-tree insertion, the system splits the block in two and redistributes the records in it (in the B+-tree–key order) to create space for the new record. The split propagates up the B+-tree in the normal fashion. When we delete a record, the system first removes it from the block containing it. If a block B becomes less than half full as a result, the records in B are redistributed with the records in an adjacent block B'. Assuming fixed-sized records, each block will hold at least one-half as many records as the maximum that it can hold. The system updates the nonleaf nodes of the B+-tree in the usual fashion.

When we use a B+-tree for file organization, space utilization is particularly important, since the space occupied by the records is likely to be much more than the space occupied by keys and pointers. We can improve the utilization of space in a B+-tree by involving more sibling nodes in redistribution during splits and merges. The technique is applicable to both leaf nodes and nonleaf nodes, and it works as follows:

During insertion, if a node is full, the system attempts to redistribute some of its entries to one of the adjacent nodes, to make space for a new entry. If this attempt fails because the adjacent nodes are themselves full, the system splits the node and divides the entries evenly among one of the adjacent nodes and the two nodes that it obtained by splitting the original node. Since the three nodes together contain one more record

than can fit in two nodes, each node will be about two-thirds full. More precisely, each node will have at least $\lfloor 2n/3 \rfloor$ entries, where n is the maximum number of entries that the node can hold. ($\lfloor x \rfloor$ denotes the greatest integer that is less than or equal to x; that is, we drop the fractional part, if any.)

During deletion of a record, if the occupancy of a node falls below $\lfloor 2n/3 \rfloor$, the system attempts to borrow an entry from one of the sibling nodes. If both sibling nodes have $\lfloor 2n/3 \rfloor$ records, instead of borrowing an entry, the system redistributes the entries in the node and in the two siblings evenly between two of the nodes and deletes the third node. We can use this approach because the total number of entries is $3\lfloor 2n/3 \rfloor - 1$, which is less than $2n$. With three adjacent nodes used for redistribution, each node can be guaranteed to have $\lfloor 3n/4 \rfloor$ entries. In general, if m nodes ($m-1$ siblings) are involved in redistribution, each node can be guaranteed to contain at least $\lfloor (m-1)n/m \rfloor$ entries. However, the cost of update becomes higher as more sibling nodes are involved in the redistribution.

Note that in a B^+-tree index or file organization, leaf nodes that are adjacent to each other in the tree may be located at different places on disk. When a file organization is newly created on a set of records, it is possible to allocate blocks that are mostly contiguous on disk to leaf nodes that are contiguous in the tree. Thus, a sequential scan of leaf nodes would correspond to a mostly sequential scan on disk. As insertions and deletions occur on the tree, sequentiality is increasingly lost, and sequential access has to wait for disk seeks increasingly often. An index rebuild may be required to restore sequentiality.

B^+-tree file organizations can also be used to store large objects, such as SQL clobs and blobs, which may be larger than a disk block, and as large as multiple gigabytes. Such large objects can be stored by splitting them into sequences of smaller records that are organized in a B^+-tree file organization. The records can be sequentially numbered, or numbered by the byte offset of the record within the large object, and the record number can be used as the search key.

14.4.2 Secondary Indices and Record Relocation

Some file organizations, such as the B^+-tree file organization, may change the location of records even when the records have not been updated. As an example, when a leaf node is split in a B^+-tree file organization, a number of records are moved to a new node. In such cases, all secondary indices that store pointers to the relocated records would have to be updated, even though the values in the records may not have changed. Each leaf node may contain a fairly large number of records, and each of them may be in different locations on each secondary index. Thus, a leaf-node split may require tens or even hundreds of I/O operations to update all affected secondary indices, making it a very expensive operation.

A widely used solution for this problem is as follows: In secondary indices, in place of pointers to the indexed records, we store the values of the primary-index search-key

attributes. For example, suppose we have a primary index on the attribute *ID* of relation *instructor*; then a secondary index on *dept_name* would store with each department name a list of instructor's *ID* values of the corresponding records, instead of storing pointers to the records.

Relocation of records because of leaf-node splits then does not require any update on any such secondary index. However, locating a record using the secondary index now requires two steps: First we use the secondary index to find the primary-index search-key values, and then we use the primary index to find the corresponding records.

This approach thus greatly reduces the cost of index update due to file reorganization, although it increases the cost of accessing data using a secondary index.

14.4.3 Indexing Strings

Creating B^+-tree indices on string-valued attributes raises two problems. The first problem is that strings can be of variable length. The second problem is that strings can be long, leading to a low fanout and a correspondingly increased tree height.

With variable-length search keys, different nodes can have different fanouts even if they are full. A node must then be split if it is full, that is, there is no space to add a new entry, regardless of how many search entries it has. Similarly, nodes can be merged or entries redistributed depending on what fraction of the space in the nodes is used, instead of being based on the maximum number of entries that the node can hold.

The fanout of nodes can be increased by using a technique called **prefix compression**. With prefix compression, we do not store the entire search key value at nonleaf nodes. We only store a prefix of each search key value that is sufficient to distinguish between the key values in the subtrees that it separates. For example, if we had an index on names, the key value at a nonleaf node could be a prefix of a name; it may suffice to store "Silb" at a nonleaf node, instead of the full "Silberschatz" if the closest values in the two subtrees that it separates are, say, "Silas" and "Silver" respectively.

14.4.4 Bulk Loading of B^+-Tree Indices

As we saw earlier, insertion of a record in a B^+-tree requires a number of I/O operations that in the worst case is proportional to the height of the tree, which is usually fairly small (typically five or less, even for large relations).

Now consider the case where a B^+-tree is being built on a large relation. Suppose the relation is significantly larger than main memory, and we are constructing a nonclustering index on the relation such that the index is also larger than main memory. In this case, as we scan the relation and add entries to the B^+-tree, it is quite likely that each leaf node accessed is not in the database buffer when it is accessed, since there is no particular ordering of the entries. With such randomly ordered accesses to blocks, each time an entry is added to the leaf, a disk seek will be required to fetch the block containing the leaf node. The block will probably be evicted from the disk buffer before another entry is added to the block, leading to another disk seek to write the block back

to disk. Thus, a random read and a random write operation may be required for each entry inserted.

For example, if the relation has 100 million records, and each I/O operation takes about 10 milliseconds on a magnetic disk, it would take at least 1 million seconds to build the index, counting only the cost of reading leaf nodes, not even counting the cost of writing the updated nodes back to disk. This is clearly a very large amount of time; in contrast, if each record occupies 100 bytes, and the disk subsystem can transfer data at 50 megabytes per second, it would take just 200 seconds to read the entire relation.

Insertion of a large number of entries at a time into an index is referred to as **bulk loading** of the index. An efficient way to perform bulk loading of an index is as follows: First, create a temporary file containing index entries for the relation, then sort the file on the search key of the index being constructed, and finally scan the sorted file and insert the entries into the index. There are efficient algorithms for sorting large relations, described later in Section 15.4, which can sort even a large file with an I/O cost comparable to that of reading the file a few times, assuming a reasonable amount of main memory is available.

There is a significant benefit to sorting the entries before inserting them into the B^+-tree. When the entries are inserted in sorted order, all entries that go to a particular leaf node will appear consecutively, and the leaf needs to be written out only once; nodes will never have to be read from disk during bulk load, if the B^+-tree was empty to start with. Each leaf node will thus incur only one I/O operation even though many entries may be inserted into the node. If each leaf contains 100 entries, the leaf level will contain 1 million nodes, resulting in only 1 million I/O operations for creating the leaf level. Even these I/O operations can be expected to be sequential, if successive leaf nodes are allocated on successive disk blocks, and few disk seeks would be required. With magnetic disks, 1 millisecond per block is a reasonable estimate for mostly sequential I/O operations, in contrast to 10 milliseconds per block for random I/O operations.

We shall study the cost of sorting a large relation later, in Section 15.4, but as a rough estimate, the index which would have otherwise taken up to 1,000,000 seconds to build on a magnetic disk can be constructed in well under 1000 seconds by sorting the entries before inserting them into the B^+-tree.

If the B^+-tree is initially empty, it can be constructed faster by building it bottom-up, from the leaf level, instead of using the usual insert procedure. In **bottom-up B^+-tree construction**, after sorting the entries as we just described, we break up the sorted entries into blocks, keeping as many entries in a block as can fit in the block; the resulting blocks form the leaf level of the B^+-tree. The minimum value in each block, along with the pointer to the block, is used to create entries in the next level of the B^+-tree, pointing to the leaf blocks. Each further level of the tree is similarly constructed using the minimum values associated with each node one level below, until the root is created. We leave details as an exercise for the reader.

Most database systems implement efficient techniques based on sorting of entries, and bottom-up construction, when creating an index on a relation, although they use

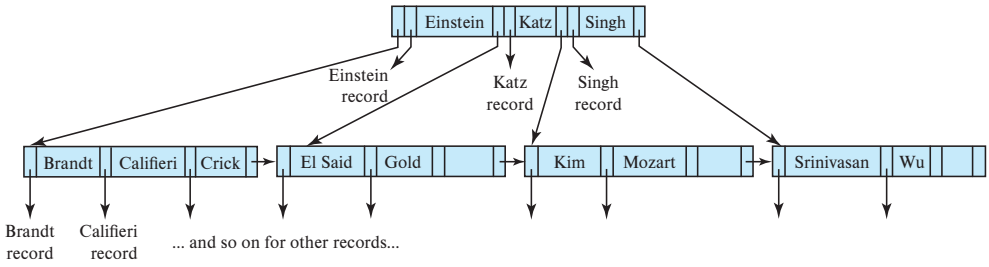


Figure 14.23 B-tree equivalent of B+-tree in Figure 14.9.

the normal insertion procedure when tuples are added one at a time to a relation with an existing index. Some database systems recommend that if a very large number of tuples are added at once to an already existing relation, indices on the relation (other than any index on the primary key) should be dropped, and then re-created after the tuples are inserted, to take advantage of efficient bulk-loading techniques.

14.4.5 B-Tree Index Files

B-tree indices are similar to B+-tree indices. The primary distinction between the two approaches is that a B-tree eliminates the redundant storage of search-key values. In the B+-tree of Figure 14.9, the search keys "Einstein", "Gold", "Mozart", and "Srinivasan" appear in nonleaf nodes, in addition to appearing in the leaf nodes. Every search-key value appears in some leaf node; several are repeated in nonleaf nodes.

A B-tree allows search-key values to appear only once (if they are unique), unlike a B+-tree, where a value may appear in a nonleaf node, in addition to appearing in a leaf node. Figure 14.23 shows a B-tree that represents the same search keys as the B+-tree of Figure 14.9. Since search keys are not repeated in the B-tree, we may be able to store the index in fewer tree nodes than in the corresponding B+-tree index. However, since search keys that appear in nonleaf nodes appear nowhere else in the B-tree, we are forced to include an additional pointer field for each search key in a nonleaf node. These additional pointers point to either file records or buckets for the associated search key.

It is worth noting that many database system manuals, articles in industry literature, and industry professionals use the term B-tree to refer to the data structure that we call the B+-tree. In fact, it would be fair to say that in current usage, the term B-tree is assumed to be synonymous with B+-tree. However, in this book we use the terms B-tree and B+-tree as they were originally defined, to avoid confusion between the two data structures.

A generalized B-tree leaf node appears in Figure 14.24a; a nonleaf node appears in Figure 14.24b. Leaf nodes are the same as in B+-trees. In nonleaf nodes, the pointers P_i are the tree pointers that we used also for B+-trees, while the pointers B_i are bucket or file-record pointers. In the generalized B-tree in the figure, there are $n - 1$ keys in

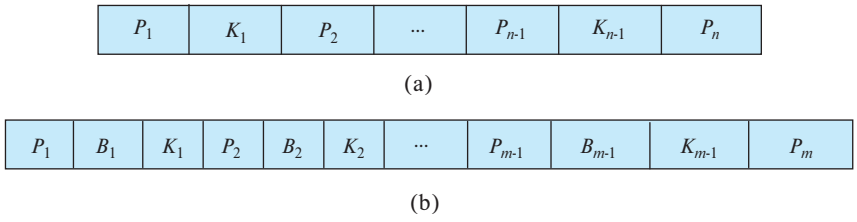


Figure 14.24 Typical nodes of a B-tree. (a) Leaf node. (b) Nonleaf node.

the leaf node, but there are $m - 1$ keys in the nonleaf node. This discrepancy occurs because nonleaf nodes must include pointers B_i, thus reducing the number of search keys that can be held in these nodes. Clearly, $m < n$, but the exact relationship between m and n depends on the relative size of search keys and pointers.

The number of nodes accessed in a lookup in a B-tree depends on where the search key is located. A lookup on a B^+-tree requires traversal of a path from the root of the tree to some leaf node. In contrast, it is sometimes possible to find the desired value in a B-tree before reaching a leaf node. However, roughly n times as many keys are stored in the leaf level of a B-tree as in the nonleaf levels, and, since n is typically large, the benefit of finding certain values early is relatively small. Moreover, the fact that fewer search keys appear in a nonleaf B-tree node, compared to B^+-trees, implies that a B-tree has a smaller fanout and therefore may have depth greater than that of the corresponding B^+-tree. Thus, lookup in a B-tree is faster for some search keys but slower for others, although, in general, lookup time is still proportional to the logarithm of the number of search keys.

Deletion in a B-tree is more complicated. In a B^+-tree, the deleted entry always appears in a leaf. In a B-tree, the deleted entry may appear in a nonleaf node. The proper value must be selected as a replacement from the subtree of the node containing the deleted entry. Specifically, if search key K_i is deleted, the smallest search key appearing in the subtree of pointer P_{i+1} must be moved to the field formerly occupied by K_i. Further actions need to be taken if the leaf node now has too few entries. In contrast, insertion in a B-tree is only slightly more complicated than is insertion in a B^+-tree.

The space advantages of B-trees are marginal for large indices and usually do not outweigh the disadvantages that we have noted. Thus, pretty much all database-system implementations use the B^+-tree data structure, even if (as we discussed earlier) they refer to the data structure as a B-tree.

14.4.6 Indexing on Flash Storage

In our description of indexing so far, we have assumed that data are resident on magnetic disks. Although this assumption continues to be true for the most part, flash storage capacities have grown significantly, and the cost of flash storage per gigabyte has dropped correspondingly, and flash based SSD storage has now replaced magnetic-disk storage for many applications.

Standard B+-tree indices can continue to be used even on SSDs, with acceptable update performance and significantly improved lookup performance compared to disk storage.

Flash storage is structured as pages, and the B+-tree index structure can be used with flash based SSDs. SSDs provide much faster random I/O operations than magnetic disks, requiring only around 20 to 100 microseconds for a random page read, instead of about 5 to 10 milliseconds with magnetic disks. Thus, lookups run much faster with data on SSDs, compared to data on magnetic disks.

The performance of write operations is more complicated with flash storage. An important difference between flash storage and magnetic disks is that flash storage does not permit in-place updates to data at the physical level, although it appears to do so logically. Every update turns into a copy+write of an entire flash-storage page, requiring the old copy of the page to be erased subsequently. A new page can be written in 20 to 100 microseconds, but eventually old pages need to be erased to free up the pages for further writes. Erases are done at the level of blocks containing multiple pages, and a block erase takes 2 to 5 milliseconds.

The optimum B+-tree node size for flash storage is smaller than that with magnetic disk, since flash pages are smaller than disk blocks; it makes sense for tree-node sizes to match to flash pages, since larger nodes would lead to multiple page writes when a node is updated. Although smaller pages lead to taller trees and more I/O operations to access data, random page reads are so much faster with flash storage that the overall impact on read performance is quite small.

Although random I/O is much cheaper with SSDs than with magnetic disks, bulk loading still provides significant performance benefits, compared to tuple-at-a-time insertion, with SSDs. In particular, bottom-up construction reduces the number of page writes compared to tuple-at-a-time insertion, even if the entries are sorted on the search key. Since page writes on flash cannot be done in place and require relatively expensive block erases at a later point in time, the reduction of number of page writes with bottom-up B+-tree construction provides significant performance benefits.

Several extensions and alternatives to B+-trees have been proposed for flash storage, with a focus on reducing the number of erase operations that result due to page rewrites. One approach is to add buffers to internal nodes of B+-trees and record updates temporarily in buffers at higher levels, pushing the updates down to lower levels lazily. The key idea is that when a page is updated, multiple updates are applied together, reducing the number of page writes per update. Another approach creates multiple trees and merges them; the log-structured merge tree and its variants are based on this idea. In fact, both these approaches are also useful for reducing the cost of writes on magnetic disks; we outline both these approaches in Section 14.8.

14.4.7 Indexing in Main Memory

Main memory today is large and cheap enough that many organizations can afford to buy enough main memory to fit all their operational data in-memory. B+-trees can

be used to index in-memory data, with no change to the structure. However, some optimizations are possible.

First, since memory is costlier than disk space, internal data structures in main memory databases have to be designed to reduce space requirements. Techniques that we saw in Section 14.4.1 to improve B^+-tree storage utilization can be used to reduce memory usage for in-memory B^+-trees.

Data structures that require traversal of multiple pointers are acceptable for in-memory data, unlike in the case of disk-based data, where the cost of the I/Os to traverse multiple pages would be excessively high. Thus, tree structures in main memory databases can be relatively deep, unlike B^+-trees.

The speed difference between cache memory and main memory, and the fact that data are transferred between main memory and cache in units of a *cache-line* (typically about 64 bytes), results in a situation where the relationship between cache and main memory is not dissimilar to the relationship between main memory and disk (although with smaller speed differences). When reading a memory location, if it is present in cache the CPU can complete the read in 1 or 2 nanoseconds, whereas a cache miss results in about 50 to 100 nanoseconds of delay to read data from main memory.

B^+-trees with small nodes that fit in a cache line have been found to provide very good performance with in-memory data. Such B^+-trees allow index operations to be completed with far fewer cache misses than tall, skinny tree structures such as binary trees, since each node traversal is likely to result in a cache miss. Compared to B^+-trees with nodes that match cache lines, trees with large nodes also tend to have more cache misses since locating data within a node requires either a full scan of the node content, spanning multiple cache lines, or a binary search, which also results in multiple cache misses.

For databases where data do not fit entirely in memory, but frequently used data are often memory resident, the following idea is used to create B^+-tree structures that offer good performance on disk as well as in-memory. Large nodes are used to optimize disk-based access, but instead of treating data in a node as single large array of keys and pointers, the data within a node are structured as a tree, with smaller nodes that match the size of a cache line. Instead of scanning data linearly or using binary search within a node, the tree-structure within the large B^+-tree node is used to access the data with a minimal number of cache misses.

14.5 Hash Indices

Hashing is a widely used technique for building indices in main memory; such indices may be transiently created to process a join operation (as we will see in Section 15.5.5) or may be a permanent structure in a main memory database. Hashing has also been used as a way of organizing records in a file, although hash file organizations are not very widely used. We initially consider only in-memory hash indices, and we consider disk-based hashing later in this section.

In our description of hashing, we shall use the term **bucket** to denote a unit of storage that can store one or more records. For in-memory hash indices, a bucket could be a linked list of index entries or records. For disk-based indices, a bucket would be a linked list of disk blocks. In a **hash file organization**, instead of record pointers, buckets store the actual records; such structures only make sense with disk-resident data. The rest of our description does not depend on whether the buckets store record pointers or actual records.

Formally, let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A **hash function** h is a function from K to B. Let h denote a hash function. With in-memory hash indices, the set of buckets is simply an array of pointers, with the ith bucket at offset i. Each pointer stores the head of a linked list containing the entries in that bucket.

To insert a record with search key K_i, we compute $h(K_i)$, which gives the address of the bucket for that record. We add the index entry for the record to the list at offset i. Note that there are other variants of hash indices that handle the case of multiple records in a bucket differently; the form described here is the most widely used variant and is called **overflow chaining**.

Hash indexing using overflow chaining is also called **closed addressing** (or, less commonly, **closed hashing**). An alternative hashing scheme called open addressing is used in some applications, but is not suitable for most database indexing applications since open addressing does not support deletes efficiently. We do not consider it further.

Hash indices efficiently support equality queries on search keys. To perform a lookup on a search-key value K_i, we simply compute $h(K_i)$, then search the bucket with that address. Suppose that two search keys, K_5 and K_7, have the same hash value; that is, $h(K_5) = h(K_7)$. If we perform a lookup on K_5, the bucket $h(K_5)$ contains records with search-key values K_5 and records with search-key values K_7. Thus, we have to check the search-key value of every record in the bucket to verify that the record is one that we want.

Unlike B^+-tree indices, hash indices do not support range queries; for example, a query that wishes to retrieve all search key values v such that $l \leq v \leq u$ cannot be efficiently answered using a hash index.

Deletion is equally straightforward. If the search-key value of the record to be deleted is K_i, we compute $h(K_i)$, then search the corresponding bucket for that record and delete the record from the bucket. With a linked list representation, deletion from the linked list is straightforward.

In a disk-based hash index, when we insert a record, we locate the bucket by using hashing on the search key, as described earlier. Assume for now that there is space in the bucket to store the record. Then, the record is stored in that bucket. If the bucket does not have enough space, a **bucket overflow** is said to occur. We handle bucket overflow by using **overflow buckets**. If a record must be inserted into a bucket b, and b is already full, the system provides an overflow bucket for b and inserts the record into the overflow bucket. If the overflow bucket is also full, the system provides another overflow bucket, and so on. All the overflow buckets of a given bucket are chained together in a linked

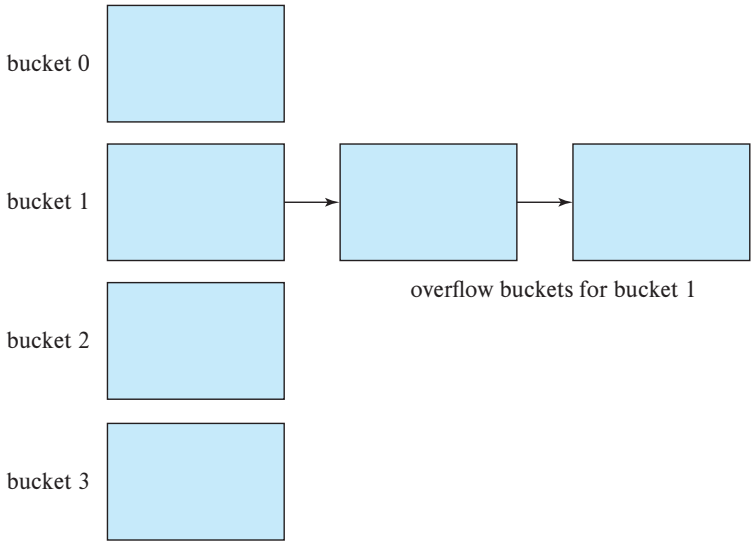


Figure 14.25 Overflow chaining in a disk-based hash structure.

list, as in Figure 14.25. With overflow chaining, given search key k, the lookup algorithm must then search not only bucket $h(k)$, but also the overflow buckets linked from bucket $h(k)$.

Bucket overflow can occur if there are insufficient buckets for the given number of records. If the number of records that are indexed is known ahead of time, the required number of buckets can be allocated; we will shortly see how to deal with situations where the number of records becomes significantly more than what was initially anticipated. Bucket overflow can also occur if some buckets are assigned more records than are others, resulting in one bucket overflowing even when other buckets still have a lot of free space.

Such **skew** in the distribution of records can occur if multiple records may have the same search key. But even if there is only one record per search key, skew may occur if the chosen hash function results in nonuniform distribution of search keys. This chance of this problem can be minimized by choosing hash functions carefully, to ensure the distribution of keys across buckets is uniform and random. Nevertheless, some skew may occur.

So that the probability of bucket overflow is reduced, the number of buckets is chosen to be $(n_r/f_r) * (1 + d)$, where n_r denotes the number of records, f_r denotes the number of records per bucket, d is a fudge factor, typically around 0.2. With a fudge factor of 0.2, about 20 percent of the space in the buckets will be empty. But the benefit is that the probability of overflow is reduced.

Despite allocation of a few more buckets than required, bucket overflow can still occur, especially if the number of records increases beyond what was initially expected.

Hash indexing as described above, where the number of buckets is fixed when the index is created, is called **static hashing**. One of the problems with static hashing is that we need to know how many records are going to be stored in the index. If over time a large number of records are added, resulting in far more records than buckets, lookups would have to search through a large number of records stored in a single bucket, or in one or more overflow buckets, and would thus become inefficient.

To handle this problem, the hash index can be rebuilt with an increased number of buckets. For example, if the number of records becomes twice the number of buckets, the index can be rebuilt with twice as many buckets as before. However, rebuilding the index has the drawback that it can take a long time if the relations are large, causing disruption of normal processing. Several schemes have been proposed that allow the number of buckets to be increased in a more incremental fashion. Such schemes are called **dynamic hashing** techniques; the *linear hashing* technique and the *extendable hashing* technique are two such schemes; see Section 24.5 for further details of these techniques.

14.6 Multiple-Key Access

Until now, we have assumed implicitly that only one index on one attribute is used to process a query on a relation. However, for certain types of queries, it is advantageous to use multiple indices if they exist, or to use an index built on a multiattribute search key.

14.6.1 Using Multiple Single-Key Indices

Assume that the *instructor* file has two indices: one for *dept_name* and one for *salary*. Consider the following query: "Find all instructors in the Finance department with salary equal to $80,000." We write

```
select ID
from instructor
where dept_name = 'Finance' and salary = 80000;
```

There are three strategies possible for processing this query:

1. Use the index on *dept_name* to find all records pertaining to the Finance department. Examine each such record to see whether *salary* = 80000.
2. Use the index on *salary* to find all records pertaining to instructors with salary of $80,000. Examine each such record to see whether the department name is "Finance".

3. Use the index on *dept_name* to find *pointers* to all records pertaining to the Finance department. Also, use the index on *salary* to find pointers to all records pertaining to instructors with a salary of $80,000. Take the intersection of these two sets of pointers. Those pointers that are in the intersection point to records pertaining to instructors of the Finance department and with salary of $80,000.

The third strategy is the only one of the three that takes advantage of the existence of multiple indices. However, even this strategy may be a poor choice if all of the following hold:

- There are many records pertaining to the Finance department.
- There are many records pertaining to instructors with a salary of $80,000.
- There are only a few records pertaining to *both* the Finance department and instructors with a salary of $80,000.

If these conditions hold, we must scan a large number of pointers to produce a small result. An index structure called a "bitmap index" can in some cases greatly speed up the intersection operation used in the third strategy. Bitmap indices are outlined in Section 14.9.

14.6.2 Indices on Multiple Keys

An alternative strategy for this case is to create and use an index on a composite search key (*dept_name*, *salary*)—that is, the search key consisting of the department name concatenated with the instructor salary.

We can use an ordered (B^+-tree) index on the preceding composite search key to answer efficiently queries of the form

```
select ID
from instructor
where dept_name = 'Finance' and salary = 80000;
```

Queries such as the following query, which specifies an equality condition on the first attribute of the search key (*dept_name*) and a range on the second attribute of the search key (*salary*), can also be handled efficiently since they correspond to a range query on the search attribute.

```
select ID
from instructor
where dept_name = 'Finance' and salary < 80000;
```

We can even use an ordered index on the search key (*dept_name*, *salary*) to answer the following query on only one attribute efficiently:

```
select ID
from instructor
where dept_name = 'Finance';
```

An equality condition *dept_name* = "Finance" is equivalent to a range query on the range with lower end (Finance, $-\infty$) and upper end (Finance, $+\infty$). Range queries on just the *dept_name* attribute can be handled in a similar manner.

The use of an ordered-index structure on a composite search key, however, has a few shortcomings. As an illustration, consider the query

```
select ID
from instructor
where dept_name < 'Finance' and salary < 80000;
```

We can answer this query by using an ordered index on the search key (*dept_name*, *salary*): For each value of *dept_name* that is less than "Finance" in alphabetic order, the system locates records with a *salary* value of 80000. However, each record is likely to be in a different disk block, because of the ordering of records in the file, leading to many I/O operations.

The difference between this query and the previous two queries is that the condition on the first attribute (*dept_name*) is a comparison condition, rather than an equality condition. The condition does not correspond to a range query on the search key.

To speed the processing of general composite search-key queries (which can involve one or more comparison operations), we can use several special structures. We shall consider *bitmap indices* in Section 14.9. There is another structure, called the *R-tree*, that can be used for this purpose. The R-tree is an extension of the B^+-tree to handle indexing on multiple dimensions and is discussed in Section 14.10.1.

14.6.3 Covering Indices

Covering indices are indices that store the values of some attributes (other than the search-key attributes) along with the pointers to the record. Storing extra attribute values is useful with secondary indices, since they allow us to answer some queries using just the index, without even looking up the actual records.

For example, suppose that we have a nonclustering index on the *ID* attribute of the *instructor* relation. If we store the value of the *salary* attribute along with the record pointer, we can answer queries that require the salary (but not the other attribute, *dept_name*) without accessing the *instructor* record.

The same effect could be obtained by creating an index on the search key (*ID*, *salary*), but a covering index reduces the size of the search key, allowing a larger fanout in the nonleaf nodes, and potentially reducing the height of the index.

14.7 Creation of Indices

Although the SQL standard does not specify any specific syntax for creation of indices, most databases support SQL commands to create and drop indices. As we saw in Section 4.6, indices can be created using the following syntax, which is supported by most databases.

create index <index-name> **on** <relation-name> (<attribute-list>);

The *attribute-list* is the list of attributes of the relations that form the search key for the index. Indices can be dropped using a command of the form

drop index <index-name>;

For example, to define an index named *dept_index* on the *instructor* relation with *dept_name* as the search key, we write:

create index *dept_index* **on** *instructor* (*dept_name*);

To declare that an attribute or list of attributes is a candidate key, we can use the syntax **create unique index** in place of **create index** above. Databases that support multiple types of indices also allow the type of index to be specified as part of the index creation command. Refer to the manual of your database system to find out what index types are available, and the syntax for specifying the index type.

When a user submits an SQL query that can benefit from using an index, the SQL query processor automatically uses the index.

Indices can be very useful on attributes that participate in selection conditions or join conditions of queries, since they can reduce the cost of queries significantly. Consider a query that retrieves *takes* records for a particular student ID 12345 (expressed in relational algebra as $\sigma_{ID=12345}(takes)$). If there were an index on the ID attribute of *takes*, pointers to the required records could be obtained with only a few I/O operations. Since students typically only take a few tens of courses, even fetching the actual records would take only a few tens of I/O operations subsequently. In contrast, in the absence of this index, the database system would be forced to read all *takes* records and select those with matching ID values. Reading an entire relation can be very expensive if there are a large number of students.

However, indices do have a cost, since they have to be updated whenever there is an update to the underlying relation. Creating too many indices would slow down update processing, since each update would have to also update all affected indices.

Sometimes performance problems are apparent during testing, for example, if a query takes tens of seconds, it is clear that it is quite slow. However, suppose each query takes 1 second to scan a large relation without an index, versus 10 milliseconds to retrieve the same records using an index. If testers run one query at a time, queries

respond quickly, even without an index. However, suppose that the queries are part of a registration system that is used by a thousand students in an hour, and the actions of each student require 10 such queries to be executed. The total execution time would then be 10,000 seconds for queries submitted in 1 hour, that is, 3600 seconds. Students are then likely to find that the registration system is extremely slow, or even totally unresponsive. In contrast, if the required indices were present, the execution time required would be 100 seconds for queries submitted in 1 hour, and the performance of the registration system would be very good.

It is therefore important when building an application to figure out which indices are important for performance and to create them before the application goes live.

If a relation is declared to have a primary key, most database systems automatically create an index on the primary key. Whenever a tuple is inserted into the relation, the index can be used to check that the primary-key constraint is not violated (i.e., there are no duplicates on the primary-key value). Without the index on the primary key, whenever a tuple is inserted, the entire relation has to be scanned to ensure that the primary-key constraint is satisfied.

Although most database systems do not automatically create them, it is often a good idea to create indices on foreign-key attributes, too. Most joins are between foreign-key and primary-key attributes, and queries containing such joins, where there is also a selection condition on the referenced table, are not uncommon. Consider a query $takes \bowtie \sigma_{name=\text{Shankar}}(student)$, where the foreign-key attribute ID of *takes* references the primary-key attribute ID of student. Since very few students are likely to be named Shankar, the index on the foreign-key attribute *takes*.ID can be used to efficiently retrieve the *takes* tuples corresponding to these students.

Many database systems provide tools that help database administrators track what queries and updates are being executed on the system and recommend the creation of indices depending on the frequencies of the queries and updates. Such tools are referred to as index tuning wizards or advisors.

Some recent cloud-based database systems also support completely automated creation of indices whenever the system finds that doing so would avoid repeated relation scans, without the intervention of a database administrator.

14.8 Write-Optimized Index Structures

One of the drawbacks of the B^+-tree index structure is that performance can be quite poor with random writes. Consider an index that is too large to fit in memory; since the bulk of the space is at the leaf level, and memory sizes are quite large these days, we assume for simplicity that higher levels of the index fit in memory.

Now suppose writes or inserts are done in an order that does not match the sort order of the index. Then, each write/insert is likely to touch a different leaf node; if the number of leaf nodes is significantly larger than the buffer size, most of these leaf accesses would require a random read operation, as well as a subsequent write operation

to write the updated leaf page back to disk. On a system with a magnetic disk, with a 10-millisecond access time, the index would support not more than 100 writes/inserts per second per disk; and this is an optimistic estimate, assuming that the seek takes the bulk of the time, and the head has not moved between the read and the write of a leaf page. On a system with flash based SSDs, random I/O is much faster, but a page write still has a significant cost since it (eventually) requires a page erase, which is an expensive operation. Thus, the basic B^+-tree structure is not ideal for applications that need to support a very large number of random writes/inserts per second.

Several alternative index structures have been proposed to handle workloads with a high write/insert rate. The **log-structured merge tree** or LSM tree and its variants are write-optimized index structures that have seen very significant adoption. The buffer tree is an alternative approach, which can be used with a variety of search tree structures. We outline these structures in the rest of this section.

14.8.1 LSM Trees

An LSM tree consists of several B^+-trees, starting with an in-memory tree, called L_0, and on-disk trees $L_1, L_2, \ldots, L_k$ for some k, where k is called the level. Figure 14.26 depicts the structure of an LSM tree for $k = 3$.

An index lookup is performed by using separate lookup operations on each of the trees $L_0, \ldots, L_k$, and merging the results of the lookups. (We assume for now that there are only inserts, and no updates or deletes; index lookups in the presence of updates/deletes are more complicated and are discussed later.)

When a record is first inserted into an LSM tree, it is inserted into the in-memory B^+-tree structure L_0. A fairly large amount of memory space is allocated for this tree. The tree grows as more inserts are processed, until it fills the memory allocated to it. At this point, we need to move data from the in-memory structure to a B^+-tree on disk.

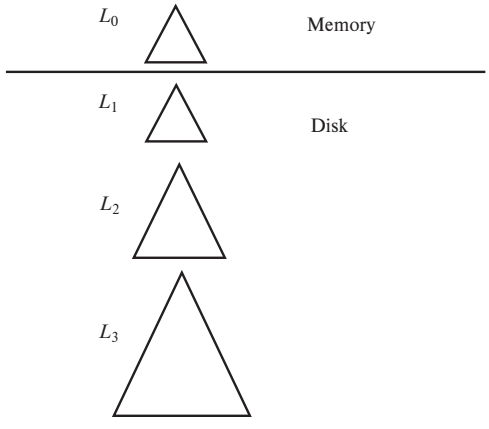


Figure 14.26 Log-structured merge tree with three levels.

If tree L_1 is empty, the entire in-memory tree L_0 is written to disk to create the initial tree L_1. However, if L_1 is not empty, the leaf level of L_0 is scanned in increasing key order, and entries are merged with the leaf level entries of L_1 (also scanned in increasing key order). The merged entries are used to create a new B^+-tree, using the bottom-up build process. The new tree with the merged entries then replaces the old L_1. In either case, after entries of L_0 have been moved to L_1, all entries in L_0 as well as the old L_1, if it existed, are deleted. Inserts can then be made to the now empty L_0 in-memory.

Note that all entries in the leaf level of the old L_1 tree, including those in leaf nodes that do not have any updates, are copied to the new tree instead of performing updates on the existing L_1 tree node. This gives the following benefits.

1. The leaves of the new tree are sequentially located, avoiding random I/O during subsequent merges.
2. The leaves are full, avoiding the overhead of partially occupied leaves that can occur with page splits.

There is, however, a cost to using the LSM structure as described above: the entire contents of the tree are copied each time a set of entries from L_0 are copied into L_1. One of two techniques is used to reduce this cost:

1. Multiple levels are used, with level L_{i+1} trees having a maximum size that is k times the maximum size of level L_i trees. Thus, each record is written at most k times at a particular level. The number of levels is proportional $\log_k(I/M)$ where I is the number of entries and M is the number of entries that fit in the in-memory tree L_0.
2. Each level (other than L_0) can have up to some number b of trees, instead of just 1 tree. When an L_0 tree is written to disk, a new L_1 tree is created instead of merging it with an existing L_1 tree. When there are b such L_1 trees, they are merged into a single new L_2 tree. Similarly, when there are b trees at level L_i they are merged into a new L_{i+1} tree.

   This variant of the LSM tree is called a **stepped-merge index**. The stepped-merge index decreases the insert cost significantly compared to having only one tree per level, but it can result in an increase in query cost, since multiple trees may need to be searched. Bitmap-based structures called *Bloom filters*, described in Section 24.1, are used to reduce the number of lookups by efficiently detecting that a search key is not present in a particular tree. Bloom filters occupy very little space, but they are quite effective at reducing query cost.

Details of all these variants of LSM trees can be found in Section 24.2.

So far we have only described inserts and lookups. Deletes are handled in an interesting manner. Instead of directly finding an index entry and deleting it, deletion

results in insertion of a new **deletion entry** that indicates which index entry is to be deleted. The process of inserting a deletion entry is identical to the process of inserting a normal index entry.

However, lookups have to carry out an extra step. As mentioned earlier, lookups retrieve entries from all the trees and merge them in sorted order of key value. If there is a deletion entry for some entry, both of them would have the same key value. Thus, a lookup would find both the deletion entry and the original entry for that key, which is to be deleted. If a deletion entry is found, the to-be-deleted entry is filtered out and not returned as part of the lookup result.

When trees are merged, if one of the trees contains an entry, and the other had a matching deletion entry, the entries get matched up during the merge (both would have the same key), and are both discarded.

Updates are handled in a manner similar to deletes, by inserting an update entry. Lookups need to match update entries with the original entries and return the latest value. The update is actually applied during a merge, when one tree has an entry and another has its matching update entry; the merge process would find a record and an update record with the same key, apply the update, and discard the update entry.

LSM trees were initially designed to reduce the write and seek overheads of magnetic disks. Flash based SSDs have a relatively low overhead for random I/O operations since they do not require seek, and thus the benefit of avoiding random I/O that LSM tree variants provide is not particularly important with SSDs.

However, recall that flash memory does not allow in-place update, and writing even a single byte to a page requires the whole page to be rewritten to a new physical location; the original location of the page needs to be erased eventually, which is a relatively expensive operation. The reduction in number of writes using LSM tree variants, as compared to traditional B^+-trees, can provide substantial performance benefits when LSM trees are used with SSDs.

A variant of the LSM tree similar to the stepped-merge index, with multiple trees in each layer, was used in Google's BigTable system, as well as in Apache HBase, the open source clone of BigTable. These systems are built on top of distributed file systems that allow appends to files but do not support updates to existing data. The fact that LSM trees do not perform in-place update made LSM trees a very good fit for these systems.

Subsequently, a large number of BigData storage systems such as Apache Cassandra, Apache AsterixDB, and MongoDB added support for LSM trees, with most implementing versions with multiple trees in each layer. LSM trees are also supported in MySQL (using the MyRocks storage engine) and in the embedded database systems SQLite4 and LevelDB.

14.8.2 Buffer Tree

The buffer tree is an alternative to the log-structured merge tree approach. The key idea behind the **buffer tree** is to associate a buffer with each internal node of a B^+-tree,

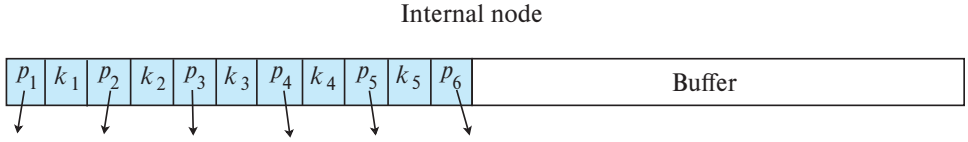


Figure 14.27 Structure of an internal node of a buffer tree.

including the root node; this is depicted pictorially in Figure 14.27. We first outline how inserts and lookups are handled, and subsequently we outline how deletes and updates are handled.

When an index record is inserted into the buffer tree, instead of traversing the tree to the leaf, the index record is inserted into the buffer of the root. If the buffer becomes full, each index record in the buffer is pushed one level down the tree to the appropriate child node. If the child node is an internal node, the index record is added to the child node's buffer; if that buffer is full, all records in that buffer are similarly pushed down. All records in a buffer are sorted on the search key before being pushed down. If the child node is a leaf node, index records are inserted into the leaf in the usual manner. If the insert results in an overfull leaf node, the node is split in the usual B^+-tree manner, with the split potentially propagating to parent nodes. Splitting of an overfull internal node is done in the usual way, with the additional step of also splitting the buffer; the buffer entries are partitioned between the two split nodes based on their key values.

Lookups are done by traversing the B^+-tree structure in the usual way, to find leaves that contain records matching the lookup key. But there is one additional step: at each internal node traversed by a lookup, the node's buffer must be examined to see if there are any records matching the lookup key. Range lookups are done as in a normal B^+-tree, but they must also examine the buffers of all internal nodes above any of the leaf nodes that are accessed.

Suppose the buffer at an internal node holds k times as many records as there are child nodes. Then, on average, k records would be pushed down at a time to each child (regardless of whether the child is an internal node or a leaf node). Sorting of records before they are pushed ensures that all these records are pushed down consecutively. The benefit of the buffer-tree approach for inserts is that the cost of accessing the child node from storage, and of writing the updated node back, is amortized (divided), on average, between k records. With sufficiently large k, the savings can be quite significant compared to inserts in a regular B^+-tree.

Deletes and updates can be processed in a manner similar to LSM trees, using deletion entries or update entries. Alternatively, deletes and updates could be processed using the normal B^+-tree algorithms, at the risk of a higher I/O cost per delete/update as compared to the cost when using deletion/update entries.

Buffer trees provide better worst-case complexity bounds on the number of I/O operations than do LSM tree variants. In terms of read cost, buffer trees are significantly faster than LSM trees. However, write operations on buffer trees involve random I/O,

requiring more seeks, in contrast to sequential I/O operations with LSM tree variants. For magnetic disk storage, the high cost of seeks results in buffer trees performing worse than LSM trees on write-intensive workloads. LSM trees have thus found greater acceptance for write-intensive workloads with data stored on magnetic disk. However, since random I/O operations are very efficient on SSDs, and buffer trees tend to perform fewer write operations overall compared to LSM trees, buffer trees can provide better write performance on SSDs. Several index structures designed for flash storage make use of the buffer concept introduced by buffer trees.

Another benefit of buffer trees is that the key idea of associating buffers with internal nodes, to reduce the number of writes, can be used with any type of tree-structured index. For example, buffering has been used as a way of supporting bulk loading of spatial indices such as R-trees (which we study in Section 14.10.1), as well as other types of indices, for which sorting and bottom-up construction are not applicable.

Buffer trees have been implemented as part of the **Generalized Search Tree** (**GiST**) index structure in PostgreSQL. The GiST index allows user-defined code to be executed to implement search, update, and split operations on nodes and has been used to implement R-trees and other spatial index structures.

14.9 Bitmap Indices

Bitmap indices are a specialized type of index designed for easy querying on multiple keys, although each bitmap index is built on a single key. We describe key features of bitmap indices in this section but provide further details in Section 24.3.

For bitmap indices to be used, records in a relation must be numbered sequentially, starting from, say, 0. Given a number n, it must be easy to retrieve the record numbered n. This is particularly easy to achieve if records are fixed in size and allocated on consecutive blocks of a file. The record number can then be translated easily into a block number and a number that identifies the record within the block.

Consider a relation with an attribute that can take on only one of a small number (e.g., 2 to 20) of values. For instance, consider a relation *instructor_info*, which has (in addition to an ID attribute) an attribute *gender*, which can take only values m (male) or f (female). Suppose the relation also has an attribute *income_level*, which stores the income level, where income has been broken up into five levels: $L1$: 0 – 9999, $L2$: 10,000 – 19,999, $L3$: 20,000 – 39,999, $L4$: 40,000 – 74,999, and $L5$: 75,000 – ∞. Here, the raw data can take on many values, but a data analyst has split the values into a small number of ranges to simplify analysis of the data. An instance of this relation is shown on the left side of Figure 14.28.

A **bitmap** is simply an array of bits. In its simplest form, a **bitmap index** on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation. The ith bit of the bitmap for value v_j is set to 1 if the record numbered i has the value v_j for attribute A. All other bits of the bitmap are set to 0.

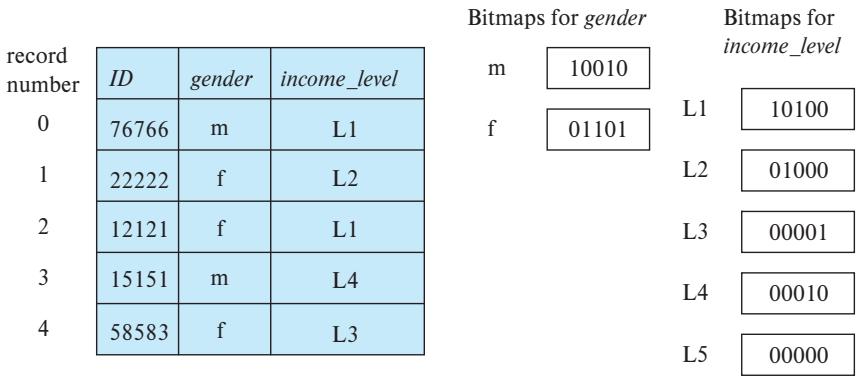


Figure 14.28 Bitmap indices on relation *instructor_info*.

In our example, there is one bitmap for the value m and one for f. The *i*th bit of the bitmap for m is set to 1 if the *gender* value of the record numbered *i* is m. All other bits of the bitmap for m are set to 0. Similarly, the bitmap for f has the value 1 for bits corresponding to records with the value f for the *gender* attribute; all other bits have the value 0. Figure 14.28 shows bitmap indices on the *gender* and *income_level* attributes of *instructor_info* relation, for the relation instance shown in the same figure.

We now consider when bitmaps are useful. The simplest way of retrieving all records with value m (or value f) would be to simply read all records of the relation and select those records with value m (or f, respectively). The bitmap index doesn't really help to speed up such a selection. While it would allow us to read only those records for a specific gender, it is likely that every disk block for the file would have to be read anyway.

In fact, bitmap indices are useful for selections mainly when there are selections on multiple keys. Suppose we create a bitmap index on attribute *income_level*, which we described earlier, in addition to the bitmap index on *gender*.

Consider now a query that selects women with income in the range 10,000 to 19,999. This query can be expressed as

```
select *
from instructor_info
where gender = 'f' and income_level = 'L2';
```

To evaluate this selection, we fetch the bitmaps for *gender* value f and the bitmap for *income_level* value *L2*, and perform an **intersection** (logical-and) of the two bitmaps. In other words, we compute a new bitmap where bit *i* has value 1 if the *i*th bit of the two bitmaps are both 1, and has a value 0 otherwise. In the example in Figure 14.28, the intersection of the bitmap for *gender* = f (01101) and the bitmap for *income_level* = *L2* (01000) gives the bitmap 01000.

Since the first attribute can take two values, and the second can take five values, we would expect only about 1 in 10 records, on an average, to satisfy a combined condition on the two attributes. If there are further conditions, the fraction of records satisfying all the conditions is likely to be quite small. The system can then compute the query result by finding all bits with value 1 in the intersection bitmap and retrieving the corresponding records. If the fraction is large, scanning the entire relation would remain the cheaper alternative.

More detailed coverage of bitmap indices, including how to efficiently implement aggregate operations, how to speed up bitmap operations, and hybrid indices that combine B^+-trees with bitmaps, can be found in Section 24.3.

14.10 Indexing of Spatial and Temporal Data

Traditional index structures, such as hash indices and B^+-trees, are not suitable for indexing of spatial data, which are typically of two or more dimensions. Similarly, when tuples have temporal intervals associated with them, and queries may specify time points or time intervals, the traditional index structures may result in poor performance.

14.10.1 Indexing of Spatial Data

In this section we provide an overview of techniques for indexing spatial data. Further details can be found in Section 24.4. Spatial data refers to data referring to a point or a region in two or higher dimensional space. For example, the location of restaurants, identified by a (latitude, longitude) pair, is a form of spatial data. Similarly, the spatial extent of a farm or a lake can be identified by a polygon, with each corner identified by a (latitude, longitude) pair.

There are many forms of queries on spatial data, which need to be efficiently supported using indices. A query that asks for restaurants at a precisely specified (latitude, longitude) pair can be answered by creating a B^+-tree on the composite attribute (latitude, longitude). However, such a B^+-tree index cannot efficiently answer a query that asks for all restaurants that are within a 500-meter radius of a user's location, which is identified by a (latitude, longitude) pair. Nor can such an index efficiently answer a query that asks for all restaurants that are within a rectangular region of interest. Both of these are forms of **range queries**, which retrieve objects within a specified area. Nor can such an index efficiently answer a query that asks for the nearest restaurant to a specified location; such a query is an example of a **nearest neighbor** query.

The goal of spatial indexing is to support different forms of spatial queries, with range and nearest neighbor queries being of particular interest, since they are widely used.

To understand how to index spatial data consisting of two or more dimensions, we consider first the indexing of points in one-dimensional data. Tree structures, such as binary trees and B^+-trees, operate by successively dividing space into smaller parts. For

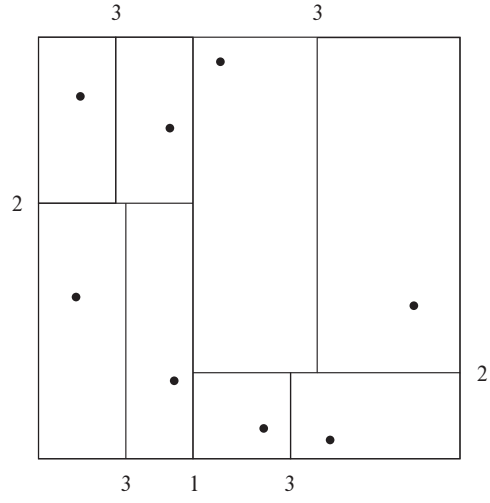


Figure 14.29 Division of space by a k-d tree.

instance, each internal node of a binary tree partitions a one-dimensional interval in two. Points that lie in the left partition go into the left subtree; points that lie in the right partition go into the right subtree. In a balanced binary tree, the partition is chosen so that approximately one-half of the points stored in the subtree fall in each partition. Similarly, each level of a B^+-tree splits a one-dimensional interval into multiple parts.

We can use that intuition to create tree structures for two-dimensional space as well as in higher-dimensional spaces. A tree structure called a **k-d tree** was one of the early structures used for indexing in multiple dimensions. Each level of a k-d tree partitions the space into two. The partitioning is done along one dimension at the node at the top level of the tree, along another dimension in nodes at the next level, and so on, cycling through the dimensions. The partitioning proceeds in such a way that, at each node, approximately one-half of the points stored in the subtree fall on one side and one-half fall on the other. Partitioning stops when a node has less than a given maximum number of points.

Figure 14.29 shows a set of points in two-dimensional space, and a k-d tree representation of the set of points, where the maximum number of points in a leaf node has been set at 1. Each line in the figure (other than the outside box) corresponds to a node in the k-d tree. The numbering of the lines in the figure indicates the level of the tree at which the corresponding node appears.

Rectangular range queries, which ask for points within a specified rectangular region, can be answered efficiently using a k-d tree as follows: Such a query essentially specifies an interval on each dimension. For example, a range query may ask for all points whose x dimension lies between 50 and 80, and y dimension lies between 40 and 70. Recall that each internal node splits space on one dimension, and as in a B^+-

tree. Range search can be performed by the following recursive procedure, starting at the root:

1. Suppose the node is an internal node, and let it be split on a particular dimension, say x, at a point x_i. Entries in the left subtree have x values $< x_i$, and those in the right subtree have x values $\geq x_i$. If the query range contains x_i, search is recursively performed on both children. If the query range is to the left of x_i, search is recursively performed only on the left child, and otherwise it is performed only on the right subtree.
2. If the node is a leaf, all entries that are contained in the query range are retrieved.

Nearest neighbor search is more complicated, and we shall not describe it here, but nearest neighbor queries can also be answered quite efficiently using k-d trees.

The **k-d-B tree** extends the k-d tree to allow multiple child nodes for each internal node, just as a B-tree extends a binary tree, to reduce the height of the tree. k-d-B trees are better suited for secondary storage than k-d trees. Range search as outlined above can be easily extended to k-d-B trees, and nearest neighbor queries too can be answered quite efficiently using k-d-B trees.

There are a number of alternative index structures for spatial data. Instead of dividing the data one dimension at a time, **quadtrees** divide up a two-dimensional space into four quadrants at each node of the tree. Details may be found in Section 24.4.1.

Indexing of regions of space, such as line segments, rectangles, and other polygons, presents new problems. There are extensions of k-d trees and quadtrees for this task. A key idea is that if a line segment or polygon crosses a partitioning line, it is split along the partitioning line and represented in each of the subtrees in which its pieces occur. Multiple occurrences of a line segment or polygon can result in inefficiencies in storage, as well as inefficiencies in querying.

A storage structure called an **R-tree** is useful for indexing of objects spanning regions of space, such as line segments, rectangles, and other polygons, in addition to points. An R-tree is a balanced tree structure with the indexed objects stored in leaf nodes, much like a B^+-tree. However, instead of a range of values, a rectangular **bounding box** is associated with each tree node. The bounding box of a leaf node is the smallest rectangle parallel to the axes that contains all objects stored in the leaf node. The bounding box of internal nodes is, similarly, the smallest rectangle parallel to the axes that contains the bounding boxes of its child nodes. The bounding box of an object (such as a polygon) is defined, similarly, as the smallest rectangle parallel to the axes that contains the object.

Each internal node stores the bounding boxes of the child nodes along with the pointers to the child nodes. Each leaf node stores the indexed objects.

Figure 14.30 shows an example of a set of rectangles (drawn with a solid line) and the bounding boxes (drawn with a dashed line) of the nodes of an R-tree for the set of rectangles. Note that the bounding boxes are shown with extra space inside them, to make them stand out pictorially. In reality, the boxes would be smaller and fit tightly

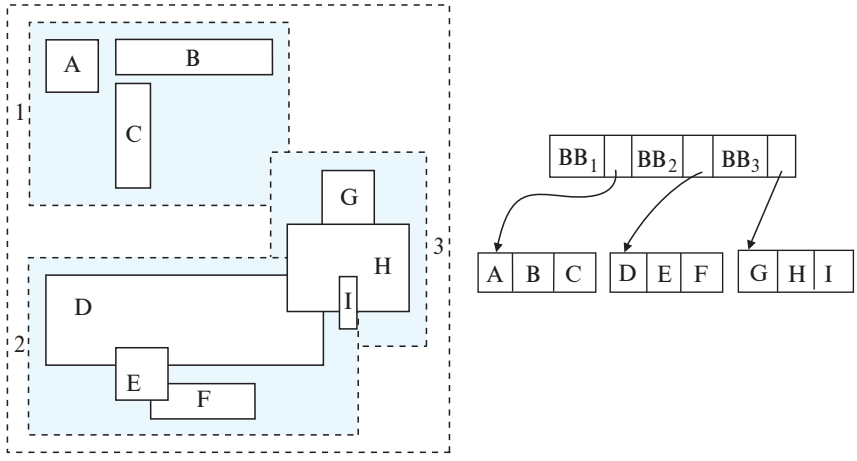


Figure 14.30 An R-tree.

on the objects that they contain; that is, each side of a bounding box B would touch at least one of the objects or bounding boxes that are contained in B.

The R-tree itself is at the right side of Figure 14.30. The figure refers to the coordinates of bounding box i as BB_i in the figure. More details about R-trees, including details of how to answer range queries using R-trees, may be found in Section 24.4.2.

Unlike some alternative structures for storing polygons and line segments, such as R*-trees and interval trees, R-trees store only one copy of each object, and we can ensure easily that each node is at least half full. However, querying may be slower than with some of the alternatives, since multiple paths have to be searched. However, because of their better storage efficiency and their similarity to B-trees, R-trees and their variants have proved popular in database systems that support spatial data.

14.10.2 Indexing Temporal Data

Temporal data refers to data that has an associated time period, as discussed in Section 7.10. The time period associated with a tuple indicates the period of time for which the tuple is valid. For example, a particular course identifier may have its title changed at some point of time. Thus, a course identifier is associated with a title for a given time interval, after which the same course identifier is associated with a different title. This can be modeled by having two or more tuples in the *course* relation with the same *course_id*, but different *title* values, each with its own valid time interval.

A **time interval** has a start time and an end time. Further a time interval indicates whether the interval starts at the start time, or just after the start time, that is, whether the interval is **closed** or **open** at the start time. Similarly, the time interval indicates whether it is closed or open at the end time. To represent the fact that a tuple is valid currently, until it is next updated, the end time is conceptually set to infinity (which can be represented by a suitably large time, such as midnight of 9999-12-31).

In general, the valid period for a particular fact may not consist of just one time interval; for example, a student may be registered in a university one academic year, take a leave of absence for the next year, and register again the following year. The valid period for the student's registration at the university is clearly not a single time interval. However, any valid period can be represented by multiple intervals; thus, a tuple with any valid period can be represented by multiple tuples, each of which has a valid period that is a single time interval. We shall therefore only consider time intervals when modeling temporal data.

Suppose we wish to retrieve the value of a tuple, given a value v for an attribute a, and a point in time t_1. We can create an index on the a, and use it to retrieve all tuples with value v for attribute a. While such an index may be adequate if the number of time intervals for that search-key value is small, in general the index may retrieve a number of tuples whose time intervals do not include the time point t_1.

A better solution is to use a spatial index such as an R-tree, with the indexed tuple treated as having two dimensions, one being the indexed attribute a, and the other being the time dimension. In this case, the tuple forms a line segment, with value v for dimension a, and the valid time interval of the tuple as interval in the time dimension.

One issue that complicates the use of a spatial index such as an R-tree is that the end time interval may be infinity (perhaps represented by a very large value), whereas spatial indices typically assume that bounding boxes are finite, and may have poor performance if bounding boxes are very large. This problem can be dealt with as follows:

- All current tuples (i.e., those with end time as infinity, which is perhaps represented by a large time value) are stored in a separate index from those tuples that have a non-infinite end time. The index on current tuples can be a B^+-tree index on (a, *start_time*), where a is the indexed attribute and *start_time* is the start time, while the index for non-current tuples would be a spatial index such as an R-tree.
- Lookups for a key value v at a point in time t_i would need to search on both indices; the search on the current-tuple index would be for tuples with $a = v$, and *start_ts* $\leq t_i$, which can be done by a simple range query. Queries with a time range can be handled similarly.

Instead of using spatial indices that are designed for multidimensional data, one can use specialized indices, such as the interval B^+-tree, that are designed to index intervals in a single dimension, and provide better complexity guarantees than R-tree indices. However, most database implementations find it simpler to use R-tree indices instead of implementing yet another type of index for time intervals.

Recall that with temporal data, more than one tuple may have the same value for a primary key, as long as the tuples with the same primary-key value have non-overlapping time intervals. Temporal indices on the primary key attribute can be used to efficiently determine if the temporal primary key constraint is violated when a new tuple is inserted or the valid time interval of an existing tuple is updated.

14.11 Summary

- Many queries reference only a small proportion of the records in a file. To reduce the overhead in searching for these records, we can construct *indices* for the files that store the database.
- There are two types of indices that we can use: dense indices and sparse indices. Dense indices contain entries for every search-key value, whereas sparse indices contain entries only for some search-key values.
- If the sort order of a search key matches the sort order of a relation, an index on the search key is called a *clustering index*. The other indices are called *nonclustering* or *secondary indices*. Secondary indices improve the performance of queries that use search keys other than the search key of the clustering index. However, they impose an overhead on modification of the database.
- Index-sequential files are one of the oldest index schemes used in database systems. To permit fast retrieval of records in search-key order, records are stored sequentially, and out-of-order records are chained together. To allow fast random access, we use an index structure.
- The primary disadvantage of the index-sequential file organization is that performance degrades as the file grows. To overcome this deficiency, we can use a B^+*-tree index*.
- A B^+-tree index takes the form of a *balanced* tree, in which every path from the root of the tree to a leaf of the tree is of the same length. The height of a B^+-tree is proportional to the logarithm to the base N of the number of records in the relation, where each nonleaf node stores N pointers; the value of N is often around 50 or 100. B^+-trees are much shorter than other balanced binary-tree structures such as AVL trees, and therefore require fewer disk accesses to locate records.
- Lookup on B^+-trees is straightforward and efficient. Insertion and deletion, however, are somewhat more complicated, but still efficient. The number of operations required for lookup, insertion, and deletion on B^+-trees is proportional to the logarithm to the base N of the number of records in the relation, where each nonleaf node stores N pointers.
- We can use B^+-trees for indexing a file containing records, as well as to organize records into a file.
- B-tree indices are similar to B^+-tree indices. The primary advantage of a B-tree is that the B-tree eliminates the redundant storage of search-key values. The major disadvantages are overall complexity and reduced fanout for a given node size. System designers almost universally prefer B^+-tree indices over B-tree indices in practice.

- Hashing is a widely used technique for building indices in main memory as well as in disk-based systems.
- Ordered indices such as B^+-trees can be used for selections based on equality conditions involving single attributes. When multiple attributes are involved in a selection condition, we can intersect record identifiers retrieved from multiple indices.
- The basic B^+-tree structure is not ideal for applications that need to support a very large number of random writes/inserts per second. Several alternative index structures have been proposed to handle workloads with a high write/insert rate, including the log-structured merge tree and the buffer tree.
- Bitmap indices provide a very compact representation for indexing attributes with very few distinct values. Intersection operations are extremely fast on bitmaps, making them ideal for supporting queries on multiple attributes.
- R-trees are a multidimensional extension of B-trees; with variants such as R^+-trees and R^*-trees, they have proved popular in spatial databases. Index structures that partition space in a regular fashion, such as quadtrees, help in processing spatial join queries.
- There are a number of techniques for indexing temporal data, including the use of spatial index and the interval B^+-tree specialized index.

Review Terms

- Index type
  - Ordered indices
  - Hash indices
- Evaluation factors
  - Access types
  - Access time
  - Insertion time
  - Deletion time
  - Space overhead
- Search key
- Ordered indices
  - Ordered index
  - Clustering index
  - Primary indices;
  - Nonclustering indices
  - Secondary indices
  - Index-sequential files
- Index entry
- Index record
- Dense index
- Sparse index
- Multilevel indices
- Nonunique search key
- Composite search key
- B^+-tree index files
  - Balanced tree
  - Leaf nodes

- Nonleaf nodes
- Internal nodes
- Range queries
- Node split
- Node coalesce
- Redistribute of pointers
- Uniquifier

- B^+-tree extensions
  - Prefix compression
  - Bulk loading
  - Bottom-up B^+-tree construction
- B-tree indices
- Hash file organization
  - Hash function
  - Bucket
  - Overflow chaining
  - Closed addressing
  - Closed hashing
  - Bucket overflow
  - Skew
  - Static hashing
  - Dynamic hashing
- Multiple-key access
- Covering indices
- Write-optimized index structure
  - Log-structured merge (LSM) tree
  - Stepped-merge index
  - Buffer tree
- Bitmap index
- Bitmap intersection
- Indexing of spatial data
  - Range queries
  - Nearest neighbor queries
  - k-d tree
  - k-d-B tree
  - Quadtrees
  - R-tree
  - Bounding box
- Temporal indices
- Time interval
- Closed interval
- Open interval

Practice Exercises

14.1 Indices speed query processing, but it is usually a bad idea to create indices on every attribute, and every combination of attributes, that are potential search keys. Explain why.

14.2 Is it possible in general to have two clustering indices on the same relation for different search keys? Explain your answer.

14.3 Construct a B^+-tree for the following set of key values:

$$(2, 3, 5, 7, 11, 17, 19, 23, 29, 31)$$

Assume that the tree is initially empty and values are added in ascending order. Construct B^+-trees for the cases where the number of pointers that will fit in one node is as follows:

a. Four

b. Six

c. Eight

14.4 For each B^+-tree of Exercise 14.3, show the form of the tree after each of the following series of operations:

a. Insert 9.

b. Insert 10.

c. Insert 8.

d. Delete 23.

e. Delete 19.

14.5 Consider the modified redistribution scheme for B^+-trees described on page 651. What is the expected height of the tree as a function of n?

14.6 Give pseudocode for a B^+-tree function findRangeIterator(), which is like the function findRange(), except that it returns an iterator object, as described in Section 14.3.2. Also give pseudocode for the iterator class, including the variables in the iterator object, and the next() method.

14.7 What would the occupancy of each leaf node of a B^+-tree be if index entries were inserted in sorted order? Explain why.

14.8 Suppose you have a relation r with n_r tuples on which a secondary B^+-tree is to be constructed.

a. Give a formula for the cost of building the B^+-tree index by inserting one record at a time. Assume each block will hold an average of f entries and that all levels of the tree above the leaf are in memory.

b. Assuming a random disk access takes 10 milliseconds, what is the cost of index construction on a relation with 10 million records?

c. Write pseudocode for bottom-up construction of a B^+-tree, which was outlined in Section 14.4.4. You can assume that a function to efficiently sort a large file is available.

14.9 The leaf nodes of a B^+-tree file organization may lose sequentiality after a sequence of inserts.

a. Explain why sequentiality may be lost.

b. To minimize the number of seeks in a sequential scan, many databases allocate leaf pages in extents of n blocks, for some reasonably large n. When the first leaf of a B^+-tree is allocated, only one block of an n-block unit is used, and the remaining pages are free. If a page splits, and its n-block unit has a free page, that space is used for the new page. If the n-block unit is full, another n-block unit is allocated, and the first $n/2$ leaf pages are placed in one n-block unit and the remaining one in the second n-block unit. For simplicity, assume that there are no delete operations.

i. What is the worst-case occupancy of allocated space, assuming no delete operations, after the first n-block unit is full?

ii. Is it possible that leaf nodes allocated to an n-node block unit are not consecutive, that is, is it possible that two leaf nodes are allocated to one n-node block, but another leaf node in between the two is allocated to a different n-node block?

iii. Under the reasonable assumption that buffer space is sufficient to store an n-page block, how many seeks would be required for a leaf-level scan of the B^+-tree, in the worst case? Compare this number with the worst case if leaf pages are allocated a block at a time.

iv. The technique of redistributing values to siblings to improve space utilization is likely to be more efficient when used with the preceding allocation scheme for leaf blocks. Explain why.

14.10 Suppose you are given a database schema and some queries that are executed frequently. How would you use the above information to decide what indices to create?

14.11 In write-optimized trees such as the LSM tree or the stepped-merge index, entries in one level are merged into the next level only when the level is full. Suggest how this policy can be changed to improve read performance during periods when there are many reads but no updates.

14.12 What trade offs do buffer trees pose as compared to LSM trees?

14.13 Consider the *instructor* relation shown in Figure 14.1.

a. Construct a bitmap index on the attribute *salary*, dividing *salary* values into four ranges: below 50,000, 50,000 to below 60,000, 60,000 to below 70,000, and 70,000 and above.

b. Consider a query that requests all instructors in the Finance department with salary of 80,000 or more. Outline the steps in answering the query, and show the final and intermediate bitmaps constructed to answer the query.

14.14 Suppose you have a relation containing the x, y coordinates and names of restaurants. Suppose also that the only queries that will be asked are of the

following form: The query specifies a point and asks if there is a restaurant exactly at that point. Which type of index would be preferable, R-tree or B-tree? Why?

14.15 Suppose you have a spatial database that supports region queries with circular regions, but not nearest-neighbor queries. Describe an algorithm to find the nearest neighbor by making use of multiple region queries.

Exercises

14.16 When is it preferable to use a dense index rather than a sparse index? Explain your answer.

14.17 What is the difference between a clustering index and a secondary index?

14.18 For each B^+-tree of Exercise 14.3, show the steps involved in the following queries:

a. Find records with a search-key value of 11.

b. Find records with a search-key value between 7 and 17, inclusive.

14.19 The solution presented in Section 14.3.5 to deal with nonunique search keys added an extra attribute to the search key. What effect could this change have on the height of the B^+-tree?

14.20 Suppose there is a relation $r(A, B, C)$, with a B^+-tree index with search key (A, B).

a. What is the worst-case cost of finding records satisfying $10 < A < 50$ using this index, in terms of the number of records retrieved n_1 and the height h of the tree?

b. What is the worst-case cost of finding records satisfying $10 < A < 50 \wedge 5 < B < 10$ using this index, in terms of the number of records n_2 that satisfy this selection, as well as n_1 and h defined above?

c. Under what conditions on n_1 and n_2 would the index be an efficient way of finding records satisfying $10 < A < 50 \wedge 5 < B < 10$?

14.21 Suppose you have to create a B^+-tree index on a large number of names, where the maximum size of a name may be quite large (say 40 characters) and the average name is itself large (say 10 characters). Explain how prefix compression can be used to maximize the average fanout of nonleaf nodes.

14.22 Suppose a relation is stored in a B^+-tree file organization. Suppose secondary indices store record identifiers that are pointers to records on disk.

a. What would be the effect on the secondary indices if a node split happened in the file organization?

b. What would be the cost of updating all affected records in a secondary index?

c. How does using the search key of the file organization as a logical record identifier solve this problem?

d. What is the extra cost due to the use of such logical record identifiers?

14.23 What trade-offs do write-optimized indices pose as compared to B^+-tree indices?

14.24 An *existence bitmap* has a bit for each record position, with the bit set to 1 if the record exists, and 0 if there is no record at that position (for example, if the record were deleted). Show how to compute the existence bitmap from other bitmaps. Make sure that your technique works even in the presence of null values by using a bitmap for the value *null*.

14.25 Spatial indices that can index spatial intervals can conceptually be used to index temporal data by treating valid time as a time interval. What is the problem with doing so, and how is the problem solved?

14.26 Some attributes of relations may contain sensitive data, and may be required to be stored in an encrypted fashion. How does data encryption affect index schemes? In particular, how might it affect schemes that attempt to store data in sorted order?

Further Reading

B-tree indices were first introduced in [Bayer and McCreight (1972)] and [Bayer (1972)]. B^+-trees are discussed in [Comer (1979)],[Bayer and Unterauer (1977)], and [Knuth (1973)]. [Gray and Reuter (1993)] provide a good description of issues in the implementation of B^+-trees.

The log-structured merge (LSM) tree is presented in [O'Neil et al. (1996)], while the stepped merge tree is presented in [Jagadish et al. (1997)]. The buffer tree is presented in [Arge (2003)]. [Vitter (2001)] provides an extensive survey of external-memory data structures and algorithms.

Bitmap indices are described in [O'Neil and Quass (1997)]. They were first introduced in the IBM Model 204 file manager on the AS 400 platform. They provide very large speedups on certain types of queries and are today implemented on most database systems.

[Samet (2006)] and [Shekhar and Chawla (2003)] provide textbook coverage of spatial data structures and spatial databases. [Bentley (1975)] describes the k-d tree,

and [Robinson (1981)] describes the k-d-B tree. The R-tree was originally presented in [Guttman (1984)].

Bibliography

[Arge (2003)] L. Arge, "The Buffer Tree: A Technique for Designing Batched External Data Structures", *Algorithmica*, Volume 37, Number 1 (2003), pages 1–24.

[Bayer (1972)] R. Bayer, "Symmetric Binary B-trees: Data Structure and Maintenance Algorithms", *Acta Informatica*, Volume 1, Number 4 (1972), pages 290–306.

[Bayer and McCreight (1972)] R. Bayer and E. M. McCreight, "Organization and Maintenance of Large Ordered Indices", *Acta Informatica*, Volume 1, Number 3 (1972), pages 173–189.

[Bayer and Unterauer (1977)] R. Bayer and K. Unterauer, "Prefix B-trees", *ACM Transactions on Database Systems*, Volume 2, Number 1 (1977), pages 11–26.

[Bentley (1975)] J. L. Bentley, "Multidimensional Binary Search Trees Used for Associative Searching", *Communications of the ACM*, Volume 18, Number 9 (1975), pages 509–517.

[Comer (1979)] D. Comer, "The Ubiquitous B-tree", *ACM Computing Surveys*, Volume 11, Number 2 (1979), pages 121–137.

[Gray and Reuter (1993)] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann (1993).

[Guttman (1984)] A. Guttman, "R-Trees: A Dynamic Index Structure for Spatial Searching", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1984), pages 47–57.

[Jagadish et al. (1997)] H. V. Jagadish, P. P. S. Narayan, S. Seshadri, S. Sudarshan, and R. Kanneganti, "Incremental Organization for Data Recording and Warehousing", In *Proceedings of the 23rd International Conference on Very Large Data Bases*, VLDB '97 (1997), pages 16–25.

[Knuth (1973)] D. E. Knuth, *The Art of Computer Programming, Volume 3*, Addison Wesley, Sorting and Searching (1973).

[O'Neil and Quass (1997)] P. O'Neil and D. Quass, "Improved Query Performance with Variant Indexes", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1997), pages 38–49.

[O'Neil et al. (1996)] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The Log-structured Merge-tree (LSM-tree)", *Acta Inf.*, Volume 33, Number 4 (1996), pages 351–385.

[Robinson (1981)] J. Robinson, "The k-d-B Tree: A Search Structure for Large Multidimensional Indexes", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1981), pages 10–18.

[Samet (2006)] H. Samet, *Foundations of Multidimensional and Metric Data Structures*, Morgan Kaufmann (2006).

[Shekhar and Chawla (2003)] S. Shekhar and S. Chawla, *Spatial Databases: A TOUR*, Pearson (2003).

[Vitter (2001)] J. S. Vitter, "External Memory Algorithms and Data Structures: Dealing with Massive Data", *ACM Computing Surveys*, Volume 33, (2001), pages 209–271.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.